

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 14.09.2026, 11:30:06
Файлов исходного кода: 96
Суммарный объём кода: 1.41 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts
25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/ColoringViz.tsx
37. src/components/ComplexityClassesViz.tsx
38. src/components/ComponentsViz.tsx
39. src/components/DfsBridgesSimulator.tsx
40. src/components/DijkstraViz.tsx
41. src/components/DPVisualizer.tsx
42. src/components/EulerSimulator.tsx
43. src/components/ExpressQuiz.tsx
44. src/components/FloydViz.tsx
45. src/components/GraphTraversalViz.tsx
46. src/components/HeapViz.tsx
47. src/components/hints/demoKit.tsx
48. src/components/hints/demosExtra.tsx
49. src/components/hints/demosGraph.tsx
50. src/components/hints/demosStruct.tsx
51. src/components/hints/MiniDemo.tsx
52. src/components/hints/SimulatorModal.tsx
53. src/components/hints/TermPopover.tsx
54. src/components/JohnsonViz.tsx
55. src/components/KosarajuViz.tsx
56. src/components/KruskalSimulator.tsx
57. src/components/MnemonicCards.tsx

```
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
91. scripts/selftest/bundle.ts
92. scripts/selftest/harness-dump.ts
93. scripts/selftest/harness-entry.ts
94. scripts/selftest/lazy.ts
```

CI / автоматизация

```
95. .github/workflows/deploy-pages.yml
96. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/96: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

## ` .github/workflows/deploy-pages.yml`

```
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
```


- name: Setup Node.js
uses: actions/setup-node@v7
with:
 node-version: "24"
 cache: npm
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends
 # На Ubuntu политика ImageMagick иногда запрещает экспорт в pdf
 # разрешаем то, что нужно пайплайну (text -> pdf)
 for policy in /etc/ImageMagick-6/policy.xml /usr/lib*/ImageMagick-6/policy.xml; do
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{PDF,PSD\}" rights="write,delete" /rights="write,delete" /g'
 fi
 done
 convert -version
- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${GITHUB_EVENT_INPUTS.density:-300}
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true

```
        fi
        git commit -m "chore(export): пересборка full"
        git push origin "HEAD:${GITHUB_REF_NAME}"
    ....

### Patch относительно `main`

````diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
 - name: Checkout
 uses: actions/checkout@v4
 + uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v4
 + uses: actions/setup-node@v7
 with:
 node-version: "22"
 + node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
 } >> "$GITHUB_STEP_SUMMARY"

 - name: Upload artifacts (PDF + TXT)
 uses: actions/upload-artifact@v4
 + uses: actions/upload-artifact@v7
 with:
```

## Универсальное пособие • полный исходный код

```

--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width,
+ <link rel="icon" type="image/svg+xml" href="%BASE_
 <title> Дискретка для тупых – Интерактивный гид по
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

```
`public/favicon.svg`
```

```
Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
    <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
    <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
    <circle cx="47" cy="21" r="4" fill="#34d399"/>  
</svg>  
````
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+    <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+    <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+    <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>
```



```

        </main>
      ) : (
        <PythonCompiler />
      )}

      <SimulatorModal vizId={modalVizId} onClose={() =>

      <footer className="p-8 text-center text-slate-600"
        © 2026 Universal Educational Guide. Интерактивны
      </footer>
    </div>
  );
}
....

```

Patch относительно `main`

```

````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
 const [activeChapterId, setActiveChapterId] = useState<string>("1");
 const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
</>

```

```

 } finally {
 setBusy(false);
 }
 };

 return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center gap-2 text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 </div>
 </header>
);
}

```

```
 href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
  -import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  +import { BookOpen, Cpu, Sparkles, FileDown, FileText,
```

Универсальное пособие • полный исходный код

```
-----  
-             href="/export/full_code_all_pages.txt"  
+             href={` ${import.meta.env.BASE_URL}export/full_code_all_pages.txt`  
             download="full_code_all_pages.txt"  
             target="_blank"  
             rel="noreferrer"  
.....
```

```
## `src/components/PythonCompiler.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx  
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;
let module: PyodideModule | null = null;
```

```

runtime.setStdout({ batched: (message) => stdout.write(message) });
runtime.setStderr({ batched: (message) => stderr.write(message) });
runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
await runtime.runPythonAsync(code);

const text = [...stdout, ...stderr].join("");
setOutput(text || "Готово: программа ничего не вывела");
} catch (error) {
const captured = [...stdout, ...stderr].join("");
setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
setRuntimeMessage("Не удалось выполнить код – подробности в консоли");
} finally {
setRunning(false);
}
}, [code, stdin, running, runtimeReady]);

const reset = () => {
setCode(STARTER_CODE);
setStdin("");
setOutput("Нажмите «Запустить», чтобы увидеть результат");
};

return (
<main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
<div className="max-w-6xl mx-auto">
<div className="flex flex-col sm:flex-row sm:items-center">
<div>
<div className="inline-flex items-center gap-2">
<Terminal className="w-4 h-4" /> Боковая панель
</div>
<h2 className="text-2xl sm:text-3xl font-extrabold">
<p className="text-slate-400 mt-2 max-w-2xl">
Пишите небольшой код и запускайте его прямо в браузере,
который переводит CPython в WebAssembly.
</p>
</div>


```

```

<textarea
 value={code}
 onChange={({event}) => setCode(event.target.value)}
 onKeyDown={({event}) => {
 if ((event.ctrlKey || event.metaKey) && event.key === 'Enter')
 event.preventDefault();
 void runCode();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-y bg-transparent focus:ring-2 focus:ring-inset focus:ring-emerald-500"
 placeholder="Напишите Python-код..."
/>

<div className="border-t border-slate-800 border-b border-slate-800" style={{padding: 10px}}>
 <label className="block px-4 pt-3 text-slate-700" style={{display: block}}>
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={({event}) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например: \nАлиса'}
 className="block w-full resize-y bg-transparent border border-slate-800"
 style={{border: 1px solid #444, padding: 5px}}
 />
</div>
</section>

<section className="rounded-2xl border border-slate-800" style={{padding: 10px}}>
 <div className="flex items-center justify-between" style={{padding: 5px 0}}>
 <div className="flex items-center gap-2" style={{flex: 1}}>
 <Terminal className="w-4 h-4 text-emerald-500" style={{font-family: monospace, font-size: 0.8em}}>

```

```

new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>;
+ setStdout: (options: { batched: (message: string) => void }) => void;
+ setStderr: (options: { batched: (message: string) => void }) => void;
+ setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
+ (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL })
+ });
+ }
+}

```



```

+ setOutput(text || "Готово: программа ничего не в
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("");
+ setOutput(`${captured}${captured && !captured.en
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:p
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5
+ >
+ Как это работает <ExternalLink className="
+

```

```

+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-l
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emera
+ </div>
+ <span className={`inline-flex items-centr
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px]

```

```
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
`);
```

### Patch относительно `main`

````diff

diff --git a/vite.config.ts b/vite.config.ts

index c0456e7..e19d945 100644

--- a/vite.config.ts

+++ b/vite.config.ts

Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).
При запросе пользователя создать или сгенерировать новую страницу:
строго придерживаться готовых шаблонов разметки, не добавляя ничего нового.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть в формате JSON/TS (например, `{ "id": "...", "content": "...", "type": "..." }`) для вставки в ``src/data/content.ts``.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
 - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный).
 - "Душные" детали (код, формулы, сложные доказательства).
- **Не ломай верстку:**** Не придумывай свои CSS-классы. Используй только те, что есть в файлах ``src/data/content.ts``. Не используй чистый markdown.

ФАЙЛ 4/96: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
    <title> Универсальное пособие по алгоритмам – билет №4
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

```
    "selftest": "npx esbuild scripts/selftest/bundle.ts
      el=error --loader:.jpg=file && node /tmp/bundle-s
      at=cjs --outfile=/tmp/lazy-selftest.cjs --jsx=auto
      esbuild scripts/selftest/harness-entry.ts --bundl
      on3 scripts/selftest/harness_check.py"
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
    "svg-to-pdfkit": "^0.1.8",
    "tailwind-merge": "3.4.0"
  },
  "devDependencies": {
    "@tailwindcss/vite": "4.1.17",
    "@types/node": "22.19.17",
    "@types/pdfkit": "^0.17.6",
    "@types/react": "19.2.7",
    "@types/react-dom": "19.2.3",
    "@vitejs/plugin-react": "5.1.1",
    "esbuild": "^0.24.0",
    "tailwindcss": "4.1.17",
    "typescript": "5.9.3",
    "vite": "7.3.2",
    "vite-plugin-singlefile": "2.3.0"
  },
  "description": "Интерактивное пособие по алгоритмам и
}
```

Автоматизация (CI)

`.github/workflows/rebuild-pdf.yml` запускается **на ка...
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док...
и коммитит обновлённые `public/export/full\_code\_all\_page...`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full_co...

Аудит контента

```
```bash
node scripts/audit-coverage.mjs # какие темы без
node scripts/audit-coverage.mjs --md # markdown-табли
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS_AUDIT.md)

Как устроен интерфейс

- * **Темы 1–24** — канонический список билетов; одна стр...
(`src/data/content.ts`, таблица `CANON` задаёт точные
- * **Python-компилятор** — боковая панель (кнопка «Python...
отладчик. Список «переменных темы» для каждой страниц...
`src/data/vizSync.ts`; значения из выполняемого Python...
объекты демонстрации (матрицы, графы, деревья) напрям...
эталонный код, инициализация подставляет заранее зада...
- * **Содержание** — на десктопе боковая панель с поиском...
на телефоне — липкая строка «Содержание · N из M» и в...
- * **Переходы между темами** — кнопки «Назад / Вперёд» в...
«Предыдущая / Следующая тема» под текстом; на клавиату...
- * **Всплывающие подсказки по терминам** — текст главы а...
известные термины (BFS, куча, бор, суффиксная ссылка,...
подчёркиваются, а по наведению/тапу показывается карт...
мини-анимацией и кнопкой «Показать симулятор» (симуля...
Словарь — `src/data/glossary.ts`, мини-анимации — `sr...
- * **Реестр интерактивов** — `src/components/vizRegistry

Универсальное пособие • полный исходный код

Вместо обычных `` в приложении используются векторы

* Они помещены внутрь оберток с классом `

* Все отрисовки находятся внутри векторного тега ``

* Подпись к картинке, если она есть, идет сразу после SVG

Списки

* Стандартные списки оформлены через `

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `

` и `` заменяются на `##` и `###`

2. Элементы ``.analogy-badge` получают выделение маркера`

3. Теги `` заменяются строкой-заглушкой `[ Иллюстрация]`.

4. Элементы списка `- ` получают маркер `-`.

5. Результат извлекается в виде текста и чистится от лишнего

ФАЙЛ 8/96: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

> * **Аналогия** — в HTML главы есть блок «Аналогия ...»

> * **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |

| --- | --- | --- |

Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (

> Замкнуто: «5. Двумерная матрица префиксных сумм» полн

| Глава | id | Чего не хватает |
|--------------------------------------|--------------------|---------------------|
| 3. Декартово дерево (Trear) | `trear` | Нет ни визуализации |
| 8. Графы. Компоненты связности | `graph-components` | |
| 24. Классы сложности, сведение задач | `complexity-cl | |

Сводная таблица по всем главам

| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|----|---|----------|---------|-----|----------|
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | | | | |
| 3 | 3. Декартово дерево (Trear) | 2 | — | — | — |
| 4 | 4. Splay-дерево | 2 | — | — | |
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | — |
| — | Планарность: K5, K3,3 и формула Эйлера | — | | | |
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | — | — |
| 8 | 8. Графы. Компоненты связности | 1 | — | — | — |
| 9 | 9. Графы. Топологическая сортировка | 1 | | | — |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | |
| 12 | 12. Графы. Точки сочленения | 1 | | — | — |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | — |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | |
| 16 | 16. Графы. Кратчайший путь. Флойд | — | | | — |
| 17 | 17. Графы. Алгоритм Джонсона | — | | — | — |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | | | — |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | | | — |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | | | — |
| 21 | 21. Префикс-функция (π), КМП | 4 | | — | — |


```
"isolatedModules": true,
"noEmit": true,
"jsx": "react-jsx",

/* Path mapping */
"baseUrl": ".",
"paths": {
  "@/*": ["src/*"]
},

/* Linting */
"strict": true,
"noUnusedLocals": true,
"noUnusedParameters": true,
"noFallthroughCasesInSwitch": true
},
"include": ["src", "vite.config.ts"]
}
```

ФАЙЛ 10/96: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локаль
```

```

-----
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/Simu
import { PythonCompiler } from "../components/PythonComp

export default function App() {
  const [activeChapterId, setActiveChapterId] = useStat
  const [modalVizId, setModalVizId] = useState<string |
  /** Боковая (на мобильном – нижняя) панель компилятора
  const [compilerOpen, setCompilerOpen] = useState(false
  /** Ширина панели компилятора (desktop), растягивается
  const [compilerWidth, setCompilerWidth] = useState<num
    const cached = typeof window !== "undefined" ? Number
    return Number.isFinite(cached) && cached >= 380 ? c
  });
  useEffect(() => {
    window.localStorage.setItem("compilerWidth", String
  }, [compilerWidth]);
  /** Высота терминала: null оставляет адаптивную высот
  const [compilerHeight, setCompilerHeight] = useState<
    const cached = typeof window !== "undefined" ? Number
    return Number.isFinite(cached) && cached >= 260 ? c
  });
  useEffect(() => {
    if (compilerHeight !== null) window.localStorage.se
  }, [compilerHeight]);
  /** Содержание свернуто (по умолчанию сворачиваем сам
  const [sidebarCollapsed, setSidebarCollapsed] = useSt
    const cached = typeof window !== "undefined" ? wind
    if (cached !== null) return cached === "1";
    return typeof window !== "undefined" && window.inne
  });
  const toggleSidebar = useCallback(() => {
    setSidebarCollapsed((c) => {
      window.localStorage.setItem("sidebarCollapsed", c
      return !c;
    });
  });

```

```

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
  const onKey = (e: KeyboardEvent) => {
    const target = e.target as HTMLElement | null;
    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight" && next) goTo(next.id);
    if (e.key === "ArrowLeft" && prev) goTo(prev.id);
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar
      compilerOpen={compilerOpen}
      onToggleCompiler={() => setCompilerOpen((o) => !o)}
    />

    <div ref={contentRef} className={compilerOpen ? "h-screen" : "h-[calc(100vh-50px)]"}>
      <MobileToc
        selectedChapterId={activeChapterId}
        setSelectedChapterId={goTo}
        currentTitle={activeChapter.title}
        index={index}
        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row min-h-[calc(100vh-100px)]">
        <div
          className={`hidden lg:block shrink-0 border-right border-slate-800 p-4 transition-all duration-200 ${sidebarCollapsed ? "hidden" : ""}`}
        >

```

```

        key={activeChapter.id}
        chapter={activeChapter}
        prev={prev}
        next={next}
        index={index}
        total={chapters.length}
        onGo={goTo}
        onOpenSimulator={setModalVizId}
        onOpenCompiler={openCompilerWithViz}
      />
    </main>
  </div>
</>

<footer className="p-8 text-center text-slate-600">
  © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() =>
  setModalVizId(null)}>
  {compilerOpen && (
    <PythonCompiler
      chapterId={activeChapter.id}
      chapterTitle={activeChapter.title}
      width={compilerWidth}
      height={compilerHeight}
      onWidthChange={setCompilerWidth}
      onHeightChange={setCompilerHeight}
      onOpenGuide={() =>
        document.getElementById("chapter-viz")?.scrollTo(0,0)}
      onClose={() => setCompilerOpen(false)}
    />
  )}
</div>
);
}

```

```

*/
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();
  const walker = document.createTreeWalker(root, NodeFilter.
    acceptNode(node) {
      const parent = (node as Text).parentElement;
      if (!parent) return NodeFilter.FILTER_REJECT;
      if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
      if (!node.nodeValue || node.nodeValue.trim().length === 0)
        return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    },
    true);

  const targets: Text[] = [];
  let current = walker.nextNode();
  while (current) {
    targets.push(current as Text);
    current = walker.nextNode();
  }

  for (const textNode of targets) {
    const text = textNode.nodeValue ?? "";
    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
      const surface = m[1].toLowerCase();
      const id = labelToId.get(surface);
      if (!id) continue;

```

```
    } catch {
      /* если браузер не умеет lookbehind – просто оста
    }
  };

  // при смене главы
  // eslint-disable-next-line react-hooks/exhaustive-dep
  useEffect(run, deps);

  // страховка: контент перерисовали, а подсказок в нём
  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    if (!el.querySelector(".term-hint")) run();
  });
}
```

ФАЙЛ 13/96: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
@import "tailwindcss";

@layer utilities {
  .neon-glow-blue {
    box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
  }
  .neon-glow-emerald {
    box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
  }
  .neon-glow-rose {
    box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
  }
  .neon-glow-yellow {
    box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
  }
}
```

```

        transform: translateY(4px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.term-popover {
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {

```

```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```



```
}
```

ФАЙЛ 16/96: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {  
  const src: string;  
  export default src;  
}  
declare module '*.jpg?url' {  
  const src: string;  
  export default src;  
}  
declare module '*.jpeg' {  
  const src: string;  
  export default src;  
}  
declare module '*.png' {  
  const src: string;  
  export default src;  
}
```

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

ФАЙЛ 17/96: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 866 | РАЗМЕР: 10 КБ

```
import { Chapter } from "../types";
```

```

        <p class="text-slate-300 text-sm mb-4">
            нужно покрыть отрезок, мы берем две самые бо
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border
        <summary class="p-4 font-bold text-slate-300
            -b border-slate-700/50">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300
                <pre class="bg-slate-950 p-4 rounded
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
            </div>
        </details>
    </div>

    <div class="bg-slate-700/50 p-6 rounded-xl border
        <h3 class="text-xl font-bold text-amber-400">
        <p class="text-slate-300 text-sm mb-4">Част
            ти за ту же скорость». <b class="text-rose-500">
            и платит она не зря, потому что умеет то, ч
        <div class="grid grid-cols-1 xl:grid-cols-2
            <div class="bg-slate-800 p-6 rounded-lg
                <div class="absolute -top-3 left-4
border border-emerald-500 shadow-md">
                    Префиксная матрица сумм (биле
                </div>
                <ul class="text-slate-300 text-sm sp
                    <li>• Размер: <code class="bg-slate-900/40 p-2
-800">N × M</code> – одна копия матрицы, и
                    <li>• Запрос суммы на прямоуголь
                    <li>• Умеет <b>только сумму</b>
                    <li>• Менять элементы нельзя –
                </ul>
            </div>
        </div>
    </div>

```

```

        летки дважды. Сумма  $\neq$  идемпотентна.</p>
        <p class="text-slate-400 text-sm mb-1">
ации.</p>
        <p class="text-slate-400 text-sm">• Они
го минимума. Выбирай не «какая круче», а «к
</div>
    </div>
</div>
</section>`,
    }, {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приор
category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border
            <h3 class="text-xl font-bold text-blue-400 m
            <div class="bg-slate-900 p-4 rounded-lg mb-1
                <p class="text-lg text-blue-300 italic m
                <p class="text-xl font-mono text-white">
                    двоичная куча. Пара – точка на декартовой
            </div>

        <p class="text-slate-300 text-sm mb-4">Постр
            платит  $\approx n \cdot \log n$  сравнений, counting по ма
            точка вставляется спуском от корня:  $x \leq$  уз
            единственно. Весь процесс – в симуляторе под
            «соединить сам»</b> – кликаешь две точки,
            ь родитель», «получился бы цикл», «равные »,
            «Миф  $2k/2k+1$ »</b> и <b>«Поиск  $\neq$  сортировка»
            ждый запрещённый ход.</p>

```

аются merge за $O(h)$.

Сортировка — `merge sort` (сортировка + n спусков). Худший случай — $O(n^2)$.

и останется набор букв, как treeар без у ост. но кричат по «!»: корень самый громкий, род ; равные x — идут влево по уставу.

Код: split (Скрыто)

```

pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}

```

```

type: "html",
description: "Самонастраивающееся дерево: поднимаем
category: "Продвинутые структуры",
content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ротов поднимает x в корень. Единственный ин
      </div>

      <div class="grid grid-cols-1 md:grid-cols-3">
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">мож
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-emerald-300 font-bold">
          <p class="text-slate-400 text-xs mt">
        </div>
        <div class="bg-slate-900 rounded-lg p-4">
          <p class="text-xs uppercase tracking-wider">
          <p class="text-white font-bold">O(n
          <p class="text-slate-400 text-xs mt">
        </div>
      </div>
    </div>

    <p class="text-slate-300 text-sm mb-4">От к
    ет O(n). AVL и красно-чёрные выбирают <b>пр

```

```

<p class="text-slate-300 text-sm mb-3">Смотрим
<div class="overflow-x-auto -mx-2 px-2">
  <table class="w-full text-xs sm:text-sm" >
    <thead>
      <tr class="text-left text-slate-300">
        <th class="py-2 pr-3 font-bold">Имя
        <th class="py-2 pr-3 font-bold">Возраст
        <th class="py-2 pr-3 font-bold">Пол
        <th class="py-2 font-bold">Родители
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-200">
        <td class="py-3 pr-3 font-bold">Иван
        <td class="py-3 pr-3">деда
        <td class="py-3 pr-3">сам х
        <td class="py-3">х — корень
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-3 pr-3 font-bold">Петр
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span>
        <td class="py-3">ветка укор
      </tr>
      <tr>
        <td class="py-3 pr-3 font-bold">Анна
        <td class="py-3 pr-3">х и п
        <td class="py-3 pr-3"><span>
        <td class="py-3">п и г стал
      </tr>
    </tbody>
  </table>
</div>
<p class="text-slate-300 text-sm mt-3">И по
у. Zig случается только последним.</p>

<div class="bg-rose-950/30 rounded-xl borde
  <p class="text-rose-300 font-bold text-

```

```

        <details class="bg-slate-900/40 rounded-lg"
            <summary class="p-4 font-bold text-slate-300"
                -b border-slate-700/50">
                Код: rotate + splay + find (Скрыт)
            </summary>
            <div class="p-5 text-sm text-slate-300"
                <pre class="bg-slate-950 p-4 rounded"
# узел: { key, left, right, parent }

def rotate_up(x):                                # x встаёт на место
    p = x.parent
    g = p.parent
    if p.left is x:                               # правый поворот
        p.left = x.right
        if x.right: x.right.parent = p
        x.right = p
    else:                                         # левый (зеркально)
        p.right = x.left
        if x.left: x.left.parent = p
        x.left = p
    p.parent = x
    x.parent = g
    if g:
        if g.left is p: g.left = x
        else: g.right = x

def splay(x):                                    # поднять x в корень
    while x.parent:
        p, g = x.parent, x.parent.parent
        if g is None:                            # ZIG
            rotate_up(x)
        elif (g.left is p) == (p.left is x):    # ZIG-ZIG
            rotate_up(p)
            rotate_up(x)
        else:                                     # ZIG-ZAG: НИЖНЯЯ
            rotate_up(x)
            rotate_up(x)

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; margin: 10px 0;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    × Аналогия 1: Вырезание прямоугольника
  </div>
  <p class="text-slate-300 text-sm mb-4">
    Из него маленький целевой прямоугольник в центре.
    Его верхний угол отрезается дважды! Поэтому
  </p>
</div>
</div>
</div>
</section>`,
},
{
  id: "graph-planar-colors",
  title: "7. Графы. Планарные. Покраска",
  type: "html",
  description: "Формула Эйлера и теорема о 4 красках.",
  category: "Графы. Теория",
  content: `
<section id="graph-planar-colors" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7
    <h2 class="text-2xl sm:text-3xl font-bold text-white">7. Графы. Планарные. Покраска
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Покраска графов
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Теорема 7.1 (Теорема о 4 красках)
        <p class="text-xl font-mono text-white">
          Любой планарный граф можно раскрасить 4 цветами.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">

```



```

        еская сортировка — это расписание, которое
        p>
            </div>
            <div class="bg-slate-900 p-6 rounded-lg"
              <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
                Ограничения: Циклы
            </div>
            <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
            </div>
        </div>
    </div>
</div>
</section>`,
    },
    {
      id: "scc-kosaraju",
      title: "10. Графы. Компоненты сильной связности (SCC)",
      type: "html",
      description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
      category: "Графы. Продвинутое",
      content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Ограничения: Циклы
        <p class="text-xl font-mono text-white">Инвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
            ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает связность",
        category: "Графы. Продвинутое",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            12. Графы. Точки сочленения
        </span>
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
            Точки сочленения
        </h2>
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
            <h3 class="text-xl font-bold text-rose-400">
                Точки сочленения
            </h3>
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">
                    Точка сочленения — это вершина, удаление которой
                    увеличивает количество компонент связности графа.
                </p>
                <p class="text-xl font-mono text-white">
                    Например, в графе ниже вершина 1 является
                    точкой сочленения. Если ее удалить, граф распадется
                    на две компоненты: {2, 3, 4} и {5, 6, 7, 8, 9}.
                </p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 right-4 bottom-4
                        border border-emerald-500 shadow-md">
                        Аналогия 1: Центральный роутер
                    </div>
                    <p class="text-slate-300 text-sm mb-6">
                        Если центральный роутер выйдет из строя,
                        связь между двумя сегментами офиса будет
                        потеряна.
                    </p>
                </div>
            </div>
        </div>
    </div>
</section>`
    }
]

```

```

    },
    {
      id: "pathfinding-accel",
      title: "17. Графы. Ускорения поиска",
      type: "html",
      description: "",
      category: "Графы. Пути",
      content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">17
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
      <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Аналогия: Копаем туннель под землей. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
        <p class="text-xl font-mono text-white">д-длина.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Копаем туннель под землей. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ней земли. Но если начать копать одновременно в 2 РАЗА меньше работы (геометрически: площадь
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "mst-kruskal",

```

```

        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            ✖ Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ли вирус). Мы стартуем из одного города, и
            Сеть растет только из одного связного куска
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
      },
      {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
i]), который одновременно является её суффикс
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb-1"
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
      </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg"
      <summary class="p-4 font-bold text-slate-300"
      -b border-slate-700/50">
        Код (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300"
        <pre class="bg-slate-950 p-4 rounded"
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
  int j = pi[i-1];
  while (j > 0 && s[i] != s[j]) j = pi[j-1];
  if (s[i] == s[j]) j++;
  pi[i] = j;
}</pre>
      </div>
    </details>

```



```
export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">Эйлер
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400 m

    <div class="bg-slate-900 p-4 rounded-lg mb-4
      <p class="text-lg text-blue-300 italic m
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounde
          <p class="text-sm text-blue-300
          <p><strong class="text-white">П
          <p><strong class="text-white">Ц
        </div>
      </div>
    </div>
  </div>
  <div class="p-6 bg-slate-800 rounded-xl
    <p class="text-slate-300 text-sm">
      <strong>Билет 13 (Различие пути и цикла
      <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
      <br>• <span class="text-yellow-400 font
финиш).
    </p>
  </div>`
  }
]
```

```
<p>
    Связный граф содержит эйлеров цикл ⇔
    <br>Эйлеров путь (не цикл) ⇔ ровно 0 или 2 вершины с нечётной степенью
</p>
<p class="text-rose-400">Почему Гамильтон?
</p>
    Задача коммивояжёра (TSP) – это взвешенный граф.
    TSP – NP-трудная: полиномиальный алгоритм не известен.
    В общем, не парься. Просто запомни!
</p>
</div>
</details>
</div>
</section>',
},
{
    id: "bridges-code",
    title: "Мосты в графах: код + визуализация",
    type: "html",
    content: `<section id="bridges-code" class="mb-20">
        <div class="flex items-center mb-6">
            <span class="bg-emerald-600 text-white px-4 py-2 rounded">Код
            <h2 class="text-3xl font-bold text-white">Мосты
        </div>

        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-emerald-400">Задача коммивояжёра
            </div>
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">Вспомогательные функции
                <div class="text-left font-mono text-sm">
                    <p><strong class="text-white">tin[v] ← 0
                    <p><strong class="text-white">low[v] ← v
                    <div class="p-3 bg-slate-950 rounded">
                        <span class="text-rose-400 font-weight-normal">Функция
                        <p class="text-xl font-bold text-white">Нахождение минимального
                        <p class="text-xs text-slate-400">Решение задачи коммивояжёра
                    </div>
                </div>
            </div>
        </div>
    </section>`
}
```



```
<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
```

Полный код на C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
  <p>Классическая реализация – <code class="text-sm text-slate-400">
    оходов.</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg border-1">
  <pre class="text-xs font-mono text-slate-400">
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежных
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```
    visited[v] = true;
```

```
    tin[v] = low[v] = timer++; // устанавливаем
```

```
    for (int to : adj[v]) {
```

```
        if (to == p) continue; // не идём назад
```

```
        if (visited[to]) {
```

```
            // обратное ребро: обновляем low
```

```
            low[v] = min(low[v], tin[to]);
```

```
        } else {
```

```
            // ребро дерева DFS
```

```
            dfs(to, v); // рекурсивный вызов
```

```
            low[v] = min(low[v], low[to]);
```

```
            if (low[to] > tin[v]) {
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-blue-300 italic">
  <p class="text-3xl font-mono text-white">
  <div class="text-sm text-slate-400 space-x-2">
    <p><span class="text-blue-400 font-mono">V=5,
    <p><span class="text-emerald-400 font-mono">E=9
    <p><span class="text-rose-400 font-mono">E=10
  </div>
</div>

<p class="text-slate-300 text-sm">
  <strong>Следствие:</strong> Для планарного графа:
  <code>E < 2V - 2</code>.
  Если ребер больше – граф гарантированно непланарен!
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
    <div class="absolute -top-3 left-4 bg-slate-900 text-white border border-emerald-500 shadow-md">
      K5: Полный граф на 5 вершинах
    </div>
    <p class="text-slate-300 text-sm mb-4">
      У K5: <code class="text-white">V=5, E=10</code> – БАХ, непланарен!
      Все 5 вершин соединены со всеми. Напоминает K5.
      Теорема Куратовского: K5 – эталон непланарности.
      Если внутри графа спрятан K5 – забудь про планарность!
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
    <div class="absolute -top-3 left-4 bg-rose-500 text-white border border-rose-500 shadow-md">
      K3,3: 3 дома и 3 колодца
    </div>
    <p class="text-slate-300 text-sm mb-4">
```



```
<div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-slate-800 border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком (удалят вершину) — районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
```

Душный мод: Код и корень DFS (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Особый
```

```
<p>
```

Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для него **больше 1 независимого ребенка**

```
</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
```

```
<pre class="text-xs font-mono text-slate-400">
```

```
.....
        . Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
];
.....
```

ФАЙЛ 19/96: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 129 | РАЗМ

```
.....
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets1to5 } from "../tickets/ticket1_5";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as extraChapters } from "../content_te

/**
 * Канонический порядок тем (билеты 1–24) и «правильные
 * одна страница может раскрывать одну тему или несколь
 * поэтому названия выверяются по единому списку ниже.
 */
const CANON: Record<string, { num: number; title: string }> = {
    "segment-trees": { num: 1, title: "1. Дерево отрезков" },
    "sparse-table": { num: 2, title: "2. Двумерная разреж
    treap: { num: 3, title: "3. Декартово дерево (Treap)" },
    "splay-tree": { num: 4, title: "4. Splay-дерево" },
    "prefix-sums-2d": { num: 5, title: "5. Двумерная матри
    "graph-dfs-bfs": { num: 6, title: "6. Графы. Представ
    "planarity-euler-formula": { num: 7, title: "7. Графы
    "graph-components": { num: 8, title: "8. Графы. Компон
    "top-sort": { num: 9, title: "9. Графы. Топологическа
.....
```

```

-----
// Дубликаты: берём последнюю (самую свежую) версию стр.
const dedup = new Map<string, Chapter>();
merged.forEach((c) => {
  if (c.id === "alg-map") return; // страница-заглушка
  if (stale.has(c.id)) return;
  dedup.set(c.id, c);
});

// Нормализуем названия по каноническому списку тем, а
// по учебным блокам: блоки идут строго в порядке чтения
// номерам билетов. Близкие темы (1–5, 14–17, 18–20, 21–24)
const BLOCKS: [number, number, string][] = [
  [1, 5, "Билеты 1–5 · Структуры и матрицы"],
  [6, 13, "Билеты 6–13 · Графы: обходы и связность"],
  [14, 17, "Билеты 14–17 · Кратчайшие пути"],
  [18, 20, "Билеты 18–20 · Остовные деревья"],
  [21, 23, "Билеты 21–23 · Строки"],
  [24, 24, "Билет 24 · Теория сложности"],
];
const blockOf = (num: number): string => BLOCKS.find(([_, num2]) => num2 === num);

const withCanon = Array.from(dedup.values()).map((c) => {
  const canon = CANON[c.id];
  if (canon) return { ...c, title: canon.title, category: canon.category };
  return { ...c, category: "Введение" }; // страницы вне канона
});

// Сначала вводная страница, затем билеты 1–24 по порядку
const numbered = withCanon.filter((c) => CANON[c.id]);
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);
const extras = withCanon
  .filter((c) => !CANON[c.id])
  .sort((a, b) => a.title.localeCompare(b.title, "ru"));

export const chapters: Chapter[] = [...extras, ...numbered];

/**
 * «Читать рядом»: связанные билеты, которые логично от

```

Универсальное пособие • полный исходный код

```
-----
* как панель Variables в обычном отладчике.
*/

export type DebugValue = null | boolean | number | string;

export interface DebugStep {
  /** Номер строки в коде пользователя (1-based). */
  line: number;
  /** Имя функции-фрейма ("<module>" – верхний уровень) */
  func: string;
  /** Локальные переменные: имя → герг (усечённый), для
  locals: Record<string, string>;
  /**
   * JSON-снимок доступных имён (globals + locals). В о
   * можно напрямую отдать визуализации: i/j остаются ч
   */
  values?: Record<string, DebugValue>;
}

export interface DebugResult {
  steps: DebugStep[];
  /** Локалы в момент возврата из модуля – «результат»
  finalLocals?: Record<string, string>;
  /** Финальный машинно-читаемый снимок для визуализации
  finalValues?: Record<string, DebugValue>;
  /** Текст последней необработанной ошибки (или пустая
  error: string;
  /** true, если уперлись в лимит шагов. */
  truncated: boolean;
}

/** Лимит шагов трассы – защита от бесконечных циклов и
export const DEBUG_STEP_CAP = 1200;

/**
 * Строит самодостаточный python-скрипт: выполняет user
 * и возвращает JSON с шагами (строка + func + locals)
 * последнего выражения (его вернёт runPythonAsync).
```

```

dict_limit = 192 if depth == 0 else limit
    return {str(k): __snapshot(x, depth + 1) for k,
# deque, heap-подобные и другие итерируемые учебные
if type(v).__name__ == "deque":
    return [__snapshot(x, depth + 1) for x in list(v)]
return __safe(v)

def __visible(items):
    return {
        k: v for k, v in items
        if not k.startswith("__") and type(v).__name__
    }

def __capture(frame):
    local_values = __visible(frame.f_locals.items())
    # Внутри функции алгоритма важные структуры (memo,
    # глобальные. Локальные значения имеют приоритет на
    scope_values = __visible(frame.f_globals.items())
    scope_values.update(local_values)
    loc = {k: __safe(v) for k, v in local_values.items()}
    values = {k: __snapshot(v) for k, v in scope_values.items()}
    return loc, values

def __tracer(frame, event, arg):
    global __was_truncated
    if event == "line" and frame.f_code.co_filename ==
        and not (frame.f_code.co_name.startswith("<
    loc, values = __capture(frame)
    # PEP 709: с 3.12 компрехеншны встроены во фреймворк
    # на своей строке каждую итерацию. Склеиваем по
    if __steps and __steps[-1]["line"] == frame.f_lineno:
        __steps[-1]["locals"] = loc
        __steps[-1]["values"] = values
    else:
        __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
    if len(__steps) >= __CAP:
        # Нельзя просто отключить trace: бесконечный цикл
        # тогда продолжит работать вечно. Мягко прерываем

```



```

    prev: Record<string, string> | null | undefined,
    cur: Record<string, string>
  ): Record<string, boolean> {
    const out: Record<string, boolean> = {};
    for (const k of Object.keys(cur)) {
      out[k] = !prev || !(k in prev) || prev[k] !== cur[k];
    }
    return out;
  }

/** Базовое имя из карточки переменной: "tin[v]" → "tin"
export const baseVarName = (name: string): string => name.slice(0, -1);

/**
 * Порядок показа в панели переменных: сначала переменные
 * страницы (те, что подсвечивает визуализация), остальные
 */
export function orderVars(
  cur: Record<string, string>,
  syncNames: string[]
): [string, string][] {
  const inSync = new Set(syncNames);
  const syncEntries = Object.entries(cur).filter(([k]) => inSync.has(k));
  const rest = Object.entries(cur)
    .filter(([k]) => !inSync.has(k))
    .sort(([a], [b]) => a.localeCompare(b));
  return [...syncEntries, ...rest];
}

```

ФАЙЛ 21/96: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМЕР: 1.5 КБ

```

import type { DemoKind } from "../components/hints/MinimalDemo";

```

```

/**

```

```

    labels: ["DFS", "обход в глубину", "поиск в глубину",
    title: "DFS — обход в глубину",
    short:
        "Прёшь вперёд до упора, упёрся — откатываешься на
    formal:
        "Стек (или рекурсия). Из DFS растут: времена вход
    complexity: "O(V + E)",
    demo: "dfs",
    simulator: "stack-dfs",
},
{
    id: "binsect",
    labels: ["binsect", "бинсект", "бинарный поиск", "д
    title: "Бинарный поиск (и его самозванец binsect)",
    short:
        "Бинарный поиск НАХОДИТ готовый элемент в уже отс
        о не сортирует. «binsect» из конспекта — шутлив
        quicksort.",
    formal:
        "Поиск одного элемента в отсортированном массиве -
        O(n log n). Бинарный поиск сортировку не выполн
    complexity: "поиск: O(log n); сортировкой не являетс
},
{
    id: "heap",
    labels: ["куча", "кучу", "кучи", "куче", "кучей", "
    title: "Куча (priority queue)",
    short:
        "Не отсортированный список, а «мешок, из которого
    formal:
        "Полное бинарное дерево в массиве: дети узла i ле
    complexity: "вставка/извлечение O(log n), «посмотре
    demo: "heap",
    simulator: "heap-beam-search",
},
{
    id: "stack",
    labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

```

```
{
  id: "bfs-on-trie",
  labels: [
    "обход дерева",
    "обход бора",
    "обход в ширину по бору",
    "какой-то обход",
    "обходом дерева",
    "конечный автомат",
    "конечного автомата",
    "автомат",
    "автомата",
  ],
  title: "Обход бора в ширину (тот самый «какой-то обход»)",
  short:
    "В Ахо–Корасике бор обходят именно BFS: суффиксную очередь  

    о есть строго по уровням.",
  formal:
    "Кладём в очередь детей корня (их ссылка = корень).  

    Кладём с в очередь.",
  complexity: "O(суммарной длины словаря × алфавит)",
  demo: "trie-bfs",
  simulator: "aho-corasick",
},
{
  id: "suffix-link",
  labels: ["суффиксная ссылка", "суффиксные ссылки"],
  title: "Суффиксная ссылка",
  short:
    "«Куда откатиться, если следующая буква не подошла»",
  formal: "link(v) = go(link(parent(v)), ch). Считает",
  demo: "suffix-link",
  simulator: "aho-corasick",
},
{
  id: "toposort",
  labels: ["топологическая сортировка", "топсорт", "топосорт"]
```

```

    title: "Разреженная таблица (метод двоичных подъёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1, 2, ..., M.",
    formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
    complexity: "препроцессинг O(n log n), запрос O(1)",
    demo: "sparse-table",
    simulator: "sparse-table",
},
{
    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дерево отрезков"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь массив, а листья — за отдельные куски.",
    formal: "Строится за O(n), запрос и обновление — O(log n)",
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается на две части.",
    formal: "Ребро (u,v) — мост, если low[v] > tin[u], где tin[u] — время захода в вершину u, а low[v] — время выхода из вершины v, если ребро в родителя — нет.",
    complexity: "O(V + E) одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "точки сочленения"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: увольь её — и коллапс.",

```

```

    short:
        "NP — «решение проверить легко, найти трудно» (судя по тому, что оно в NP).",
    formal: "Задача NP-полна, если она в NP и к ней полноредукция",
    demo: "np",
},
{
    id: "potential",
    labels: ["потенциал", "потенциалы", "потенциалов"],
    title: "Потенциалы (перевзвешивание Джонсона)",
    short:
        "Трюк «поднять все дороги на одинаковую высоту»:  $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — потенциал, к путей сохраняется.",
    formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$ , где  $h$  — потенциал, к путей сохраняется.",
    demo: "relax",
    simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты сильной связности"],
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждой другой",
    formal: "Косарайю: DFS с сохранением порядка выхода из рекурсии",
    complexity: " $O(V + E)$ ",
    demo: "scc",
    simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функция",
        "префикс", "префикса", "суффиксом", "суффикс", "суффиксы",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начался с этой позиции и повторяется в начале строки",

```

```

    labels: ["кратчайший путь", "кратчайшие пути", "кра
    title: "Кратчайший путь",
    short:
        "Самый дешёвый маршрут из А в В. «Дешёвый» – по с
    formal:
        "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Фор
        ы,  $O(V^3)$ .",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "negative-cycle",
    labels: ["отрицательный цикл", "отрицательного цикла
    title: "Отрицательный цикл",
    short:
        "Круг, пройдя по которому вы становитесь «богаче»
    formal:
        "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "greedy",
    labels: [
        "жадный алгоритм", "жадный", "жадно", "жадная",
        "самое дешёвое ребро", "самое дешёвое ребро", "са
    ],
    title: "Жадный алгоритм",
    short:
        "На каждом шаге хватаем то, что лучше прямо сейчас
    formal:
        "Корректность обычно доказывается «леммой о разре
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова

```

```

        "Матрица смежности – таблица «есть ли ребро», быс
        й, экономно для разреженных графов.",
    formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Спис
},
{
    id: "dijkstra",
    labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
    title: "Алгоритм Дейкстры",
    short:
        "Жадный «навигатор»: всегда раскрываем ближайший
        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
    complexity: " $O(E \log V)$  с бинарной кучей",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
    title: "Алгоритм Форда–Беллмана",
    short:
        "Тупой, но честный:  $V-1$  раз проходим по ВСЕМ рёбра
        ",
    formal:
        "После  $k$ -й итерации гарантированно верны все крат
        ось – есть отрицательный цикл.",
    complexity: " $O(V \cdot E)$ ",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд–Уоршелл: разрешаем вершину  $k$ ",

```

```

    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
    formal:
        "Сортировка  $O(E \log E) + E$  операций union/find. C
    complexity: " $O(E \log E)$ ",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: " $O(V + E)$ ",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        " $G = (V, E)$ . Ребро бывает ориентированным (улица
    demo: "graph",
    simulator: "graph-dfs-bfs",

```



```

{
  id: "inclusion-exclusion",
  labels: [
    "включений-исключений", "включения-исключения", "вырезание прямоугольников",
  ],
  title: "Включения-исключения на префиксных суммах",
  short:
    "Чтобы получить сумму прямоугольника, берём больший отрезали дважды.",
  formal:
    "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
  complexity: "запрос O(1) после O(nm) предподсчёта",
  demo: "prefix-sum",
  simulator: "prefix-sums-2d",
},
{
  id: "reduction",
  labels: ["сведение", "сведения", "сведении", "сведё"],
  title: "Сведение задач",
  short:
    "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
  formal:
    "A ≤p B: есть полиномиальная функция, переводящая решение A в решение B.",
  demo: "np",
},
{
  id: "range",
  labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
  title: "Запрос на отрезке",
  short:
    "Спрашиваем не про один элемент, а про кусок массива. Нужны предподсчёты.",
  formal:
    "Суммы — префиксные суммы O(1). Минимум без изменений.",
  demo: "prefix-sum",
  simulator: "prefix-sums-2d",
}

```

```

{
  id: "zig-zig",
  labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
  title: "Zig-Zig – x и родитель с одной стороны",
  short:
    "Дерево вытянулось в «бамбук»: g → p → x идут в одну сторону, а y – в другую. Именно это вдвое сокращает глубину всего дерева.",
  formal:
    "Если крутить наивно (два раза поднимать x), бамбук превращается в AVL-дерево.",
  demo: "zig-zig",
  simulator: "splay-rotations",
},
{
  id: "zig-zag",
  labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
  title: "Zig-Zag – x и родитель с разных сторон",
  short:
    "Узлы идут «змейкой»: p – левый сын g, а x – правый сын p. И p и g становятся двумя детьми x.",
  formal: "Эквивалентно двойному повороту AVL-дерева.",
  demo: "zig-zag",
  simulator: "splay-rotations",
},
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (чтобы было удобнее читать). */
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  GLOSSARY.map((e) => ({ label: e.labels[0], id: e.id }))
    .sort((a, b) => b.label.length - a.label.length);

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
  <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
  </h3>

  <div class="bg-slate-900 p-4 rounded-lg mb-4">
    <p class="text-lg text-emerald-300 italic">Позитивное планирование — это когда вы знаете, что вы можете сделать, чтобы избежать проблемы.
    <p class="text-xl font-mono text-white">Пример: Если вы знаете, что вы можете избежать проблемы, вы можете избежать проблемы.
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
    <!-- Аналогия 1 -->
    <div class="bg-slate-800 p-6 rounded-lg border-rose-500 shadow-md">
      <div class="absolute -top-3 left-4 right-4 bottom-4">
        Аналогия 1: Позитивное планирование
      </div>
      <p class="text-slate-300 text-sm mb-4">
        (нельзя поехать и заработать). Он всегда в состоянии ожидания (нельзя поехать и заработать). Он всегда в состоянии ожидания.
      <div id="slot-chapter-image-dijkstra" class="h-40 w-100">
      </div>
    </div>

    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
      <div class="absolute -top-3 left-4 right-4 bottom-4">
        Ограничения
      </div>
      <p class="text-slate-300 text-sm mb-4">
        охождение улицы), Дейкстра сломается, так как он не знает, что такое ограничение.
      </div>
    </div>

  </div>

  <details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
    <summary class="p-4 font-bold text-slate-300">
      -b border-slate-700/50">
        Строгое доказательство (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300">
        <p>Алгоритм использует приоритетную очередь, чтобы найти кратчайший путь.

```

```

<div id="slot-chapter-image-bellman"
</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
    Отрицательный цикл
  </div>
  <p class="text-slate-300 text-sm mb-6">
г станет ЕЩЕ короче – значит мы попали во в
  </div>
</div>

<div id="slot-bellman-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-indigo-400">
    </h3>
    <div class="bg-slate-900 p-4 rounded-lg mb-6">
      <p class="text-lg text-indigo-300 italic">
      <p class="text-xl font-mono text-white">
    </div>
  </div>

```

```
</div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border-l
```

```
    <h3 class="text-xl font-bold text-blue-400 mb-4">В
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-6 text
```

```
      <p class="text-sm text-blue-300 italic mb-2">0с
```

```
      <p class="text-lg font-mono text-white">Бор (Tr
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm">Позволяет найти
```

```
      -mono text-amber-300 bg-black/30 px-1 rounded">
```

```
      y.</p>
```

```
</div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
```

```
  <div class="bg-slate-800 p-6 rounded-lg border bo
```

```
    <div class="absolute -top-3 left-4 bg-slate-700
```

```
      emerald-500 shadow-md">
```

```
      Аналогия 1: Паутина (Бор)
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">Представ
```

```
      мы движемся по веткам. Если нужного продолж
```

```
      ("нити страховки") в самый длинный из извест
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2
```

```
  <div class="absolute -top-3 left-4 bg-rose-900
```

```
    -500 shadow-md">
```

```
    Аналогия 2: Матёшка (Терминальные)
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb-4">Представ
```

```
    нашли и "he", потому что оно спрятано внутри
```

```
    минальные ссылки</b> (term_link) – прямые мос
```

```
    p>
```

```
</div>
```

```
</div>
```

```

        nodes[child].term_link = nodes[child].l
    } else {
        nodes[child].term_link = nodes[nodes[ch
    }

    q.push(child);
}
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
    <span class="bg-purple-600 text-white px-4 py-1
    <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
        <h3 class="text-xl font-bold text-purple-400
        <p class="text-slate-300 text-sm mb-4">Когда
        ут быть большими дробными числами (долгота
        ву от 0 до N, и уже по ним строим графы.</p>
    </div>
</div>
</section>`
}
];

```

```

        question: "Куда ведёт эйлеров путь, если нечётных
options: [
    "Всё ещё можно пройти по всем рёбрам ровно раз",
    "Такого графа не существует",
    "Эйлерова пути нет – нужно удалять лишние рёбра",
],
correctIndex: 2,
explanation: "Если нечётных вершин больше 2, эйлеров
    ровно 2 нечётные или 0."
    }
    ],
    "bridges-code": [
        {
            question: "После DFS у нас  $low[u] = 4$ , а  $tin[v] = 2$ ",
options: [
    "Да, так как  $low[u] (4) > tin[v] (2)$ ",
    "Нет,  $low[u]$  должно быть меньше или равно",
    "Только если граф связный и неориентированный"
],
correctIndex: 0,
explanation: "Условие моста:  $low[u] > tin[v]$ . В неориентированном графе
        },
        {
            question: "Для чего нужен массив  $low[v]$ ?",
options: [
    "Для того чтобы было больше переменных в коде",
    "Чтобы знать минимальный  $tin$  предка, до которого нужно вернуться",
    "Чтобы хранить глубину рекурсии"
],
correctIndex: 1,
explanation: " $low[v]$  хранит минимальное время входа в вершину  $v$ 
            поиска мостов и точек сочленения."
        },
        {
            question: "Какова временная сложность алгоритма поиска мостов?",
options: [
    " $O(V^2)$  – ищем мосты в квадрате",
    " $O(V + E)$  – каждый элемент один раз",

```

```
    ]  
};
```

ФАЙЛ 24/96: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 158 | РАЗМ

```
import { createContext, useContext, useEffect, useState } from "react";  
import type { DebugValue } from "../debugRunner";
```

```
/**  
 * Прямая связь «компилятор ↔ открытая визуализация».  
 *  
 * Компилятор передаёт не «магический номер строки эталонного  
 * выполненного Python-кода. Визуализация читает захардкоженные  
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие  
 * любой пользовательский код работает, если в нём использованы  
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {  
  chapterId: string;  
  /** Строка и функция реально выполненного пользователем кода */  
  line?: number;  
  func?: string;  
  /** Машинно-читаемые globals + locals: числа остаются числами, строки  
  variables?: Record<string, DebugValue>;  
  /** Имена, изменённые выделенной строкой. */  
  changed?: string[];  
}
```

```
/** Последний снимок хранится, чтобы поздно смонтировать визуализацию  
const latestByChapter = new Map<string, VizStateEventDetail>();
```



```

    return runtime;
}

export type VizStepSelector = (variables: Record<string>

/**
 * Подписка старых покадровых демонстраций. Если переда
 * номер кадра выбирается по именам/значениям переменны
 */
export function useVizStepSync(
  currentStep: number,
  goToStep: (n: number) => void,
  maxStep: number,
  selectByVariables?: VizStepSelector
) {
  const chapterId = useContext(VizChapterContext);

  useEffect(() => {
    if (!chapterId) return;

    const apply = (detail?: VizStateEventDetail) => {
      if (!detail || detail.chapterId !== chapterId) re
      // Без явного селектора компонент либо рисует use
      // либо остаётся ручным. Номер Python-строки нико
      const requested =
        selectByVariables && detail.variables ? selectB
      // null означает: компонент рисует runtime-перемен
      // не увидел знакомых захардкоженных имён.
      if (requested === null || !Number.isFinite(requested))
      const target = Math.max(0, Math.min(Math.max(0, m
      if (target !== currentStep) goToStep(target);
    };

    apply(latestByChapter.get(chapterId));
    const handler = (e: Event) => apply((e as CustomEvent)
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, ha

```

```
* Сразу отдаём последнее значение: компилятор часто от
* как пользователь переключился с 1D на 2D.
*/
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window === "undefined") return () => {};
  const latest = latestDemoByChapter.get(chapterId);
  if (latest !== undefined) cb(latest);
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

ФАЙЛ 25/96: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 883 | РАЗМЕР: 1.5 КБ

```
/**
 * Захардкоженная «карта синхронизации» компилятора с визуализацией
 *
 * Для каждой страницы (главы) здесь вручную перечислены переменные,
 * которые понимает её визуализация: i, j, k, v, dist, ...
 * Компилятор выполняет любой Python-код и передаёт результат
 * имён напрямую; привязки к номерам или тексту строк эмулируются
 */
```

```
export interface SyncVariable {
  /** Имя переменной, как в коде визуализации: i, j, k, ... */
  name: string;
  /** Роль в алгоритме (показывается во всплывающей подсказке) */
  role: string;
  /** Характерный диапазон/значение на демонстрации. */
  range?: string;
}
```

```

variables: [
  { name: "n", role: "длина исходного массива; рекурсия"},
  { name: "v", role: "текущая вершина дерева – подсчитана"},
  { name: "l, r", role: "границы отрезка, за который"},
  { name: "m", role: "середина отрезка  $m = (l + r) // 2$ " },
  { name: "ul, ur", role: "границы массива обновления"},
  { name: "add", role: "добавка  $A[ul..ur] += add$  – инкремент"},
  { name: "lazy", role: "массив обещаний:  $lazy[v] \neq 0$ " }
],
},
treap: {
  vizTitle: "Treap: плоскость → дерево → Split/Merge/Insert",
  stepNote: "Шаг = вставка очередной пары (x; y) или удаление",
  code: `pairs = [(45, 8), (3, 6), (6, 5), (2, 4), (9, 7)]
pairs.sort(key=lambda p: -p[1])      # сортируем по y – по убыванию
x, y = 6, 5                          # подсвеченная пара
i = 0                                # индекс шага: сортировка

def insert(root, x, y):               # вставка спуском по ключу
  ...                                # ключ <= узла – вставляем
def split(t, x):                      # разрез по ключу:
  ...                                # узел целиком уходит в левое/правое
  variables: [
    { name: "x", role: "ключ – координата точки по горизонтальной оси"},
    { name: "y", role: "приоритет – координата по вертикали"},
    { name: "i", role: "индекс шага: сортировка (0...3)" }
  ],
},
"splay-tree": {
  vizTitle: "Splay-дерево: большое дерево",
  stepNote: "Шаг = один кадр splay(key): прицел ребра",
  code: `keys = [50, 40, 35, 32, 30, 28, 12, 60, 55, 45]
key = 32                             # какой ключ поднимаем в корень
i = 0                                # кадр анимации: 0 ... (число кадров)
case = "zig-zig"                     # вкладка «Случай поворота»

```

```

        { name: "n, LOG", role: "длина массива и число ур
        { name: "i", role: "начало блока: st[i][j] отвеча
        { name: "j", role: "уровень таблицы: блок длины 2
        { name: "st", role: "таблица уровней – подсвечива
    ],
},
"prefix-sums-2d": {
    vizTitle: "Префиксные суммы (1D и 2D)",
    stepNote: "Шаг = заполнение очередного  $P[i] = P[i-1]$ 
    code: `a = [3, 1, 4, 1, 5, 9, 2, 6]
i = 0

P = [0]
for i in range(1, len(a) + 1):
    P.append(P[i - 1] + a[i - 1])
print("P =", P)`,
    variables: [
        { name: "a", role: "исходный массив демо (вкладка
        { name: "i", role: "индекс элемента, который сейч
        { name: "P", role: "префиксные суммы:  $P[i] = a[0]$ 
    ],
},
"dynamic-programming": {
    vizTitle: "Динамическое программирование (мемоизаци
    stepNote: "Шаг = событие внутри fibМемо(k): вызов,
    code: `n = 5
memo = {}

def fib(k):
    if k in memo:
        return memo[k]
    if k <= 2:
        return 1
    print(f"считаем fib({k-1}) + fib({k-2})")
    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
    return memo[k]

```

```

    ],
},
"queue-bfs": {
    vizTitle: "Очередь и обход в ширину",
    stepNote: "Шаг = dequeue вершины из головы очереди",
    code: `from collections import deque

q = deque(["A"])                # очередь BFS (FIFO)
q.append("B")                   # enqueue соседа в хвост
v = q.popleft()                 # шаг: обрабатываем голову
print("v =", v, "очередь:", list(q))`,
    variables: [
        { name: "q", role: "очередь (массив): гости страны" },
        { name: "v", role: "вершина, извлечённая из головы" },
    ],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к непосещенной",
    code: `adj = {"A": ["B", "C"], "B": ["A", "D", "E"], "C": ["A"], "D": ["B"], "E": ["B"]}
visited, order = set(), []      # порядок обхода
stack = ["A"]                  # DFS: стек (рекурсия)
q = []                         # BFS: очередь (deque)
v = "A"                        # шаг: достали вершину из очереди

# --- тело алгоритма ---
while stack:                    # ---- DFS ----
    v = stack.pop()
    if v in visited:
        continue
    visited.add(v)
    order.append(v)
    stack += [u for u in adj[v] if u not in visited]
print("DFS:", order)

visited, order, q = set(), [], ["A"]    # ---- BFS ----
while q:
    v = q.pop(0)
    if v in visited:
        continue
    visited.add(v)
    order.append(v)
    q += [u for u in adj[v] if u not in visited]
print("BFS:", order)
`

```

```

        q.append(to)
print(comp, "компонент:", count)`,
    variables: [
        { name: "v", role: "текущая вершина обхода – подсвечивается"},
        { name: "comp", role: "словарь меток: номер компонента"},
        { name: "q", role: "текущая очередь BFS", range: 100},
        { name: "count", role: "число запусков обхода = число компонент"},
    ],
},
"top-sort": {
    vizTitle: "Топологическая сортировка",
    stepNote: "Шаг = выход рекурсии из вершины v: она добавлена в конец списка",
    code: `adj = {0: [1, 2], 1: [3], 2: [3], 3: [4], 4: [5]},
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:
        # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)
    # выход из v!

# --- тело алгоритма ---
for v in adj:
    if v not in used:
        dfs(v)
print(order[::-1])
# разворот – ответ`,
    variables: [
        { name: "v", role: "текущая вершина DFS – подсвечивается"},
        { name: "to", role: "вершина, куда ведёт ребро из v"},
        { name: "order", role: "порядок выхода из рекурсии"},
        { name: "used", role: "множество посещённых вершин"},
    ],
},
"scc-kosaraju": {
    vizTitle: "Компоненты сильной связности (Косарайю)",
    stepNote: "Шаг = вершина из order (в обратном порядке)",
    code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [4], 4: [3]}`,
    variables: [
        { name: "v", role: "текущая вершина DFS – подсвечивается"},
        { name: "to", role: "вершина, куда ведёт ребро из v"},
        { name: "order", role: "порядок выхода из рекурсии"},
        { name: "used", role: "множество посещённых вершин"},
    ],
},
}
```

```

        { name: "order", role: "порядок выхода прохода 1"},
        { name: "adjT", role: "транспонированный граф: все ребра"},
        { name: "comp", role: "номер SCC каждой вершины"},
    ],
},
"graph-articulation": {
    vizTitle: "Точки сочленения",
    stepNote: "Шаг = обновление low[v] по ребру; вершина v",
    code: `# демо-граф страницы: треугольник 0-1-2, мост 3-4, 5-6
# DFS из 0: tin/low после полного обхода; точки сочленения
tin = {0: 1, 1: 2, 2: 3, 3: 4, 4: 5, 5: 6, 6: 7}
low = {0: 1, 1: 1, 2: 1, 3: 4, 4: 5, 5: 5, 6: 5}

v, to = 3, 4                # шаг: возврат из 4 в 3
is_cut = low[to] >= tin[v]   # 5 >= 4 → 3 — точка сочленения
v2, to2 = 1, 2
is_cut2 = low[to2] >= tin[v2] # 1 >= 2 → нет (1 — не точка сочленения)
print(f"v={v}: точка сочленения? {is_cut}; v={v2}: {is_cut2}")
    variables: [
        { name: "tin", role: "время захода DFS в вершину"},
        { name: "low", role: "минимальный tin, достижимый"},
        { name: "v, to", role: "ребро дерева DFS: проверяем"},
        { name: "is_cut", role: "low[to] ≥ tin[v] и v — не"},
    ],
},
"bridges-code": {
    vizTitle: "Мосты: DFS + tin/low",
    stepNote: "Шаг = строка псевдокода слева; ребро (v, to)",
    code: `# демо-граф страницы: A-B, B-C, A-C, C-D, D-E, E-F, F-G
# DFS из A: tin/low после полного обхода
tin = {"A": 1, "B": 2, "C": 3, "D": 4, "E": 5, "F": 6, "G": 7}
low = {"A": 1, "B": 1, "C": 1, "D": 4, "E": 4, "F": 4, "G": 7}

v, to = "B", "G"            # шаг: возврат из G в B (G — не мост)
is_bridge = low[to] > tin[v] # 7 > 2 → мост!
v2, to2 = "C", "D"
is_bridge2 = low[to2] > tin[v2] # 4 > 3 → C-D тоже мост!
print(f"{v}-{to}: мост? {is_bridge}; {v2}-{to2}: мост? {is_bridge2}")

```



```

    ],
},
"floyd": {
    vizTitle: "Флойд на матрице аэропортов",
    stepNote: "Шаг = проверка пары (i, j) при посреднике",
    code: `INF = 999 # «бесконечность»
                # поэтому отдельные проверки
d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF], [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, 0]]
n = 7
k, i, j = 0, 0, 0 # шаг: посредник k, летим

# --- тело алгоритма ---
for k in range(n): # посредник k
    for i in range(n):
        for j in range(n):
            if d[i][k] + d[k][j] < d[i][j]: # через k
                d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
    variables: [
        { name: "d", role: "матрица смежности → кратчайшие расстояния"},
        { name: "n", role: "число вершин (аэропортов)", g: 1},
        { name: "k", role: "посредник: разрешённые пути и расстояния"},
        { name: "i, j", role: "клетка d[i][j], которую проверяем"},
        { name: "d[i][j]", role: "текущая длина кратчайшего пути"},
    ],
},
"johnson-algo": {
    vizTitle: "Алгоритм Джонсона (перевзвешивание)",
    stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
    code: `# демо-граф страницы: S→{A,B,C} (0), A→B=-2, B→C=1, C→A=2
# потенциалы — реальные расстояния от фиктивной S (Беллман-Форд)
h = {"A": 0, "B": -2, "C": -1}

u, v, w = "A", "B", -2 # ребро с отрицательным весом
w_new = w + h[u] - h[v] # w' = w + h[u] - h[v] = -2
print(f"w'({u}→{v}) = {w_new}")
# остальные: B→C: 1 + (-2) - (-1) = 0 • C→A: 2 + (-1) - 0 = 1
`
}

```

```

        { name: "parent", role: "DSU: представитель компо
        { name: "mst", role: "рёбра остова; в ответе равн
    ],
},
"mst-prima": {
    vizTitle: "Прим",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребр
    code: `import heapq

# смежность демо-графа страницы: 9 вершин A..I
adj = {"A": [(2, "B"), (5, "D")], "B": [(2, "A"), (4, "C"),
      "C": [(4, "B"), (9, "F")], "D": [(5, "A"), (3, "E"),
      "F": [(7, "E"), (9, "C"), (13, "I")], "G": [(8, "H"),
      "I": [(12, "H"), (13, "F")]}
heap = [(0, "A", "-")]          # (вес, вершина, откуда)
taken, mst = set(), []
w, v, parent = 2, "B", "A"     # шаг: ребро из кучи
i = 0

# --- тело алгоритма ---
while heap and len(taken) < len(adj):
    w, v, parent = heapq.heappop(heap)
    if v in taken:
        continue
    taken.add(v)
    if parent != "-":
        mst.append((w, parent, v))
    for w2, u in adj[v]:
        if u not in taken:
            heapq.heappush(heap, (w2, u, v))
print("остов:", mst, "вес:", sum(w for w, _, _ in mst))
variables: [
    { name: "adj", role: "список смежности демо-графа
    { name: "w, v, parent", role: "ребро из кучи: вес
    { name: "heap", role: "приоритетная очередь канди
    { name: "taken", role: "вершины уже в дереве — «з
    { name: "mst", role: "рёбра остова, вес 51", rang
],

```

```

        { name: "cheapest", role: "самое дешёвое исходящее ребро",
        { name: "u, v, w", role: "ребро-кандидат: концы и вес",
        { name: "phase", role: "номер фазы параллельных шагов",
    ],
},
"string-kmp": {
    vizTitle: "Префикс-функция (КМП)",
    stepNote: "Шаг = обработка символа s[i]: откаты по префиксам",
    code: `pattern = "aba"
text = "abacaba"
s = pattern + "#" + text      # демо-страница ищет pattern в text
i, j = 1, 0
pi = [None] * len(s)          # пустые клетки: значения префикс-функции
pi[0] = 0

# --- тело алгоритма ---
for i in range(1, len(s)):    # шаг: считаем pi[i]
    j = pi[i - 1]              # длина совпадения
    while j > 0 and s[i] != s[j]:
        j = pi[j - 1]          # откат назад
    if s[i] == s[j]:
        j += 1
    pi[i] = j
print(pi)`,
    variables: [
        { name: "pattern", role: "образец: питон подставлен",
        { name: "text", role: "текст, в котором ищем",
        { name: "s", role: "склейка pattern + \"#\" + text",
        { name: "i", role: "индекс текущего символа – считаем",
        { name: "j", role: "длина текущего совпавшего префикса",
        { name: "pi", role: "префикс-функция: pi[i] – наибольшая длина",
    ],
},
"string-z-func": {
    vizTitle: "Z-функция",
    stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r] найти",
    code: `pattern = "aba"
text = "abacaba"

```

```

        { name: "v", role: "номер текущей вершины автомата",
          { name: "с / state", role: "символ и состояние —
    ],
  },
  "complexity-classes": {
    vizTitle: "Классы сложности: матрёшка  $P \subseteq NP \subseteq PSPACE$ ",
    stepNote: "Шаг = подсветка класса/задачи по имени и",
    code: `cls = "NP" # класс: "P", "NP"
task = "3-SAT" # задача: "3-SAT", "Vertex Cover"

# сведение A ≤p B: стрелка между задачами
reductions = [{"3-SAT", "Vertex Cover"}]
for a, b in reductions:
  print(f"{a} ≤p {b}")`,
  variables: [
    { name: "cls", role: "какой класс подсветить: P / NP",
    { name: "task", role: "какую задачу подсветить: 3-SAT, Vertex Cover",
    { name: "reductions", role: "список пар задач — по какому правилу сводятся",
  ],
},
intro: {
  // мнемодкарточки без шагов алгоритма
  variables: [],
},
"alg-map": {
  variables: [],
},

/* — Дополнительные демонстрации внутри страниц (включая)

"sparse-table#2d-build": {
  vizTitle: "Разреженная таблица 2D: построение",
  stepNote: "Шаг = новый уровень k: блоки  $2^k \times 2^k$  доп.",
  code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
    88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
    1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

```

```

        stepNote: "Демонстрация кликабельна вручную: наведи
        code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2 # клетки 1-индексные, как
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`,
    variables: [
        { name: "i, j", role: "текущая ячейка таблицы S (
        { name: "S[i][j]", role: "сумма прямоугольника (0
        },
        { name: "r1, c1, r2, c2", role: "углы запроса в 1
        ний", range: "1 ... n, 1 ... m" },
    ],
},
};

```

```

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null,
    (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demoId}`]
    (chapterId && PAGE_SYNC[chapterId])) ||
    { variables: [] });

```

/**

* Стартовое содержимое редактора для страницы.

*

* Формат: короткий комментарий-шапка + исполняемый реф

* Визуализация читает из его трассы знакомые имена (i,

* но точно так же примет пользовательский код с теми ж

*/

```
-----
: `# «${chapterTitle}»\n# демо на странице нет — св

if (!sync.code) return head;
const init = extractInit(sync.code);
return `${head}\n${init}\n`;
}
```

=====

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

ФАЙЛ 26/96: src/data/tickets/ticket1_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 335 | РАЗМ

```
import { Chapter } from '../..types';

export const tickets1to5: Chapter[] = [
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description: "Дерево отрезков — структура данных дл",
    category: "Оптимизации и Продвинутое структуры",
    content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div id="intro" class="bg-slate-700/50 p-6 round
      <h3 class="text-xl font-bold text-blue-400
    </div>

    <div class="bg-slate-900 p-4 rounded-lg mb-1
```

```

        tree[2 * node] += lazy[node];
        lazy[2 * node + 1] += lazy[node];
        tree[2 * node + 1] += lazy[node];
        lazy[node] = 0;
    }
}
</pre>
</div>
</details>
</div>
</div>
</section>
`
},
{
  id: "sparse-table",
  title: "2. Двумерная разреженная матрица (Sparse Table)",
  type: "html",
  description: "Разреженная таблица для идемпотентных",
  category: "Продвинутые структуры",
  content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
  
```

```

        </div>
        <ul class="text-slate-300 text-sm s
            <li>• Размер: <code class="bg-s
-800">N × M</code> – одна копия матрицы, и
            <li>• Запрос суммы на прямоугол
            <li>• Умеет <b>только сумму</b>
            <li>• Менять элементы нельзя –
        </ul>
    </div>
    <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
        2D Sparse table минимумов (эт
    </div>
    <ul class="text-slate-300 text-sm s
        <li>• Размер: <code class="bg-s
0">N × M × log N × log M</code> – жирнее в
        <li>• Запрос минимума на прямоу
        <li>• Умеет <b>min, max, gcd</b>
        <li>• Менять элементы тоже нель
    </ul>
</div>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2
    <div class="bg-slate-800 p-6 rounded-lg
        <div class="absolute -top-3 left-4
er border-amber-500 shadow-md">
        Аналогия 3: Слоёный тортик
    </div>
    <p class="text-slate-300 text-sm">П
й по каждой оси – log (для 1000×1000 это 10
ена O(1) для минимума: запекли все возможные
    </div>
    <div class="bg-slate-900 p-6 rounded-lg
        <div class="absolute -top-3 left-4
border-sky-500 shadow-md">
        Аналогия 4: Аккумулятор
    </div>

```



```

<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Удар катаной (Sp
    </div>
    <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
        Аналогия 2: Слияние капель (M
      </div>
      <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
    </div>
  </div>
<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <pre class="bg-slate-950 p-4 rounded
pair<Node*, Node*> split(Node* t, int x) {
  if (!t) return {nullptr, nullptr};
  if (t->val <= x) {
    auto [L, R] = split(t->right, x);
    t->right = L;
    return {t, R};
  } else {
    auto [L, R] = split(t->left, x);
    t->left = R;
    return {L, t};
  }
}</pre>

```

```
        border-rose-500 shadow-md">
            Аналогия 2: Балансировка чере
        </div>
        <p class="text-slate-300 text-sm mb
и становиться кривым как бамбук. Но как тол
    </p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
// Zig, Zag
function rotate(x) {
    let p = x.parent;
    if (p.left === x) { // Right rotate
        p.left = x.right;
        x.right = p;
    } else { // Left rotate
        p.right = x.left;
        x.left = p;
    }
}
}</pre>

        </div>
    </details>

    <div class="bg-indigo-950/60 p-6 rounded-xl
        <h4 class="text-lg font-bold text-indig

        <div class="space-y-4 text-sm text-slate
            <div>
                <strong class="text-white block
корень?</strong>
                <p>Оно поднимет узел, на котором
ска. За счет сдвига этого листа в корень, д
```

```

<h3 class="text-xl font-bold text-emerald-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-emerald-300 italic">
    <p class="text-xl font-mono text-white">
      , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
    </p>
  </div>
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
        * Аналогия 1: Вырезание прямоугольника
      </div>
      <p class="text-slate-300 text-sm mb-4">
        з него маленький целевой прямоугольник в центре.
        ый верхний угол отрезается дважды! Поэтому
      </p>
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border border-sky-500 shadow-md">
          Аналогия 2: Аккумулятор
        </div>
        <p class="text-slate-300 text-sm mb-4">
          аккумулирует</b>: каждая клетка P[i][j] уже
        </b> по включениям-исключениям. Фокус держи
        я печь слёный тортик из ~100 копий матрицы.
      </p>
    </div>
  </div>
  <div class="bg-slate-900/40 rounded-lg border">
    <p class="text-slate-400 text-sm"> <b>
      <b class="text-emerald-300">суммы</b>; раз
    </b>. Сумму через sparse table считать нельзя
    (это не базовая операция).</p>
  </div>
</div>
</div>
</section>
}
];

```

```

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg"
          <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
            Ограничения
          </div>
          <p class="text-slate-300 text-sm mb-1">
дный и не умеет пересматривать уже "зафиксир
          </div>
        </div>
        <div id="slot-dijkstra-viz" class="mt-8"></div>
      </div>
    </div>
  </section>`
  },
  {
    id: "bellman-ford",
    title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="bellman-ford-content" class="mb-12 scroll-py-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">15</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Пути</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">Параноик</p>
        <p class="text-xl font-mono text-white">...</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4
border border-emerald-500 shadow-md">
            ♂ Аналогия 1: Параноик

```

```

        <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
            Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Главный герой – внешний цикл
            шаг class="bg-slate-900 text-pink-400">k?</code>
            шу себе проходить через вершину k?</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
            <li><b>Шаг k=0:</b> Ты знаешь только k=0
            <li><b>Шаг k=1:</b> Разрешаем пересадки
            e> через путь class="text-indigo-300">k?</code>
            <li><b>Шаг k=2:</b> Разрешаем пересадки
            нутри этих путей уже могут быть пересадки ч
            <li>...</li>
            <li><b>Шаг k=N:</b> Ты проверил все
        </li>
        </ul>
    </div>
</div>
</section>`
    },
    {
        id: "johnson-algo",
        title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border">
                <h3 class="text-xl font-bold text-amber-400">

```

```

        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500 shadow-md">
            <h3 class="text-xl font-bold text-emerald-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-emerald-300 italic">
                        <p class="text-xl font-mono text-white">
                            (проверяем через DSU) - берем в ответ.</p>
                </div>
                <div class="grid grid-cols-1 gap-6">
                    <div class="bg-slate-800 p-6 rounded-lg">
                        <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                            Аналогия: Прокладка интернет
                        </div>
                        <p class="text-slate-300 text-sm mb-4">
                            с самых дешевых дорог в стране". Он строит
                            единую сеть.</p>
                    </div>
                </div>
            </div>
        </div>
    </section>`
    },
    {
        id: "mst-prima",
        title: "19. Графы. Остовное дерево. Прима",
        type: "html",
        category: "Графы. Остовные",
        content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
    </div>
    <div class="space-y-8">

```

```

        <p class="text-xl font-mono text-white">
ается с соседом.</p>
</div>
<div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия: Племена
        </div>
        <p class="text-slate-300 text-sm mb
изкому племени для объединения. К вечеру пл
ства шлют гонцов к ближайшему чужому царств
        </div>
    </div>
</div>
</div>
</section>`
    }
};

```

ФАЙЛ 28/96: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```

import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
  {
    id: "string-kmp",
    title: "21. Префикс-функция (π) — Взгляд назад (КМП",
    type: "html",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1

```

```

        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg l
        <summary class="p-4 font-bold text-slate
        -b border-slate-700/50">
        Пример вычисления (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <p>Тот же пример: <b>abcabcd</b></p>
        <ul class="list-disc pl-5 space-y-2
            <li><b>a</b>: Префиксов нет. <code>
            <li><b>ab</b>: Из начала ("a")
        >
            <li><b>abc</b>: Опять мимо. <code>
            <li><b>abca</b>: 0! Начало ("a"
        </code></li>
            <li><b>abcab</b>: Начало ("ab")
        /li>
            <li><b>abcabc</b>: Начало ("abc
        e></li>
            <li><b>abcabcd</b>: "d" всё исп
        с' — не совпало. Откат по  $\pi$ :  $j = \pi[2] = 0$ .
         $\pi[6] = 0$ </code></li>
        </ul>
    </div>
</details>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
    -b border-slate-700/50">
        Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];

```


я начну читать строку отсюда, насколько длин
Это самый быстрый способ найти
>, считаешь Z-функцию, и там, где `Z[i`

</p>

</div>

<!-- Аналогия 2 -->

<div class="bg-slate-900 p-6 rounded-lg

<div class="absolute -top-3 left-4

border-rose-500 shadow-md">

Аналогия 2: LLM Самокопиаст

</div>

<p class="text-slate-300 text-sm mb

В LLM Z-функция может применять
[i]</code> (где <i>i</i> указывает назад на
сама себя.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Пример вычисления (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<p>Тот же пример: abcabcd</p>

<ul class="list-disc pl-5 space-y-2

<code>Z[0]</code> – не счита

<code>Z[1]</code> (bcabcd

[1] = 0</code>

<code>Z[2]</code> (cabcd

<code>Z[3]</code> (abcd<

? Нет, в начале 'a', тут 'd'). Совпало 3 бу

<code>Z[4], Z[5], Z[6]</code>

</div>

```
    <p class="text-lg font-mono text-white">Бор (Тр
  </div>
  <p class="text-slate-300 text-sm">Позволяет найти
    го один проход.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      ет — мы падаем по <b>суффиксной ссылке</b> ("
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
      </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border b
      <summary class="p-4 font-bold text-slate-400 ou
        r-slate-700/50">
        Код (Скрыто)
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-
        <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch)
```

```

        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
        </div>
        <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg
            <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
                Аналогия 2: Машина-переводчик
            </div>
            <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведенн
аче В, а сама В лежит в NP, то В – <b>NP-по
        </div>
    </div>
</div>
</section>`
    }
];

```

ФАЙЛ 29/96: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```

    {
        id: "graph-dfs-bfs",
        title: "6. Графы. Представление, DFS, BFS",
        type: "html",
    },

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
  <h3 class="text-xl font-bold text-indigo-400"

<div class="grid grid-cols-1 xl:grid-cols-2
  <!-- Аналогия DFS -->
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
      rder border-indigo-500 shadow-md">
        ♂ DFS (Поиск в глубину) = Жадный
      </div>
      <p class="text-slate-300 text-sm mb-2">
        <b>Что делает:</b> Жадный алгоритм находит ближайшую
        ую вершину (например, минимальную по номеру) и пробует другой путь.
      </p>
      <ul class="text-xs text-indigo-300">
        <li><b>Где используется:</b></li>
        <li>Топологическая сортировка (линейное время)</li>
        <li>Поиск мостов и точек сочленения</li>
        <li>Сильно связанные компоненты Киркпатрика</li>
        <li>Поиск циклов и просто проверка связности</li>
      </ul>
    </div>

    <!-- Аналогия BFS -->
    <div class="bg-slate-900 p-6 rounded-lg
      <div class="absolute -top-3 left-4 l
        border border-emerald-500 shadow-md">
          ♂ BFS (Поиск в ширину) = Волна
        </div>
        <p class="text-slate-300 text-sm mb-2">
          <b>Что делает:</b> Концентрические волны.
          Работает через Очередь (FIFO). Продвигается от начальной
        </p>
        <ul class="text-xs text-emerald-300">
          <li><b>Где используется:</b></li>
          <li>Кратчайший путь в Невзвешенном графе</li>
          <li>Алгоритм Ахо-Корасик (сборка слов)</li>

```

```

        </div>
      </details>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Планарные. Покраска</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
        <p class="text-xl font-mono text-white">Эта теорема была доказана в 1878 году.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">Границы не пересекаются в одной точке по-
            тому что страны не сливались цветами, Всегда можно всего 4</p>
        </div>
      </div>
    </div>
  </div>
</section>`
  }
]

```

```

{
  id: "top-sort",
  title: "9. Графы. Топологическая сортировка",
  type: "html",
  description: "Разложение задач по порядку выполнения",
  category: "Графы",
  content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">9
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Топологическая сортировка
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Топологическая сортировка
        <p class="text-xl font-mono text-white">
          перед.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            border border-emerald-500 shadow-md">
            Аналогия 1: Учеба в вузе
          </div>
          <p class="text-slate-300 text-sm mb-4">Топологическая сортировка – это расписание, которое
            р>
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4
              border border-rose-500 shadow-md">
              Ограничения: Циклы
            </div>
            <p class="text-slate-300 text-sm mb-4">получить опыт, нужна работа". Алгоритм выявления
          </div>
        </div>
      </div>
    </div>
  </div>
`

```

```

{
  id: "scc-kosaraju",
  title: "10. Графы. Компоненты сильной связности (SCC)",
  type: "html",
  description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
  category: "Графы. Продвинутое",
  content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700/80">
      <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
    </div>

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Макро-вершина — это вершина орграфа, состоящая из
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">Компоненты сильной связности (SCC) — это вершины ориентированного графа, в котором
        </p>
      </div>
    </div>

    <p class="text-slate-300 text-sm mb-4">
      <strong>«Это просто цикл?» — Нет!</strong>
    </p>

    <ul class="list-disc list-inside text-sm text-slate-300">
      <li>Если у тебя есть цикл <code>A → B → C → A</code>, то это цикл.
      <li>Если ты добавишь вершину <strong>Г</strong> к этому циклу, то получишь четыре вершины — одна большая SCC.</li>
      <li>Правило: <b>Если два цикла имеют хотя бы одну общую вершину, то они образуют один цикл.</b>
    </ul>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4">

```

```

        </ol>
      </div>
    </details>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-bridges",
    title: "11. Графы. Мосты",
    type: "html",
    description: "Рёбра, удаление которых расхреначивает",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Этот мост, экономикой обоих кусков придет
          </p>
        </div>
      </div>
    </div>
  </div>
</div>

```



```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
    <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком (удалят вершину) — районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
  <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400">
    Душный мод: Код и корень DFS (Скрыто)
```

```
</summary>
  <div class="p-5 text-sm text-slate-300 space-y-2">
    <p class="text-blue-400 font-bold">Особый

```

```
<p>
```

Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для неё у него больше 1 независимого ребенка

```

        Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая <strong>соединенная</strong> вершина и есть критическая SCC! Если удалить эту вершину, то граф распадется на несколько частей. Одно слабое звено изолирует целые касты SCC.
    </ul>
</div>
</div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">13. Графы. Эйлеров цикл
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-indigo-400">Эйлеров цикл
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">Эйлеров цикл — это замкнутый путь, проходящий по каждому ребру графа ровно один раз.
        <p class="text-xl font-mono text-white">Граф имеет Эйлеров цикл тогда и только тогда, когда все вершины имеют четную степень.
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Рисование не отрывая карандаша
          </div>
          <p class="text-slate-300 text-sm mb-4">Если граф имеет Эйлеров цикл, то можно пройти по всем ребрам ровно один раз, не отрывая карандаша. Если же граф не имеет Эйлерова цикла, то придется отрывать карандаш. Если граф имеет Эйлеров цикл, то можно пройти по всем ребрам ровно один раз, не отрывая карандаша. Если же граф не имеет Эйлерова цикла, то придется отрывать карандаш.
        </div>
      </div>
    </div>
  </div>
`
  }
]

```

```

    { id: 4, x: 550, y: 150, label: "4" },
    { id: 5, x: 700, y: 50, label: "5" },
    { id: 6, x: 700, y: 250, label: "6" },
  ];

const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
  { length: Object.keys(nodes).length },
  () => [],
);

edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt

```

```

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${v}],
                v,
            );
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], currentLow[to]);

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v}],
                v,
            );

            if (currentLow[to] >= currentTin[v] && p !== -1) {
                ap.add(v);
                recordStep(
                    `Найдена точка сочленения: вершина ${v},
                    v,
                );
            }
        }
    }

    if (p === -1 && children > 1) {
        ap.add(v);
        recordStep(
            `Корень дерева DFS ${v} имеет >1 детей. Это
            v,
        );
    } else if (p === -1) {
        recordStep(
            `Корень дерева DFS ${v} имеет <=1 ребенка, он
            v,
        );
    }
}

```

```

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}
      </button>

      <button
        onClick={reset}
        className="flex items-center justify-center
        ors"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-3 g
    <div className="lg:col-span-2 relative h-96 lg:h
      <svg className="w-full h-full" viewBox="0 0 8
        {/* Edges */}
        {edges.map((edge, i) => {

```

```

        : isVisited
        ? "#34d399"
        : "#334155";
const r = isAp ? 24 : 20;

return (
  <g key={node.id}>
    <circle
      cx={node.x}
      cy={node.y}
      r={r}
      fill={fill}
      stroke={stroke}
      strokeWidth="3"
      className="transition-all duration-100"
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"
      textAnchor="middle"
      fill="white"
      fontSize="14px"
      fontWeight="bold"
      className="select-none pointer-events-none"
    >
      {node.label}
    </text>

    {isVisited && (
      <>
        <text
          x={node.x}
          y={node.y - r - 15}
          textAnchor="middle"
          fill="#94a3b8"
          fontSize="10px"
        >

```

```

<h5 className="text-xs text-slate-500 font-medium">
  <div className="flex items-center gap-2">
    <div className="w-3 h-3 rounded-full bg-rose-500 border border-rose-500">
      Точка сочленения
    </div>
    <div className="flex items-center gap-2">
      <div className="w-3 h-3 rounded-full bg-rose-500 border border-rose-500">
        Посещена
      </div>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-rose-500 border border-rose-500">
          Текущая
        </div>
        <div className="flex items-center gap-2">
          <div className="w-8 h-1 bg-rose-500 border border-rose-500">
            Мост
          </div>
        </div>
      </div>
    </div>

  <div>
    <h5 className="text-xs text-slate-500 font-medium">
      ЛОГ ДЕЙСТВИЙ:
    </h5>
    <div className="space-y-2">
      {steps
        .slice(0, currentStepIndex + 1)
        .reverse()
        .map((step, idx) => {
          <div
            key={idx}
            className={`text-xs p-2 rounded $
xt-slate-500`} `>
            >
              {step.desc}
            </div>
          </div>
        })}
    </div>
  </div>

```

```

-----
export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
      { u: 'B', v: 'C', w: -2 },
      { u: 'A', v: 'C', w: 1 },
      { u: 'C', v: 'D', w: 3 },
      { u: 'C', v: 'E', w: 4 },
      { u: 'D', v: 'F', w: 2 },
      { u: 'E', v: 'F', w: 1 },
    ];
    if (cycle) {
      return [...defaultEdges, { u: 'D', v: 'A', w: -6 }
    ] else {
      return [...defaultEdges, { u: 'D', v: 'A', w: -2 }
    ]
  };

  const edges = getEdges(hasCycle);

  const getAdjList = (ed: Edge[]) => {

```



```

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
        dist = { ...dist, [edge.v]: uDist + edge.w };
        prev = { ...prev, [edge.v]: edge.u };
        updated = true;
        anyUpdate = true;
    }

    simulationSteps.push({
        iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
        distances: { ...dist }, previous: { ...prev },
        log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
            ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + dist[edge.v] : ''}
        `,
        activeCodeLine: updated ? 6 : 5,
    });
}

if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
        iteration: iter, checkingEdge: null, distances: { ...dist },
        log: `Ранний выход после итерации ${iter}.`,
        activeCodeLine: 10,
    });
    break;
}
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
    let cycleFound = false;
    for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
            cycleFound = true;
            simulationSteps.push({
                iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
                distances: { ...dist }, previous: { ...prev },
                log: `ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v}
                    создает цикл! dist[${edge.v}] = ${dist[edge.v]} < ${dist[edge.u] + edge.w}
                `,
                activeCodeLine: 15,
            });
        }
    }
    if (cycleFound) {
        simulationSteps[simulationSteps.length - 1].log +=
            `Цикл найден! dist[${edge.v}] = ${dist[edge.v]} < ${dist[edge.u] + edge.w}`;
        simulationSteps[simulationSteps.length - 1].activeCodeLine = 16;
    }
}

```

```

const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null;
const currentStep: Step = compilerLinked
  ? {
      ...baseStep,
      iteration: liveI ?? 0,
      checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
      distances: Object.fromEntries(nodes.map((node) => [node, 0])),
      log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} -> ${liveV}`
    }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border" >
    <div className="flex flex-wrap items-center justify-center" >
      <div className="flex items-center gap-3" >
        <div className="p-2.5 bg-blue-600 text-white" >
          <span className="text-2xl" > </span>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white" >
            <p className="text-sm text-blue-300" >Полный
          </p>
        </div>
      </div>
      <div className="flex items-center gap-3" >
        <div className="bg-slate-900 p-1 rounded-lg border" >
          <button onClick={() => setHasCycle(false)} >
            <span className="bg-blue-600 text-white" : 'text-slate-400' >
          <button onClick={() => setHasCycle(true)} >
            <span className="bg-rose-600 text-white" : 'bg-slate-400' >
          </div>
          <div>
            <button onClick={() => { setIsPlaying(false);
              <span className="bg-slate-600 text-white px-3 py-2 rounded-lg" >
            <button onClick={() => setIsPlaying(!isPlaying)} >
              <span className="bg-slate-600 text-white px-3 py-2 rounded-lg" >
            <button onClick={() => { setIsPlaying(false);
              <span className="bg-slate-600 text-white px-3 py-2 rounded-lg" >
            </div>
          </div>
        </div>
      </div>
    </div>
  )

```

```

        let strokeColor = '#475569';
        let marker = 'url(#arrow-bf-normal)';
        if (isChecking) { strokeColor = '#f59e0b' }
        else if (currentStep.hasNegativeCycle &&

        return (
            <g key={idx}>
                <line x1={uNode.x} y1={uNode.y} x2={vNode.x}
                    strokeDash={currentStep.hasNegativeCycle && isCyclePart ? 4 : 2}
                    marker={currentStep.isChecking ? '4,4' : 'none'} />
                <circle cx={({uNode.x + vNode.x} / 2)}
                    stroke={currentStep.isChecking ? '#f59e0b' : isNeg ? '#f43f5e' :
                        '#475569'} strokeDash={currentStep.isChecking ? 4 : 2}
                    strokeWidth={2} />
                <text x={({uNode.x + vNode.x} / 2)} y={
                    currentStep.isChecking ? 8fafc' } fontSize="11" fontWeight="bold" textAnchor="middle" />
            </g>
        );
    });
}}}

{nodes.map((node) => {
    const dist = currentStep.distances[node.id];
    const isCheckingNode = currentStep.checkingNode === node.id;

    return (
        <g key={node.id} className="transition-appear">
            <circle cx={node.x} cy={node.y} r={isCheckingNode ? '#7dd3fc' : '#64748b'}
                stroke="#475569" strokeDash={isCheckingNode ? 4 : 2} strokeWid
            <text x={node.x} y={node.y + 5} fill="#475569" />
            <text x={node.x} y={node.y - 28} fill="#475569" fontW
                ight="bold" textAnchor="middle" className="label" />
            <text x={node.x} y={node.y + 35} fill="#475569" />
            <text x={node.x} y={node.y + 35} fill="#475569" />
            <split(' ')[0]></text>
        </g>
    );
});
}}}
</svg>

```

```

        return (
          <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`>
            <div className="text-xs text-slate
            <div className={`font-mono font-bo
          v>
            </div>
          );
        }}}
      </div>
    </div>
  </div>

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60
    <div>
      <h4 className="text-lg font-bold text-slate
      <div className="bg-slate-950/80 rounded-lg p
        {codeLines.map(item => {
          <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
          <span className="text-slate-600 w-5 s
          <span className="whitespace-pre flex-
        </div>
      })}
    </div>
  </div>
</div>
</div>
</div>
</div>
);
}

```

```

steps.push({
  activeLine: 1,
  queue: [],
  visited: [],
  currentNode: null,
  logs: "Инициализация: начинаем с корня (0). "
});

queue.push(0);
visited.add(0);
steps.push({
  activeLine: 2,
  queue: [...queue],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Добавляем стартовую вершину 0 в очередь и по
});

while (queue.length > 0) {
  steps.push({
    activeLine: 4,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: null,
    logs: `Очередь не пуста, элементов: ${queue.length}
  });

  const curr = queue.shift()!;
  steps.push({
    activeLine: 5,
    queue: [...queue],
    visited: Array.from(visited),
    currentNode: curr,
    logs: `Достаем из очереди (${curr}).`
  });

  const neighbors = ADJ[curr];
  if (neighbors.length > 0) {

```

```
      logs: "Очередь пуста. Алгоритм завершен!"
    });

    return steps;
  };

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);

  useEffect(() => {
    if (!isPlaying) return;
    if (stepIdx >= STEPS.length - 1) {
      setIsPlaying(false);
      return;
    }
    const timer = setTimeout(handleNext, 1200);
    return () => clearTimeout(timer);
  }, [isPlaying, stepIdx, handleNext]);

  return (
    <div className="bg-slate-900 border-2 border-slate-700 p-10">
      <div className="flex flex-col md:flex-row items-center justify-center gap-10">
        <div>
          <h3 className="text-2xl font-bold text-white">Шаг {stepIdx + 1} из {S
```

```

    const n1 = NODES.find(n => n.id === e.from)
    const n2 = NODES.find(n => n.id === e.to)
    const isVisited = step.visited.includes(n)
    return (
      <line
        key={idx}
        x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
        className={`transition-all duration-500`}
      >
    )
  )
}

```

```

{/* Вершины */}
{NODES.map(n => {
  const isCurrent = step.currentNode === n
  const isInQueue = step.queue.includes(n.id)
  const isVisited = step.visited.includes(n.id)

  let fill = '#1e293b'; // slate-800
  let stroke = '#475569'; // slate-600
  let scale = 1;
  void scale;

  if (isCurrent) {
    fill = '#3b82f6'; // blue-500
    stroke = '#60a5fa'; // blue-400
    scale = 1.3;
  } else if (isInQueue) {
    fill = '#059669'; // emerald-600
    stroke = '#34d399'; // emerald-400
    scale = 1.1;
  } else if (isVisited) {
    fill = '#0f172a'; // slate-900
    stroke = '#3b82f6'; // blue-500
  }

  return (

```

```

        { line: 2, code: 'queue.push(start); vi
        { line: 3, code: ' ' },
        { line: 4, code: 'while not queue.isEmp
        { line: 5, code: '   node = queue.pop()'
        { line: 6, code: '   for neighbor in gra
        { line: 7, code: '       if neighbor not in
        { line: 8, code: '           visited.add(ne
        { line: 9, code: '           queue.push(nei
    ].map((cl) => {
      const isActive = step.activeLine === cl
      return (
        <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5' : 'te
        <span className="opacity-40 w-6 inl
        <span className="whitespace-pre">{c
      </div>
    );
  }}}
</div>
</div>

<div className="bg-slate-800/80 border border
  <div className="text-sm font-bold text-eme
    <span>Состояние Очереди (Queue):</span>
    <span className="text-xs text-slate-500
  </div>
  <div className="flex items-center gap-2 mi
    {step.queue.length === 0 ? (
      <span className="text-slate-500 text-x
    ) : (
      step.queue.map((qId, i) => (
        <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
        {qId}
      </span>
    ))
  )}
</div>

```



```
      alt: 'Все ко Всем Флойд',
      caption: 'Все связаны со всеми!',
      borderColor: 'border-emerald-500/30',
      captionColor: 'text-emerald-400',
    },
  ];

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-emerald-500`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold text-emerald-400`} >
        {info.caption}
      </p>
    </div>
  );
}
```

ФАЙЛ 34/96: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  compact?: boolean;
}
```

```

<nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" style={{width: 1000px, height: 100px; border: 1px solid #ccc; padding: 10px; margin: 10px auto;}}>
  {prev ? (
    <button
      onClick={() => onGo(prev.id)}
      className="group text-left p-4 rounded-xl bg-white border border-gray-200 transition-all"
    >
      <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
        <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
      </div>
      <div className="text-sm font-bold text-slate-700">
        {prev.title}
      </div>
    </button>
  ) : (
    <div className="hidden sm:block" />
  )}

  {next && (
    <button
      onClick={() => onGo(next.id)}
      className="group text-right p-4 rounded-xl bg-white border border-gray-200 transition-all sm:col-start-2"
    >
      <div className="flex items-center justify-end text-indigo-400 mb-1">
        Следующая тема <ChevronRight className="w-3.5 h-3.5" />
      </div>
      <div className="text-sm font-bold text-slate-700">
        {next.title}
      </div>
    </button>
  )}
</nav>
);
};

```

```

interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
  /** Перейти во вкладку компилятора – там уже подставлен код */
  onOpenCompiler?: () => void;
}

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;

export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");

  useTermHints(contentRef, [chapter.id]);

  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);

  const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
  }, [cancelHide]);

  useEffect(() => {
    setAnchor(null);
    setSaving("idle");
  }, [chapter.id]);

```

```

    e.preventDefault();
    if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
  };

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>('a');
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      {/* панель загрузки: сохранить именно эту страницу */}
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small border border-gray-500 px-3 py-2"
          300 hover:text-white hover:border-emerald-500
        >
          {saving === "work" ? (
            <Loader2 className="w-3.5 h-3.5 animate-spin" />
          ) : saving === "done" ? (
            <Check className="w-3.5 h-3.5 text-emerald-500" />
          ) : (
            <Download className="w-3.5 h-3.5" />
          )}
          {saving === "done" ? "Файл сохранён" : "Скачать"}
        </button>
        <button
          onClick={() => window.print()}
          title="Распечатать или сохранить в PDF средствами браузера"
          className="flex items-center gap-1.5 text-x-small border border-gray-500 px-3 py-2"
          300 hover:text-white hover:border-indigo-500
        >
          <PrintIcon className="w-3.5 h-3.5" />
          {saving === "done" ? "Печать" : "Печать"}
        </button>
      </div>
    </article>
  </>
);

```

```

        <Maximize2 className="w-3.5 h-3.5" />
      </button>
      {onOpenCompiler && (
        <button
          onClick={onOpenCompiler}
          title="Открыть Python: код выполняе
рации"
          className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
        >
          <Terminal className="w-3.5 h-3.5" />
        </button>
      )}
    </div>
  </div>
  {viz.hint && <p className="text-xs text-sla
  <VizChapterContext.Provider value={chapter.
    <viz.Component />
  </VizChapterContext.Provider>
</section>
)}

  <RelatedTickets chapterId={chapter.id} onGo={on

  <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>

```

```

    [0, 1], [1, 2], [2, 3], [3, 4], [4, 0],
    [0, 5], [1, 5], [2, 5], [3, 5], [4, 5],
    [2, 6], [3, 6], [5, 6],
  ];

const ADJ: number[][] = (() => {
  const adj: number[][] = NODES.map(() => []);
  EDGES.forEach([u, v] => {
    adj[u].push(v);
    adj[v].push(u);
  });
  return adj;
})();

const COLORS = [
  { name: "красный", hex: "#ef4444" },
  { name: "синий", hex: "#3b82f6" },
  { name: "зелёный", hex: "#22c55e" },
  { name: "жёлтый", hex: "#eab308" },
];

const CONFLICT = "#e11d48";

/** Жадная раскраска по возрастанию номеров – честный у
function greedy(): number[] {
  const color: number[] = NODES.map(() => -1);
  for (let v = 0; v < NODES.length; v++) {
    const used = new Set(ADJ[v].map(u => color[u]).filter(c => c !== -1));
    let c = 0;
    while (used.has(c)) c++;
    color[v] = c;
  }
  return color;
}

export function ColoringViz() {
  const [colors, setColors] = useState<number[]>(NODES.map(() => -1));
  const [brush, setBrush] = useState(0);

```

```

    if (i >= 0) return i;
  }
  if (liveVNum !== null && liveVNum >= 0 && liveVNum <= 0)
    return null;
})();

const conflicts = useMemo(() => {
  const bad = new Set<number>();
  EDGES.forEach(([u, v]) => {
    const cu = shownColors[u];
    const cv = shownColors[v];
    if (cu >= 0 && cu === cv) {
      bad.add(u);
      bad.add(v);
    }
  });
  return bad;
}, [shownColors]);

const usedColors = useMemo(
  () => new Set(shownColors.filter((c) => c >= 0)).size,
  [shownColors]
);

const paint = (id: number) => {
  const next = [...colors];
  next[id] = next[id] === brush ? -1 : brush;
  setColors(next);
};

return (
  <div className="bg-slate-800/90 p-5 rounded-2xl border">
    <div className="flex flex-wrap items-center justify-center">
      <div>
        <h3 className="text-lg font-bold text-white">
          <p className="text-xs text-slate-400">
            Смежные вершины не должны совпадать по цвету
          </p>

```

```

        {n.label}
      </text>
      {c >= 0 && {
        <text x={n.x} y={n.y - 20} textAnchor=
vents-none">
          color[{n.id}]= {c}
        </text>
      }}
    </g>
  );
}}
</svg>
{compilerLinked && {
  <div className="absolute top-2 right-2 bg-emerald-300 z-20">
    Python красит: {liveVLabel ?? ({liveVNum
  </div>
}}
{conflicts.size > 0 && {
  <div className="absolute bottom-2 left-1/2
-[11px] text-rose-300 z-20">
    ▲ Конфликт: смежные вершины одного цвета
  </div>
}}
</div>

<div className="space-y-3">
  <div className="bg-slate-950 rounded-xl border"
    <div className="text-[10px] uppercase tracking
      <Palette className="w-3 h-3" /> Кисть
    </div>
    <div className="grid grid-cols-4 gap-2">
      {COLORS.map(({c, i}) => {
        <button
          key={c.name}
          onClick={() => setBrush(i)}
          title={c.name}
          className={`h-8 rounded-lg border-2 t

```



```
-----
* Диаграмма вложенности классов  $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ 
* полиномиальных сведений  $A \leq_p B$ : если B решается легко
*/
```

```
type ClassId = "P" | "NP" | "PSPACE" | "EXPTIME";
```

```
interface Problem {
  id: string;
  name: string;
  cls: ClassId;
  x: number;
  y: number;
  npComplete?: boolean;
}
```

```
const PROBLEMS: Problem[] = [
  { id: "sort", name: "сортировка\n $O(n \log n)$ ", cls: "P", x: 15, y: 150 },
  { id: "mst", name: "остов (Краскал)", cls: "P", x: 15, y: 175 },
  { id: "2sat", name: "2-SAT", cls: "P", x: 115, y: 185 },
  { id: "shortpath", name: "Дейкстра / Флойд", cls: "P", x: 15, y: 200 },
  { id: "sat", name: "3-SAT", cls: "NP", x: 330, y: 120 },
  { id: "vc", name: "Vertex Cover", cls: "NP", x: 385, y: 150 },
  { id: "ham", name: "Гамильтонов цикл", cls: "NP", x: 385, y: 175 },
  { id: "knapsack", name: "Рюкзак (точный)", cls: "NP", x: 385, y: 200 },
  { id: "circuit", name: "QBF / игра на формуле", cls: "EXPTIME", x: 385, y: 225 },
  { id: "chess", name: "обобщённые шахматы", cls: "EXPTIME", x: 385, y: 250 },
];
```

```
const REDUCTIONS: [string, string, string][] = [
  ["sat", "vc", "3-SAT  $\leq_p$  VC"],
  ["sat", "ham", "3-SAT  $\leq_p$  Гамильтон"],
  ["vc", "knapsack", "VC  $\leq_p$  Рюкзак"],
];
```

```
const CLASS_META: Record<ClassId, { title: string; color: string; note: string }> = {
  P: { title: "P", color: "#34d399", note: "решаются за полиномиальное время" },
  NP: { title: "NP", color: "#fbbf24", note: "решение не обязательно полиномиальное" },
  PSPACE: { title: "PSPACE", color: "#818cf8", note: "нужно полиномиальная память" },
  EXPTIME: { title: "EXPTIME", color: "#fbbf24", note: "нужно экспоненциальное время" },
};
```

```

const activeCls: ClassId | null =
  compilerLinked && liveCls
    ? (Object.keys(CLASS_META).find((k) => k.toLowerCase() === selected)
      : selected;

const problemHit = (p: Problem) => {
  if (compilerLinked && liveTask) return p.name.toLowerCase() === selectedProblem;
  return selectedProblem === p.id;
};

const infoText = (() => {
  if (compilerLinked && liveTask) {
    const p = PROBLEMS.find((q) => q.name.toLowerCase() === selectedProblem);
    if (p) return `${p.name.replace("\n", " ")} ∈ ${p.cls}`;
  }
  if (compilerLinked && liveCls) {
    const cls = (activeCls ?? "NP") as ClassId;
    return CLASS_INFO[cls];
  }
  if (selectedProblem) {
    const p = PROBLEMS.find((q) => q.id === selectedProblem);
    return `${p.name.replace("\n", " ")} ∈ ${p.cls}${p.cls === selectedProblem ? "" : ".cls"]}`;
  }
  if (activeCls) return CLASS_INFO[activeCls as ClassId];
  return "Кликни на класс или задачу. «A ≤p B» = «A не ≤p B»";
})();

const reset = () => {
  setSelected(null);
  setSelectedProblem(null);
};

return (
  <div className="bg-slate-800/90 p-5 rounded-2xl border">
    <div className="flex flex-wrap items-center justify-center">
      <div>
        <h3 className="text-lg font-bold text-white">

```

```

        ? (liveRedPair[0] && a.name.toLowerCase
          liveRedPair[1] && b.name.toLowerCase
          : selectedProblem === from || selectedP
const mx = (a.x + b.x) / 2;
const my = (a.y + b.y) / 2 - 4;
return (
  <g key={`${from}-${to}`} opacity={hot ?
    <line x1={a.x} y1={a.y} x2={b.x} y2={b
ay="5 4" markerEnd="url(#arrowhead)" />
    <text x={mx} y={my} textAnchor="middle
text>
    </g>
  );
}})
<defs>
  <marker id="arrowhead" markerWidth="7" ma
    <polygon points="0 0, 7 2.5, 0 5" fill=
  </marker>
</defs>

{PROBLEMS.map((p) => {
  const dim = activeCls !== null && activeC
  const hit = problemHit(p);
  return (
    <g key={p.id} onClick={() => setSelecte
      <rect
        x={p.x - 46} y={p.y - 13} width={92
        fill={hit ? "#c2410c" : p.npComple
        stroke={hit ? "#fb923c" : CLASS_MET
        strokeWidth={hit ? 2.5 : 1.2}
      />
      {p.name.split("\n").map((line, li, ar
        <text key={li} x={p.x} y={p.y + 4 +
ontWeight="600">
        {line}
      </text>
    )}}
  </g>

```

```

        </div>
    </div>
</div>
</div>
);
}

```

ФАЙЛ 38/96: src/components/ComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useEffect, useMemo, useState } from "react";
import { Pause, Play, RotateCcw, SkipForward } from "lucide-react";
import { useVizRuntime, vizArray, vizNumber, vizRecord, vizText } from "viz";
```

```
/**
 * Тема 8 «Графы. Компоненты связности».
 * Обход в ширину помечает каждую вершину номером её ко
 * сколько раз пришлось запускать обход – столько и ком
 */
```

```
interface V {
  id: number;
  label: string;
  x: number;
  y: number;
}
```

// Демо-граф: три «острова» по 3–4 вершины.

```
const NODES: V[] = [
    { id: 0, label: "A", x: 60, y: 60 },
    { id: 1, label: "B", x: 170, y: 30 },
    { id: 2, label: "C", x: 150, y: 130 },
    { id: 3, label: "D", x: 40, y: 160 },
    { id: 4, label: "E", x: 300, y: 60 },
    { id: 5, label: "F", x: 410, y: 90 },
```

```

frames.push({ comp: [...comp], count, ...f });

push({ queue: [], current: null, scanning: null, log:
  ной вершины." });

for (let s = 0; s < NODES.length; s++) {
  if (comp[s] !== null) continue;
  count += 1;
  const queue = [s];
  comp[s] = count - 1;
  push({ queue: [...queue], current: s, scanning: null,
    арует с ней.` });
  while (queue.length) {
    const v = queue.shift()!;
    push({ queue: [...queue], current: v, scanning: null,
      й.` });
    for (const to of ADJ[v]) {
      if (comp[to] === null) {
        comp[to] = comp[v];
        queue.push(to);
        push({ queue: [...queue], current: v, scanning: null,
          {comp[v]}, в очередь.` });
      } else {
        push({ queue: [...queue], current: v, scanning: null,
          });
      }
    }
  }
  push({ queue: [], current: null, scanning: null, log:
})
push({ queue: [], current: null, scanning: null, log:
  });
return frames;
}

const FRAMES = buildFrames();

export function ComponentsViz() {

```

```

    }
    return liveVNum !== null && liveVNum >= 0 && liveVNum < liveVNum;
  };
  const liveCurrent = liveIndexOf();

  const liveCompOf = (n: V): number | null => {
    if (liveCompByLabel && n.label in liveCompByLabel)
      const c = vizNumber(liveCompByLabel[n.label]);
      if (c !== null) return c;
    }
    if (liveCompByLabel && String(n.id) in liveCompByLabel)
      const c = vizNumber(liveCompByLabel[String(n.id)]);
      if (c !== null) return c;
    }
    return frame.comp[n.id];
  };
  const liveQueueLabels = (liveQ
    ? liveQ.map((x) => (typeof x === "string" ? x : vizNumber(x)))
    : null
  )?.filter(Boolean);
  const displayCount = liveCount ?? frame.count;
  const displayLog = compilerLinked
    ? `  Компилятор: v=${liveVLabel ?? liveVNum ?? "-"}
      ES[q].label)).join(", ")}]`
    : frame.log;

  const shownCurrent = compilerLinked ? liveCurrent : frame.current;
  const shownScanning = compilerLinked ? null : frame.scanning;
  const shownQueue: number[] = liveQueueLabels
    ? liveQueueLabels
      .map((lab) => NODES.findIndex((n) => n.label === lab))
      .filter((i) => i >= 0)
    : frame.queue;

  return (
    <div className="bg-slate-800/90 p-5 rounded-2xl border" >
      <div className="flex flex-wrap items-center justify-between" >
        <div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr]">
  <div className="bg-slate-950 rounded-xl border border-white" style={{width: 100%; height: 100%;}}>
    <div className="absolute inset-0 bg-[radial-gradient(circle, transparent 1px, transparent 1px)]" style={{width: 100%; height: 100%;}}>
      <svg className="w-full h-[320px] relative z-10" style={{width: 100%; height: 100%;}}>
        {EDGES.map(([u, v], i) => {
          const a = NODES[u];
          const b = NODES[v];
          const sameComp = liveCompOf(a) !== null && liveCompOf(b) !== null && liveCompOf(a) === liveCompOf(b);
          return (
            <line
              key={i}
              x1={a.x} y1={a.y} x2={b.x} y2={b.y}
              stroke={sameComp ? (COMP_COLORS[liveCompOf(a)] || "#334155") : "#334155"}
              strokeWidth={sameComp ? 3.5 : 2}
              opacity={sameComp ? 0.9 : 0.6}
            />
          );
        })}
        {NODES.map((n) => {
          const c = liveCompOf(n);
          const isCurrent = shownCurrent === n.id;
          const isScanned = shownScanning === n.id;
          const inQueue = shownQueue.includes(n.id);
          const color = c === null ? "#334155" : COMP_COLORS[c] || "#334155";
          return (
            <g key={n.id}>
              {(isCurrent || isScanned) && !inQueue && c !== null && !isCurrent && !isScanned && !inQueue ? (
                <circle cx={n.x} cy={n.y} r={18} fill={color} stroke="white" stroke-width="2" style={{animation: "pulse 1s ease-in-out 0s;"/>

```

```

        <div className="text-slate-400">{n}
        <div>{c === null ? "-" : c}</div>
      </div>
    );
  }}}
</div>
</div>
<div className="bg-slate-900/60 rounded-xl border"
  {displayLog}
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 39/96: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

```

```

interface CodeLine {
  line: string;
  highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
  { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
  { line: "    visited[v] = true;", highlight: "normal" },
  { line: "    tin[v] = low[v] = timer++;", highlight: "normal" },
  { line: "    for (int to : adj[v]) {", highlight: "normal" },
  { line: "        if (to == p) continue;", highlight: "normal" },
  { line: "        dfs(to, v);", highlight: "normal" },
  { line: "    }", highlight: "normal" },
  { line: "}", highlight: "normal" },
];

```



```
const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: [1,6], backEdge: null,
    log: "Из B идём в G. tin[G]=3, low[G]=3.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 1, visitedIndices: [0,1,6],
    tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
    activeEdge: null, backEdge: null,
    log: "У G нет соседей (кроме B). Возврат. low[G]=3 :"
```

```

currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
activeEdge: null, backEdge: [5,3],
log: "Из F видим D (посещён). Обратное ребро (F,D).",
activeCodeLine: [7,8,9]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
activeEdge: null, backEdge: null,
log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
activeCodeLine: [11,13,16,17]
},
{
currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
activeEdge: null, backEdge: null,
log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – не мо",
activeCodeLine: [13,14,15,16,17]
},
{
currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
activeEdge: null, backEdge: null,
log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – не мо",
activeCodeLine: [11,13,16,17]
},
{
currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},
activeEdge: null, backEdge: null,

```

```

        return prev + 1;
      });
    }, 2500);
  }
  return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <div className="flex items-center gap-1 bg-slate-700 rounded">
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
            disabled={dfsIdx === 0}
            className="p-1.5 hover:bg-slate-700 rounded"
            <Undo className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => setDfsAuto(!dfsAuto)}
            className={`px-2.5 py-1 rounded text-[10px]
              dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
            }`>
            {dfsAuto ? <><Pause className="h-3 w-3" /> : <><Play className="h-3 w-3" />}
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1);
            disabled={dfsIdx === DFS_STEPS.length-1}
            className="p-1.5 hover:bg-slate-700 rounded"
            <SkipForward className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
            className="p-1.5 hover:bg-slate-700 rounded"
            <RefreshCw className="h-3.5 w-3.5" />
          </button>
        </div>
      </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-5 gap-4">

```

```

        fill={cur ? "#10b981" : st ? "#064e3b" : "#000000"}
        stroke={cur ? "#fff" : st ? "#34d399" : "#000000"}
        strokeWidth={2.5}
        className="transition-all duration-300"
      <text x={n.x} y={n.y+3} textAnchor="middle">
        {label}</text>
      <vis && (
        <rect x={n.x-18} y={n.y-28} width={100} height={100}
        className="transition-all duration-300" />
        <text x={n.x} y={n.y-20} textAnchor="center">
        {l:{step.low[n.id]||"?"}</text>
      </>
    )}
  </g>
  </>
</svg>
<div className="flex items-center justify-between">
  <span>War {dfsIdx+1}/{DFS_STEPS.length}</span>
  <div className="flex-1 mx-2 bg-slate-950 h-10 rounded-lg">
    <div className="bg-indigo-600 h-full transition-all duration-300">
    </div>
  </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
  {/* Code panel */}
  <div className="bg-slate-950 rounded-lg border border-slate-800">
    <div className="bg-slate-900 px-3 py-1.5 text-slate-300">
    </div>
    <div className="p-2 overflow-x-auto">
      <pre className="text-[10px] font-mono leading-relaxed">
        {CODE_LINES.map((cl, i) => {
          const hl = step.activeCodeLine.includes(cl)
          return <div key={i}
            className={`px-1 transition-all duration-300 ${hl ? 'bg-slate-800' : 'bg-slate-900'}>
            {cl}</div>
        )}</pre>
      </div>
    </div>
  </div>
</div>

```

```
                {step.bridges.length > 0 ? step.bridges
            </span>
        </div>
    </div>
</div>
</div>
</div>
    );
}
```

ФАЙЛ 40/96: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord, vizText } from 'viz';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
  ];
```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false };
  const prev: Record<string, string | null> = { A: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из ПИ
    activeCodeLine: 2,
  });

  for (let iter = 0; iter < nodes.length; iter++) {
    let minD = 999;
    let u = null;
    for (const n of nodes) {
      if (!visited[n.id] && dist[n.id] < minD) {
        minD = dist[n.id];
        u = n.id;
      }
    }

    if (!u) break;

    visited[u] = true;
    simulationSteps.push({
      currentNode: u, checkingEdge: null,
      distances: { ...dist }, visited: { ...visited },
      log: ` Выбираем вершину ${u} с мин. непосещённ
      activeCodeLine: 5,
    });
  }
});

```

```

    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
      timer = setTimeout(() => {
        setCurrentStepIndex((prev) => prev + 1);
      }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked
  ? {
    ...baseStep,
    currentNode: liveU,
    checkingEdge: liveU && liveV ? { u: liveU, v: l
    distances: Object.fromEntries(nodes.map((node) :
    previous: Object.fromEntries(nodes.map((node) =>
    visited: Object.fromEntries(nodes.map((node) =>
    log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
  }
  : baseStep;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl bor
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justifi
      <div className="flex items-center gap-3">

```

```

<div className="w-full lg:w-2/3 bg-indigo-950/20
  justify-center min-h-[400px]">
  <div className="absolute top-3 left-3 bg-indigo-950/20
    font-bold shadow-sm">
    Шаг {currentStepIndex + 1} из {steps.length}
  </div>

  <svg className="w-full h-auto max-h-[500px]" >
    <defs>
      <marker id="arrow-dh-normal" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
      </marker>
      <marker id="arrow-dh-active" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
      </marker>
      <marker id="arrow-dh-opt" markerWidth="6"
        <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
      </marker>
    </defs>
    {/* Рисуем */}
    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === edge.u)
      const vNode = nodes.find((n) => n.id === edge.v)
      const isChecking =
        currentStep.checkingEdge === edge.id &&
        (currentStep.checkingEdge.u === edge.u &&
          currentStep.checkingEdge.v === edge.v)

      const isOptimal = currentStep.previousEdge === edge.id

      let marker = 'url(#arrow-dh-normal)';
      if (isChecking) marker = 'url(#arrow-dh-active)';
      else if (isOptimal) marker = 'url(#arrow-dh-opt)';

      return (
        <g key={idx}>
          <line
            x1={uNode.x} y1={uNode.y}
            x2={vNode.x} y2={vNode.y}
            stroke={isChecking ? 'red' : isOptimal ? 'green' : 'black'}
            stroke-width={isChecking ? 2 : 1}
          />
          <marker
            href={marker}
            dx={vNode.x - uNode.x}
            dy={vNode.y - uNode.y}
            style={{position: 'absolute', left: 0, top: 0}}
          />
        </g>
      )
    })}
  </svg>

```



```

        <text x={node.x} y={node.y + 5} fill=
            {node.id}
        </text>
        <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
            {dist === 999 ? '∞' : `d=${dist}`}
        </text>
        <text x={node.x} y={node.y + 35} fill=
            {node.name.split(' ')[0]}
        </text>
    </g>
  );
  }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
  <p className="font-semibold text-amber-400">
  <p className="min-h-[40px] flex items-center">
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50">
  <h4 className="text-lg font-bold text-emerald-400">
  <div className="space-y-2.5 flex-1 overflow-y-hidden">
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u;
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
        }`} >
          <div className="flex items-center gap-2">
            <span className={`px-2 py-0.5 rounded ${
              isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
            }`} >
              {u}
            </span>
            <span className="text-slate-400">→</span>
          </div>
        </div>
      );
    })}
  </div>

```

```

        const dist = currentStep.distances[n.id]
        const prev = currentStep.previous[n.id]
        const vis = currentStep.visited[n.id]
        const isCurr = currentStep.currentNode === n.id

        return (
          <tr key={n.id} className={`border-1 ${
            isCurr ? 'bg-indigo-950/60 font-medium' : ''
          }`} >
            <td className="py-2.5 px-3 flex items-center">
              <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-950/60' : 'bg-slate-900/60'}`} />
              {n.name}
            </td>
            <td className="py-2.5 px-3 font-medium">
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-slate-400">
              {prev || '-'}
            </td>
          </tr>
        );
      });
    </tbody>
  </table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-400">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map((item) => {
        const isActive = currentStep.activeCodeLine === item.line
        return (
          <div key={item.line} className={`py-1 px-2 ${
            isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'
          }`} >

```

```

-----
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];

```

```

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const runtime = useVizRuntime();
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

```

```

useEffect(() => {
  const generatedSteps: DPStep[] = [];
  let stepId = 0;
  const memoMap: { [key: number]: number } = {};

```

```

  function simulateFib(n: number): number {
    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вызов fibМемо(${n}). Проверяем наличие в
      codeLine: 2,
      memoState: { ...memoMap }
    });

```

```

    if (n in memoMap) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'memo_hit',
        desc: `⚡ Кэш-хит! fibМемо(${n}) уже посчитано
        codeLine: 2,
        memoState: { ...memoMap }

```

```

    return memoMap[n];
  }

  const finalRes = simulateFib(targetN);

  generatedSteps.push({
    id: stepId++,
    n: targetN,
    action: 'done',
    desc: `  Расчет завершен! fibMemo(${targetN}) = $
    codeLine: 6,
    memoState: { ...memoMap }
  });

  setSteps(generatedSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [targetN]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length -
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
  ? Object.fromEntries(Object.entries(liveMemo).flatM

```

```

        <option value={4}>N = 4</option>
        <option value={5}>N = 5</option>
        <option value={6}>N = 6</option>
        <option value={7}>N = 7</option>
      </select>
    </div>

    <div className="flex items-center bg-slate-950" >
      <button
        onClick={() => { setIsPlaying(false); set
        className="p-2 text-slate-400 hover:text-
        title="Сбросить"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
      <button
        onClick={() => { setIsPlaying(false); set
        disabled={currentStepIndex === 0}
        className="p-2 text-slate-400 hover:text-
        title="Шаг назад"
      >
        <ChevronLeft className="w-5 h-5" />
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1.5 px-
          isPlaying
            ? 'bg-amber-500 text-slate-950 hover:
            : 'bg-emerald-600 text-white hover:
          }`
      >
        {isPlaying ? <Pause className="w-4 h-4" />
        {isPlaying ? 'Пауза' : 'Пуск'}}
      </button>
      <button
        onClick={() => { setIsPlaying(false); set
        disabled={currentStepIndex === steps.length
        className="p-2 text-slate-400 hover:text-

```

```

        {currentStep && (
            <div className="mt-6 pt-4 border-t border-slate-500">
                <div className="text-sm font-medium text-slate-500">
                    <span className="inline-block w-2 h-2 rounded-full bg-current step" />
                    <span>{currentStep.desc}</span>
                </div>
                <div className="text-xs font-mono bg-slate-500 text-white p-2">
                    Текущий N: <strong className="text-emerald-400">{currentStep.n}</strong>
                </div>
            </div>
        )}
    </div>

    {/* Tabs */}
    <div className="flex items-center gap-2 border-b border-slate-500">
        <button
            onClick={() => setActiveTab('table')}
            className={`flex items-center gap-2 px-6 py-3 border ${
                activeTab === 'table'
                    ? 'border-emerald-500 text-emerald-400 bg-slate-500'
                    : 'border-transparent text-slate-400 hover:text-slate-300'
            }`}
        >
            <Eye className="w-4 h-4" />
            Шаг-бай-шаг состояние
        </button>
        <button
            onClick={() => setActiveTab('code')}
            className={`flex items-center gap-2 px-6 py-3 border ${
                activeTab === 'code'
                    ? 'border-emerald-500 text-emerald-400 bg-slate-500'
                    : 'border-transparent text-slate-400 hover:text-slate-300'
            }`}
        >
            <Code className="w-4 h-4" />
            Интерактивный код
        </button>
    </div>

```

```

        >
        <span className="w-8 text-xs text-slate-500">
        <span>{line}</span>
        </div>
    );
  }}}
</pre>
<div className="p-6 bg-slate-900/50 border-slate-800" >
  <span className="p-1.5 bg-emerald-500/20 rounded">
  <span>{currentStep?.desc}</span>
</div>
</div>
  })
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800" >
  <div className="flex items-center justify-between" >
    <span>Прогресс выполнения</span>
    <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
  </div>
  <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
    <div
      className="h-full bg-gradient-to-r from-emerald-500 to-emerald-400"
      style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
    ></div>
  </div>
</div>
</div>
  );
}

```

```

    if (!compilerLinked) return null;
    const ids = livePathLabels!
      .map((lab) => C1_NODES.find((n) => n.label === lab))
      .filter((x): x is number => x !== undefined);
    return ids;
  }, [compilerLinked, livePathLabels]);
const shownPath = livePath ?? c1Path;
const shownVisited = useMemo(() => {
  if (livePath) {
    const keys: string[] = [];
    for (let i = 1; i < livePath.length; i++) {
      const a = livePath[i - 1];
      const b = livePath[i];
      if (C1_EDGES.some((e) => (e.u === a && e.v === b))) {
        keys.push(`${Math.min(a, b)}-${Math.max(a, b)}`);
      }
    }
    return keys;
  }
  return c1VisitedEdges;
}, [livePath, c1VisitedEdges]);

const degrees = useMemo(
  () => C1_NODES.map((n) => ({ label: n.label, deg: C1_DEGREES[n.label] })),
  []
);

const clickC1 = (vId: number) => {
  if (c1Done && c1Path.length > 0) return;
  if (c1Path.length === 0) {
    setC1Path([vId]);
    setC1Msg("Старт! Кликай соседние вершины, чтобы построить путь");
    return;
  }
  const last = c1Path[c1Path.length - 1];
  const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
  if (!edge) {
    setC1Msg("Нет ребра между этими вершинами! Выбери другую вершину");
  }
};

```



```

-hidden">
<div className="absolute inset-0 bg-[radial-gra
<svg className="w-full h-[260px] z-10 relative"
  {C1_EDGES.map((e, i) => {
    const u = C1_NODES.find(n => n.id === e.u)!
    const v = C1_NODES.find(n => n.id === e.v)!
    const key = `${Math.min(e.u,e.v)}-${Math.ma
    const used = shownVisited.includes(key);
    return <line key={i} x1={u.x} y1={u.y} x2={
      stroke={used ? "#6366f1" : "#475569"}
      strokeWidth={used ? 4 : 2}
      className="transition-all duration-300" />
  })}
  {C1_NODES.map(n => {
    const isLast = shownPath[shownPath.length-1
    const visited = shownPath.includes(n.id);
    return <g key={n.id} className="cursor-poin
      {isLast && <circle cx={n.x} cy={n.y} r={1
      <circle cx={n.x} cy={n.y} r={11}
        fill={isLast ? "#6366f1" : visited ? "#
        stroke={isLast ? "#fff" : visited ? "#6
        strokeWidth={2.5}
        className="transition-all duration-200
      <text x={n.x} y={n.y+4} textAnchor="middl
    bel}</text>
  </g>;
  })}
</svg>
<div className="absolute top-2 left-2 bg-slate-
  Ребѐр: {shownVisited.length}/{C1_EDGES.length
  {compilerLinked && <span className="ml-2 text
</div>
<div className="absolute bottom-2 left-2 bg-sla
  0">
  Степени: {degrees.map((d) => `${d.label}:${d.r
</div>
</div>

```

```
    'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе полу  
    'Мы вызываем алгоритм Дейкстры для поиска нового  
  ],  
  correctAnswer: 2,  
  explanation: 'Верно! Если вершины уже в одном клане  
    Это нарушит структуру дерева.'  
},  
{  
  id: 2,  
  question: 'Почему алгоритм Дейкстры впадает в ступор  
  options: [  
    'Потому что он написан только для положительных ч  
    'Он жадный и считает, что каждый следующий шаг де  
      закрытые вершины.',  
    'Отрицательные ребра создают гравитационный колла  
    'Дейкстра просто не любил отрицательные балансы н  
  ],  
  correctAnswer: 1,  
  explanation: 'Именно! Дейкстра уверен: если ты прие  
    ля чего нужен алгоритм Беллмана-Форда.'  
},  
{  
  id: 3,  
  question: 'Какова ключевая фишка алгоритма Борувки  
  options: [  
    'Он использует меньше памяти благодаря коммунисти  
    'Он умеет работать с отрицательными весами без ци  
    'Он позволяет выполнять шаги слияния колхозов (ко  
    'Он был разработан в Токио и поэтому самый быстры  
  ],  
  correctAnswer: 2,  
  explanation: 'Абсолютно верно! В Борувке каждая ком  
    лелить вычисления на GPU или многопроцессорных кл  
},  
{  
  id: 4,  
  question: 'В чем главная суть Динамического програм  
  options: [  
    'Он использует меньше памяти благодаря коммунисти
```

```

    }
  };

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption(null);
      setIsAnswered(false);
    } else {
      setShowResults(true);
    }
  };

  const handleRestart = () => {
    setCurrentQIndex(0);
    setSelectedOption(null);
    setIsAnswered(false);
    setScore(0);
    setShowResults(false);
  };

  if (showResults) {
    return (
      <div className="bg-slate-900/90 border border-slate-800 p-12">
        <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
          <Award className="w-10 h-10 text-white" />
        </div>
        <h3 className="text-3xl font-bold text-white mb-4">Поздравляем!</h3>
        <p className="text-slate-400 text-sm mb-6">Ты правильно ответил на все вопросы!</p>

        <div className="bg-slate-950 border border-slate-800 p-12">
          <p className="text-slate-400 text-sm uppercase">Итоги викторины</p>
          <p className="text-5xl font-black text-indigo-400">Поздравляем!</p>
          <p className="text-slate-300 text-sm">
            {score === quizQuestions.length
              ? ' Идеально! Ты настоящий сеньор-раскрасник!'
            }
          </p>
        </div>
      </div>
    );
  }
}

```

```

<div className="space-y-4">
  {currentQ.options.map((option, idx) => {
    const isSelected = selectedOption === idx;
    const isCorrect = idx === currentQ.correctAnswer

    let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90"
    if (isAnswered) {
      if (isCorrect) {
        optionStyle = "bg-emerald-950/80 border-emerald-500"
      } else if (isSelected) {
        optionStyle = "bg-rose-950/80 border-rose-500"
      } else {
        optionStyle = "bg-slate-800/40 border-slate-800"
      }
    }

    return (
      <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4 rounded-lg"}
      >
        <div className="mt-0.5 shrink-0">
          {isAnswered && isCorrect ? (
            <CheckCircle2 className="w-6 h-6 text-emerald-500" />
          ) : isAnswered && isSelected && !isCorrect ? (
            <XCircle className="w-6 h-6 text-rose-500" />
          ) : (
            <div className={"w-6 h-6 rounded-full " + optionStyle}
              order-indigo-500 bg-indigo-500 text-white"
            >
              {String.fromCharCode(65 + idx)}
            </div>
          )}
        </div>
        <div className="flex-1 text-sm md:text-base">
          {option}
        </div>
      </div>
    )
  })}
</div>

```

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';

interface Step {
  k: number; i: number; j: number;
  matrix: (number | null)[][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const runtime = useVizRuntime();
  // k/i/j/d приходят напрямую; номер строки произвольный
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
    { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
    { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
  ];

  const initialMatrix = [
    [0, 4, 999, 2, 999, 999, 999],
    [999, 0, 3, 999, 999, 3, 999],
    [999, 999, 0, 999, 2, 999, 999],
    [999, 999, 999, 0, 4, 2, 999],
    [999, 999, 999, 999, 0, 999, 1],
    [999, 999, 999, 999, 999, 0, 5],
    [999, 1, 999, 999, 999, 999, 0]
```

```

        activeCodeLine: 3,
    });

    for (let i = 0; i < N; i++) {
        for (let j = 0; j < N; j++) {
            if (i === j || i === k || j === k) continue;

            const oldVal = dist[i][j];
            const viaVal = dist[i][k] + dist[k][j];
            if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
                dist[i][j] = viaVal;
                simulationSteps.push({
                    k, i, j, matrix: dist.map(r => [...r]), update: true,
                    log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
                    activeCodeLine: 7,
                });
            }
        }
    }

    simulationSteps.push({
        k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), update: true,
        log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
        activeCodeLine: 8,
    });

    setSteps(simulationSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
}, []));

useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length - 1) {
        timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
}, [isPlaying, currentStepIndex, steps]);

```

```

    </div>
    <div>
      <h3 className="text-2xl font-bold text-white">
        <p className="text-sm text-emerald-300">Полн
      </div>
    </div>

    <div className="flex items-center gap-2">
      <button onClick={() => { setIsPlaying(false);
        text-sm">  Сброс</button>
      <button onClick={() => setIsPlaying(!isPlaying
        -amber-600' : 'bg-emerald-600'}`}>{isPlaying
      <button onClick={() => { setIsPlaying(false);
        ; }} className="px-3 py-2 bg-emerald-600 te
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    {/* Граф */}
    <div className="w-full lg:w-2/3 bg-indigo-950/2
      stify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indi
        ont-bold shadow-sm">
          {currentStep.k >= 0 && currentStep.k < 7 ?
        </div>

      <svg className="w-full h-auto max-h-[500px]"
        <defs>
          <marker id="arrow-fw-normal" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
          <marker id="arrow-fw-active" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          <marker id="arrow-fw-via" markerWidth="6"
          th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
        </defs>
        {Object.keys(adjList).flatMap((uStr) => {
          const u = parseInt(uStr);
          return adjList[u].map((edge) => {

```

```

    <div className="w-full bg-slate-950/80 p-4 rounded" >
      <p className="font-semibold text-amber-400" >
        <p className="min-h-[40px] flex items-center" >
      </div>
    </div>

```

```

  { /* Список смежности */ }
  <div className="w-full lg:w-1/3 bg-emerald-950/80" >
    <h4 className="text-lg font-bold text-emerald-400" >
      <div className="space-y-2 text-sm font-mono text-white" >
        {Object.keys(adjList).map((uStr) => {
          const u = parseInt(uStr);
          const isI = currentStep.i === u;
          const isK = currentStep.k === u;
          return (
            <div key={u} className={`flex flex-wrap ${
              isI ? 'bg-amber-900/40 border-amber-400' :
              isK ? 'bg-indigo-900/40 border-indigo-400' : ''
            }`} >
              <span className="font-bold text-white text-400">→</span>
              {adjList[u].map((edge, idx) => {
                const isUpd = currentStep.i === u;
                const isVia = currentStep.k !== u;
                return (
                  <span key={idx} className={`px-2 py-1 rounded ${
                    isVia ? 'bg-indigo-500 text-white border-indigo-400' : ''
                  }`} >
                    {edge.v} ({edge.w})
                  </span>
                )
              })}
            </div>
          )
        })}
      </div>
    </div>
  );
}
</div>
</div>

```



```

        </table>
      </div>
    </div>

    {/* Код алгоритма */}
    <div className="w-full lg:w-1/2 bg-slate-900/60" >
      <div>
        <h4 className="text-lg font-bold text-slate-100">Алгоритм
        <div className="bg-slate-950/80 rounded-lg p-4" >
          {codeLines.map(item => (
            <div key={item.line} className={`py-1.5 px-4
            e === item.line ? 'bg-blue-900/80 text-amber-400' : 'bg-slate-950/80 text-slate-100'}`>
              <span className="text-slate-600 w-5 shrink-0">{item.line}</span>
              <span className="whitespace-pre flex-grow-1">{item.code}</span>
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
</div>
);
}

```

ФАЙЛ 45/96: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [

```

```

function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Зашли в узел ${v}`, visitedNodes: Array.from(vis) });

    for (const u of adj[v]) {
        if (!vis.has(u)) {
            steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}`, stack: [...stack], visitedNodes: Array.from(vis) });
            dfs(u);
        }
    }

    stack.pop();
    steps.push({ type: 'backtrack', node: v, desc: `Вышли из узла ${v}`, ueOrStack: [...stack], visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начали с узла ${start}`, visitedNodes: Array.from(vis) });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}`, q: [...q], visitedNodes: [...Array.from(vis)] });
        q.shift();
    }
}

```

```

useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [mode]);

useEffect(() => {
  if (autoPlay) {
    const t = setInterval(() => {
      setStepIdx(p => {
        if (p >= steps.length - 1) {
          setAutoPlay(false);
          return p;
        }
        return p + 1;
      });
    }, 1200);
    return () => clearInterval(t);
  }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
  <div className="space-y-4">
    <div className="flex bg-slate-900 border border-slate-800 p-4">
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-medium rounded-md
        text-slate-400 hover:text-slate-300`} `>
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-medium rounded-md
        text-slate-400 hover:text-slate-300`} `>
        BFS (В ширину / Волна)
      </button>
    </div>
  </div>
);

```

```

        if (current && step.type !== 'edge') {
          strokeColor = "stroke-amber-500";
          textColor = "fill-amber-500";
          radius = 22;
        }

        return (
          <g key={n.id} className="graph-node">
            <circle cx={n.x} cy={n.y}
              className={`stroke-${strokeColor} fill-${fillColor} stroke-width=2`}
            />
            <text x={n.x} y={n.y}
              className={`text-${textColor} font-size=14`}
              {n.label}
            />
            {
              /* Ripple for BFS */
              current && mode === 'bfs' && n === current
                ? <circle cx={n.x} cy={n.y}
                  className="stroke-amber-500 stroke-width=4"
                />
                : null
            }
          />
        )
      )
    )
  />
</div>

{/* Structure View (Stack / Queue) */}
<div className="bg-slate-900 border border-slate-700 p-10"
  r">
  <div className={`absolute -top-3 left-3 right-3 bottom-3
    ${mode === 'dfs' ? 'border border-emerald-500' : 'border border-slate-700'}
    ld-300 border border-emerald-500`} `>
    {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
  </div>

  <div className="flex flex-col gap-2 mt-10">
    {step.queueOrStack && step.queueOrStack.length > 0
      ? step.queueOrStack.map((n, idx) => (
          <div key={idx} className={`w-40 h-20 flex items-center justify-center
            border border-slate-700 rounded-md text-slate-400 font-size=12`}
            >{n.label}</div>
        ))
      : null
    }
  </div>
</div>

```

```
        </button>
      </div>

      {/* Description Log */}
      <div className="flex-1 bg-slate-900/50 p-10
ow-hidden">
        <div className="text-[10px] text-slate-100">
          {steps.length}</div>
        <div className="text-sm text-slate-100">
          {step.desc}
        </div>
      </div>
    </div>
  </div>
);
}
```

ФАЙЛ 46/96: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../hooks';

interface Patient {
  id: string;
  name: string;
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}

// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
```

```

-----
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
    return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
  { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
  { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
  { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
  { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
  { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
  let arr: Patient[] = [];
  const start = [
    { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🔪' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
  ];
  start.forEach((p, i) => {
    arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
  });
  return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
  const [heap, setHeap] = useState<Patient[]>([]);
  const runtime = useVizRuntime();
  const liveHeap = vizArray(runtime?.variables?.h) ?? v

```

```

const newPatient: Patient = {
  id: `p${uid++}`,
  name: finalName,
  emoji: finalEmoji,
  symptom: finalSymptom,
  priority,
  animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x => x);
// Mark specifically this new patient as 'in' in the heap
const target = nextHeap.find(x => x.priority === priority);
const marked = nextHeap.map(x => x.id === (target?.id));

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

setTimeout(() => {
  setHeap(nextHeap);
  setBusy(false);
}, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
  const randomShift = Math.floor(Math.random() * 10);
  const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
  insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
  e.preventDefault();
  if (!customName.trim()) return;
  insertPatient(customName, customPriority);
  setCustomName('');
};

```

```

        }, 600);
    }, 800);

    }, 350);
  }, 650);
}, [busy, heap]);

const reset = () => {
  setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
  setLog([' База данных скорой помощи перезагружена.']);
  setHealing(null);
  setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 } ))), 1000);
};

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
  const res: React.ReactElement[] = [];
  const p = nodePos(idx);
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < visualHeap.length) {
    const lp = nodePos(l);
    res.push(
      <line key={`el${idx}`}
        x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
        x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
        stroke="#475569" strokeWidth="2"
      />
    );
  }
  if (r < visualHeap.length) {
    const rp = nodePos(r);
    res.push(
      <line key={`er${idx}`}
        x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
        x2={` ${rp.x}%` } y2={` ${rp.y - 4}%` }
        stroke="#475569" strokeWidth="2"
      />
    );
  }
});

```



```

        <input
          type="text"
          required
          value={customName}
          onChange={e => setCustomName(e.target.value)}
          placeholder="Имя пациента"
          className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
        />
      <div className="flex flex-col items-center min-w-[300px] gap-4">
        <span className="text-[9px] font-mono text-slate-400">Приоритет</span>
        <input
          type="range"
          min="10"
          max="99"
          value={customPriority}
          onChange={e => setCustomPriority(Number(e.target.value))}
          className="w-20 accent-emerald-500 cursor-pointer"
        />
      </div>
      <button
        type="submit"
        disabled={busy || !customName.trim() || heap.length === 0}
        className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
      >
        Везти
      </button>
    </form>

    {/* Забрать самого тяжелого */}
    <button
      onClick={extractMax}
      disabled={busy || heap.length === 0}
      className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center gap-2"
    >

```

```

        ${patient.animState === 'winner' ?
        ${patient.animState === 'out' ? 'an
        ${isCompilerNode ? 'ring-4 ring-amb
        ${isRoot ? 'bg-rose-950/80 border-r
    `}
    style={{
      left: `${pos.x}%`,
      top: `${pos.y}%`,
      width: isRoot ? 60 : 52,
      height: isRoot ? 60 : 52,
    }}
  >
    <span className="text-2xl leading-non
    <span className="text-[10px] font-mon
      {patient.priority}%
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full
      <div className="bg-slate-950 border
0 shadow-xl">
      <div className="font-bold text-wh
      <div className="text-emerald-400"
      </div>
    </div>
  </div>
    );
  }}}
</div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun
  <div className="absolute top-4 left-4 text-[1
    Операционная койка
  </div>

```

```
    <div className="text-[10px] font-mono text-slate-500">
      {log.map((entry, idx) => (
        <div
          key={idx}
          className={`text-xs font-mono flex items-center justify-between gap-2`}
        >
          {idx === 0 ? <span className="text-emerald-500 font-weight-bold">
            <span>{entry}</span>
          </div>
        )}}
      </div>
    </div>
  );
};
```

ФАЙЛ 47/96: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```

        {caption}
      </div>
    ))
    <div className="text-[9px] text-slate-600 text-center"
      {paused ? "пауза – курсор на карточке" : "наведи"}
    </div>
  </div>
);
};

```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
    {label !== undefined && (
      <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
        {label}
      </text>
    )}
  </g>
);

```

```

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    stroke-width={width}
    stroke-dasharray={dashed ? [4, 4]} : []
  />
);

```

```

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
  </Svg>
);
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [
      ["A", "B"], ["B", "C"],
      ["D", "E"],
      ["F", "G"], ["G", "H"], ["F", "H"],
    ],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => (

```

```

        fontWeight="700"
        fill={C.warn}
      >
        {w}
      </text>
    )}
  </g>
);
}}
{Object.entries(pos).map(([k, [x, y]]) => {
  <Node key={k} x={x} y={y} label={k} fill={C.idle}
  >>
})}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={30} />
        </g>
      )}
    >

```

```

const f = frames[step];
const color = (id: string) =>
  id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f87193" : "#f08080";

return (
  <Svg caption={captions[step]}>
    {f.edges.map(([a, b], i) => (
      <line
        key={i}
        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: MOVE }}
      />
    ))}
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
        <circle r={13} fill={color(id)} stroke={C.idle} />
        <text textAnchor="middle" dominantBaseline="bottom">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: x под p, p под x"]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x", "x": "p"}]},
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"p": "x", "x": "p"}]}
    ]}
  />
);

```

```

};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "E"], ["D", "F"],
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order[step - 1] : null;
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`} >
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
          stroke={wave?.includes(k) ? "#a5b4fc" : undefined} />
      })}
    </Svg>
  );
};

```

```

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""} >
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
          fill={onPath ? C.onPath : visited.has(u) || visited.has(v) ? C.visited : C.inactive}
          stroke={onPath ? "#a5b4fc" : undefined} />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={visited.has(k) ? C.visited : C.inactive}
          stroke={visited.has(k) ? "#a5b4fc" : undefined} />
      })}
    </Svg>
  );
};

```



```

    </Svg>
  );
};

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь  

      <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />  

      <text x={130} y={54} textAnchor="middle" fontSize=14 />  

      <Node x={60} y={70} r={20} fill={C.active} label="3" />  

      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />  

      <text x={60} y={106} textAnchor="middle" fontSize=14 />  

      <text x={200} y={106} textAnchor="middle" fontSize=14 />  

      <text x={130} y={22} textAnchor="middle" fontSize=14 />  

    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : "Граф связен"} />
    {edges.map(([u, v], i) => {
      const isBridge = u === "C" && v === "D";
      if (isBridge && cut) return null;
      return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (
      <Node x={x} y={y} r={20} fill={C[k]} label={k} />
    ))}
  );
};

```

```

const taken = new Set(chosen.slice(0, step));
return (
  <Svg caption={`Жадно берём самые лёгкие рёбра без ц
    {edges.map((e, i) => {
      const on = taken.has(`${e.u}-${e.v}`);
      return (
        <g key={i}>
          <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
          <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
          </g>
        );
      });
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45
  ];
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step :
      })}
    </Svg>
  );
};

```

```

        <text x={130} y={111 - i * 26} textAnchor="middle">{v}</text>
      </g>
    )}
    <text x={130} y={16} textAnchor="middle" fontSize={14}>{v}</text>
  </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к концу" width={300} height={150}>
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6"
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} fontSize={9} fill={C.dim}>Выход</text>
      <text x={206} y={44} fontSize={9} fill={C.dim}>Вход</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={20} fill={C.dim}>
            <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
          </rect>
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};

```

```

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 950);
  const p = parents[step];
  const pos: [number, number][] = [[50, 95], [105, 95], [160, 95], [215, 95], [270, 95], [325, 95], [380, 95], [435, 95], [490, 95], [545, 95], [600, 95], [655, 95], [710, 95], [765, 95], [820, 95], [875, 95], [930, 95], [985, 95], [1040, 95], [1095, 95], [1150, 95], [1205, 95], [1260, 95], [1315, 95], [1370, 95], [1425, 95], [1480, 95], [1535, 95], [1590, 95], [1645, 95], [1700, 95], [1755, 95], [1810, 95], [1865, 95], [1920, 95], [1975, 95], [2030, 95], [2085, 95], [2140, 95], [2195, 95], [2250, 95], [2305, 95], [2360, 95], [2415, 95], [2470, 95], [2525, 95], [2580, 95], [2635, 95], [2690, 95], [2745, 95], [2800, 95], [2855, 95], [2910, 95], [2965, 95], [3020, 95], [3075, 95], [3130, 95], [3185, 95], [3240, 95], [3295, 95], [3350, 95], [3405, 95], [3460, 95], [3515, 95], [3570, 95], [3625, 95], [3680, 95], [3735, 95], [3790, 95], [3845, 95], [3900, 95], [3955, 95], [4010, 95], [4065, 95], [4120, 95], [4175, 95], [4230, 95], [4285, 95], [4340, 95], [4395, 95], [4450, 95], [4505, 95], [4560, 95], [4615, 95], [4670, 95], [4725, 95], [4780, 95], [4835, 95], [4890, 95], [4945, 95], [5000, 95], [5055, 95], [5110, 95], [5165, 95], [5220, 95], [5275, 95], [5330, 95], [5385, 95], [5440, 95], [5495, 95], [5550, 95], [5605, 95], [5660, 95], [5715, 95], [5770, 95], [5825, 95], [5880, 95], [5935, 95], [5990, 95], [6045, 95], [6100, 95], [6155, 95], [6210, 95], [6265, 95], [6320, 95], [6375, 95], [6430, 95], [6485, 95], [6540, 95], [6595, 95], [6650, 95], [6705, 95], [6760, 95], [6815, 95], [6870, 95], [6925, 95], [6980, 95], [7035, 95], [7090, 95], [7145, 95], [7200, 95], [7255, 95], [7310, 95], [7365, 95], [7420, 95], [7475, 95], [7530, 95], [7585, 95], [7640, 95], [7695, 95], [7750, 95], [7805, 95], [7860, 95], [7915, 95], [7970, 95], [8025, 95], [8080, 95], [8135, 95], [8190, 95], [8245, 95], [8300, 95], [8355, 95], [8410, 95], [8465, 95], [8520, 95], [8575, 95], [8630, 95], [8685, 95], [8740, 95], [8795, 95], [8850, 95], [8905, 95], [8960, 95], [9015, 95], [9070, 95], [9125, 95], [9180, 95], [9235, 95], [9290, 95], [9345, 95], [9400, 95], [9455, 95], [9510, 95], [9565, 95], [9620, 95], [9675, 95], [9730, 95], [9785, 95], [9840, 95], [9895, 95], [9950, 95], [10005, 95], [10060, 95], [10115, 95], [10170, 95], [10225, 95], [10280, 95], [10335, 95], [10390, 95], [10445, 95], [10500, 95], [10555, 95], [10610, 95], [10665, 95], [10720, 95], [10775, 95], [10830, 95], [10885, 95], [10940, 95], [10995, 95], [11050, 95], [11105, 95], [11160, 95], [11215, 95], [11270, 95], [11325, 95], [11380, 95], [11435, 95], [11490, 95], [11545, 95], [11600, 95], [11655, 95], [11710, 95], [11765, 95], [11820, 95], [11875, 95], [11930, 95], [11985, 95], [12040, 95], [12095, 95], [12150, 95], [12205, 95], [12260, 95], [12315, 95], [12370, 95], [12425, 95], [12480, 95], [12535, 95], [12590, 95], [12645, 95], [12700, 95], [12755, 95], [12810, 95], [12865, 95], [12920, 95], [12975, 95], [13030, 95], [13085, 95], [13140, 95], [13195, 95], [13250, 95], [13305, 95], [13360, 95], [13415, 95], [13470, 95], [13525, 95], [13580, 95], [13635, 95], [13690, 95], [13745, 95], [13800, 95], [13855, 95], [13910, 95], [13965, 95], [14020, 95], [14075, 95], [14130, 95], [14185, 95], [14240, 95], [14295, 95], [14350, 95], [14405, 95], [14460, 95], [14515, 95], [14570, 95], [14625, 95], [14680, 95], [14735, 95], [14790, 95], [14845, 95], [14900, 95], [14955, 95], [15010, 95], [15065, 95], [15120, 95], [15175, 95], [15230, 95], [15285, 95], [15340, 95], [15395, 95], [15450, 95], [15505, 95], [15560, 95], [15615, 95], [15670, 95], [15725, 95], [15780, 95], [15835, 95], [15890, 95], [15945, 95], [16000, 95], [16055, 95], [16110, 95], [16165, 95], [16220, 95], [16275, 95], [16330, 95], [16385, 95], [16440, 95], [16495, 95], [16550, 95], [16605, 95], [16660, 95], [16715, 95], [16770, 95], [16825, 95], [16880, 95], [16935, 95], [16990, 95], [17045, 95], [17100, 95], [17155, 95], [17210, 95], [17265, 95], [17320, 95], [17375, 95], [17430, 95], [17485, 95], [17540, 95], [17595, 95], [17650, 95], [17705, 95], [17760, 95], [17815, 95], [17870, 95], [17925, 95], [17980, 95], [18035, 95], [18090, 95], [18145, 95], [18200, 95], [18255, 95], [18310, 95], [18365, 95], [18420, 95], [18475, 95], [18530, 95], [18585, 95], [18640, 95], [18695, 95], [18750, 95], [18805, 95], [18860, 95], [18915, 95], [18970, 95], [19025, 95], [19080, 95], [19135, 95], [19190, 95], [19245, 95], [19300, 95], [19355, 95], [19410, 95], [19465, 95], [19520, 95], [19575, 95], [19630, 95], [19685, 95], [19740, 95], [19795, 95], [19850, 95], [19905, 95], [19960, 95], [20015, 95], [20070, 95], [20125, 95], [20180, 95], [20235, 95], [20290, 95], [20345, 95], [20400, 95], [20455, 95], [20510, 95], [20565, 95], [20620, 95], [20675, 95], [20730, 95], [20785, 95], [20840, 95], [20895, 95], [20950, 95], [21005, 95], [21060, 95], [21115, 95], [21170, 95], [21225, 95], [21280, 95], [21335, 95], [21390, 95], [21445, 95], [21500, 95], [21555, 95], [21610, 95], [21665, 95], [21720, 95], [21775, 95], [21830, 95], [21885, 95], [21940, 95], [21995, 95], [22050, 95], [22105, 95], [22160, 95], [22215, 95], [22270, 95], [22325, 95], [22380, 95], [22435, 95], [22490, 95], [22545, 95], [22600, 95], [22655, 95], [22710, 95], [22765, 95], [22820, 95], [22875, 95], [22930, 95], [22985, 95], [23040, 95], [23095, 95], [23150, 95], [23205, 95], [23260, 95], [23315, 95], [23370, 95], [23425, 95], [23480, 95], [23535, 95], [23590, 95], [23645, 95], [23700, 95], [23755, 95], [23810, 95], [23865, 95], [23920, 95], [23975, 95], [24030, 95], [24085, 95], [24140, 95], [24195, 95], [24250, 95], [24305, 95], [24360, 95], [24415, 95], [24470, 95], [24525, 95], [24580, 95], [24635, 95], [24690, 95], [24745, 95], [24800, 95], [24855, 95], [24910, 95], [24965, 95], [25020, 95], [25075, 95], [25130, 95], [25185, 95], [25240, 95], [25295, 95], [25350, 95], [25405, 95], [25460, 95], [25515, 95], [25570, 95], [25625, 95], [25680, 95], [25735, 95], [25790, 95], [25845, 95], [25900, 95], [25955, 95], [26010, 95], [26065, 95], [26120, 95], [26175, 95], [26230, 95], [26285, 95], [26340, 95], [26395, 95], [26450, 95], [26505, 95], [26560, 95], [26615, 95], [26670, 95], [26725, 95], [26780, 95], [26835, 95], [26890, 95], [26945, 95], [27000, 95], [27055, 95], [27110, 95], [27165, 95], [27220, 95], [27275, 95], [27330, 95], [27385, 95], [27440, 95], [27495, 95], [27550, 95], [27605, 95], [27660, 95], [27715, 95], [27770, 95], [27825, 95], [27880, 95], [27935, 95], [27990, 95], [28045, 95], [28100, 95], [28155, 95], [28210, 95], [28265, 95], [28320, 95], [28375, 95], [28430, 95], [28485, 95], [28540, 95], [28595, 95], [28650, 95], [28705, 95], [28760, 95], [28815, 95], [28870, 95], [28925, 95], [28980, 95], [29035, 95], [29090, 95], [29145, 95], [29200, 95], [29255, 95], [29310, 95], [29365, 95], [29420, 95], [29475, 95], [29530, 95], [29585, 95], [29640, 95], [29695, 95], [29750, 95], [29805, 95], [29860, 95], [29915, 95], [29970, 95], [30025, 95], [30080, 95], [30135, 95], [30190, 95], [30245, 95], [30300, 95], [30355, 95], [30410, 95], [30465, 95], [30520, 95], [30575, 95], [30630, 95], [30685, 95], [30740, 95], [30795, 95], [30850, 95], [30905, 95], [30960, 95], [31015, 95], [31070, 95], [31125, 95], [31180, 95], [31235, 95], [31290, 95], [31345, 95], [31400, 95], [31455, 95], [31510, 95], [31565, 95], [31620, 95], [31675, 95], [31730, 95], [31785, 95], [31840, 95], [31895, 95], [31950, 95], [32005, 95], [32060, 95], [32115, 95], [32170, 95], [32225, 95], [32280, 95], [32335, 95], [32390, 95], [32445, 95], [32500, 95], [32555, 95], [32610, 95], [32665, 95], [32720, 95], [32775, 95], [32830, 95], [32885, 95], [32940, 95], [32995, 95], [33050, 95], [33105, 95], [33160, 95], [33215, 95], [33270, 95], [33325, 95], [33380, 95], [33435, 95], [33490, 95], [33545, 95], [33600, 95], [33655, 95], [33710, 95], [33765, 95], [33820, 95], [33875, 95], [33930, 95], [33985, 95], [34040, 95], [34095, 95], [34150, 95], [34205, 95], [34260, 95], [34315, 95], [34370, 95], [34425, 95], [34480, 95], [34535, 95], [34590, 95], [34645, 95], [34700, 95], [34755, 95], [34810, 95], [34865, 95], [34920, 95], [34975, 95], [35030, 95], [35085, 95], [35140, 95], [35195, 95], [35250, 95], [35305, 95], [35360, 95], [35415, 95], [35470, 95], [35525, 95], [35580, 95], [35635, 95], [35690, 95], [35745, 95], [35800, 95], [35855, 95], [35910, 95], [35965, 95], [36020, 95], [36075, 95], [36130, 95], [36185, 95], [36240, 95], [36295, 95], [36350, 95], [36405, 95], [36460, 95], [36515, 95], [36570, 95], [36625, 95], [36680, 95], [36735, 95], [36790, 95], [36845, 95], [36900, 95], [36955, 95], [37010, 95], [37065, 95], [37120, 95], [37175, 95], [37230, 95], [37285, 95], [37340, 95], [37395, 95], [37450, 95], [37505, 95], [37560, 95], [37615, 95], [37670, 95], [37725, 95], [37780, 95], [37835, 95], [37890, 95], [37945, 95], [38000, 95], [38055, 95], [38110, 95], [38165, 95], [38220, 95], [38275, 95], [38330, 95], [38385, 95], [38440, 95], [38495, 95], [38550, 95], [38605, 95], [38660, 95], [38715, 95], [38770, 95], [38825, 95], [38880, 95], [38935, 95], [38990, 95], [39045, 95], [39100, 95], [39155, 95], [39210, 95], [39265, 95], [39320, 95], [39375, 95], [39430, 95], [39485, 95], [39540, 95], [39595, 95], [39650, 95], [39705, 95], [39760, 95], [39815, 95], [39870, 95], [39925, 95], [39980, 95], [40035, 95], [40090, 95], [40145, 95], [40200, 95], [40255, 95], [40310, 95], [40365, 95], [40420, 95], [40475, 95], [40530, 95], [40585, 95], [40640, 95], [40695, 95], [40750, 95], [40805, 95], [40860, 95], [40915, 95], [40970, 95], [41025, 95], [41080, 95], [41135, 95], [41190, 95], [41245, 95], [41300, 95], [41355, 95], [41410, 95], [41465, 95], [41520, 95], [41575, 95], [41630, 95], [41685, 95], [41740, 95], [41795, 95], [41850, 95], [41905, 95], [41960, 95], [42015, 95], [42070, 95], [42125, 95], [42180, 95], [42235, 95], [42290, 95], [42345, 95], [42400, 95], [42455, 95], [42510, 95], [42565, 95], [42620, 95], [42675, 95], [42730, 95], [42785, 95], [42840, 95], [42895, 95], [42950, 95], [43005, 95], [43060, 95], [43115, 95], [43170, 95], [43225, 95], [43280, 95], [43335, 95], [43390, 95], [43445, 95], [43500, 95], [43555, 95], [43610, 95], [43665, 95], [43720, 95], [43775, 95], [43830, 95], [43885, 95], [43940, 95], [43995, 95], [44050, 95], [44105, 95], [44160, 95], [44215, 95], [44270, 95], [44325, 95], [44380, 95], [44435, 95], [44490, 95], [44545, 95], [44600, 95], [44655, 95], [44710, 95], [44765, 95], [44820, 95], [44875, 95], [44930, 95], [44985, 95], [45040, 95], [45095, 95], [45150, 95], [45205, 95], [45260, 95], [45315, 95], [45370, 95], [45425, 95], [45480, 95], [45535, 95], [45590, 95], [45645, 95], [45700, 95], [45755, 95], [45810, 95], [45865, 95], [45920, 95], [45975, 95], [46030, 95], [46085, 95], [46140, 95], [46195, 95], [46250, 95], [46305, 95], [46360, 95], [46415, 95], [46470, 95], [46525, 95], [46580, 95], [46635, 95], [46690, 95], [46745, 95], [46800, 95], [46855, 95], [46910, 95], [46965, 95], [47020, 95], [47075, 95], [47130, 95], [47185, 95], [47240, 95], [47295, 95], [47350, 95], [47405, 95], [47460, 95], [47515, 95], [47570, 95], [47625, 95], [47680, 95], [47735, 95], [47790, 95], [47845, 95], [47900, 95], [47955, 95], [48010, 95], [48065, 95], [48120, 95], [48175, 95], [48230, 95], [48285, 95], [48340, 95], [48395, 95], [48450, 95], [48505, 95], [48560, 95], [48615, 95], [48670, 95], [48725, 95], [48780, 95], [48835, 95], [48890, 95], [48945, 95], [49000, 95], [49055, 95], [49110, 95], [49165, 95], [49220, 95], [49275, 95], [49330, 95], [49385, 95], [49440, 95], [49495, 95], [49550, 95], [49605, 95], [49660, 95], [49715, 95], [49770, 95], [49825, 95], [49880, 95], [49935, 95], [49990, 95], [50045, 95], [50100, 95], [50155, 95], [50210, 95], [50265, 95], [50320, 95], [50375, 95], [50430, 95], [50485, 95], [50540, 95], [50595, 95], [50650, 95], [50705, 95], [50760, 95], [50815, 95], [50870, 95], [50925, 95], [50980, 95], [51035, 95], [51090, 95], [51145, 95], [51200, 95], [51255, 95], [51310, 95], [51365, 95], [51420, 95], [51475, 95], [51530, 95], [51585, 95], [51640, 95], [51695, 95], [51750, 95], [51805, 95], [51860, 95], [51915, 95], [51970, 95], [52025, 95], [52080, 95], [52135, 95], [52190, 95], [52245, 95], [52300, 95], [52355, 95], [52410, 95], [52465, 95], [52520, 95], [52575, 95], [52630, 95], [52685, 95], [52740, 95], [52795, 95], [52850, 95], [52905, 95], [52960, 95], [53015, 95], [53070, 95], [53125, 95], [53180, 95], [53235, 95], [53290, 95], [53345, 95], [53400, 95], [53455, 95], [53510, 95], [53565, 95], [53620, 95], [53675, 95], [53730, 95], [53785, 95], [53840, 95], [53895, 95], [53950, 95], [54005, 95], [54060, 95], [54115, 95], [54170, 95], [54225, 95], [54280, 95], [54335, 95], [54390, 95], [54445, 95], [54500, 95], [54555, 95], [54610, 95], [54665, 95], [54720, 95], [54775, 95], [54830, 95], [54885, 95], [54940, 95], [54995, 95], [55050, 95], [55105, 95], [55160, 95], [55215, 95], [55270, 95], [55325, 95], [55380, 95], [55435, 95], [55490, 95], [55545, 95], [55600, 95], [55655, 95], [55710, 95], [55765, 95], [55820, 95], [55875, 95], [55930, 95], [55985, 95], [56040, 95], [56095, 95], [56150, 95], [56205, 95], [56260, 95], [56315, 95], [56370, 95], [56425, 95], [56480, 95], [56535, 95], [56590, 95], [56645, 95], [56700, 95], [56755, 95], [56810, 95], [56865, 95], [56920, 95], [56975, 95], [57030, 95], [57085, 95], [57140, 95], [57195, 95], [57250, 95], [57305, 95], [57360, 95], [57415, 95], [57470, 95], [57525, 95], [57580, 95], [57635, 95], [57690, 95], [57745, 95], [57800, 95], [57855, 95], [57910, 95], [57965, 95], [58020, 95], [58075, 95], [58130, 95], [58185, 95], [58240, 95], [58295, 95], [58350, 95], [58405, 95], [58460, 95], [58515, 95], [58570, 95], [58625, 95], [58680, 95], [58735, 95], [58790, 95], [58845, 95], [58900, 95], [58955, 95], [5901
```



```

<text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
{a.map((v, i) => (
  <g key={i}>
    <rect x={20 + i * 39} y={24} width={34} height={12}
      fill={show && i >= l && i <= r ? C.active : C.dim}>
    <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
  </g>
)}}
<text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
{pref.map((v, i) => (
  <g key={i}>
    <rect x={20 + i * 33} y={78} width={29} height={12}
      fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
      stroke={C.idleStroke} style={{ transition: "stroke 0.5s" }}>
    <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
  </g>
)}}
</Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.dim}>
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.dim}>

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={44}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.dim}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">π</text>
          <rect x={18 + idx * 33} y={70} width={28} height={44}
            fill={idx <= i && step > 0 ? C.warn : C.idle} />
        </g>
      ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12}>
      π[i] — длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово ${step}`
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (

```

```

-----
| "suffix-link" | "toposort" | "relax" | "prefix-sum"
| "segment-tree" | "bridge" | "articulation" | "mst"
| "prefix-function" | "dp" | "floyd" | "components" |
| "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
  scc: SccDemo,
  "prefix-function": PrefixFunctionDemo,
  dp: DpDemo,
  floyd: FloydDemo,
  components: ComponentsDemo,
  graph: GraphBasicsDemo,
  "coord-compress": CoordCompressDemo,
  zig: ZigDemo,
  "zig-zig": ZigZigDemo,
  "zig-zag": ZigZagDemo,
};

export const MiniDemo: React.FC<{ kind: DemoKind }> = (

```

```

    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
        window.removeEventListener("keydown", onKey);
        document.body.style.overflow = "";
    };
}, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
    <div className="fixed inset-0 z-[110] flex items-end" >
        <div className="absolute inset-0 bg-slate-950/80" >
            <div className="relative w-full sm:max-w-5xl max-h-
                ded-2xl shadow-2xl flex flex-col" >
                <div className="flex items-center gap-3 px-4 py-4" >
                    <div className="min-w-0 flex-1" >
                        <h3 className="font-bold text-white text-sm" >
                            {viz.hint} && <p className="text-[11px] text-white" >
                                {viz.description}
                        </div>
                    <button
                        onClick={onClose}
                        className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
                        aria-label="Заккрыть симулятор"
                    >
                        <X className="w-5 h-5" />
                    </button>
                </div>
                <div className="overflow-y-auto overscroll-contain" >
                    <Component />
                </div>
            </div>
        </div>,
    document.body
);

```



```

        return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height)), left }, [anchor]);

    useEffect(() => {
        if (!anchor) return;
        const onKey = (e: KeyboardEvent) => {
            if (e.key === "Escape") onClose();
        };
        window.addEventListener("keydown", onKey);
        window.addEventListener("scroll", onClose, true);
        return () => {
            window.removeEventListener("keydown", onKey);
            window.removeEventListener("scroll", onClose, true);
        };
    }, [anchor, onClose]);

    if (!anchor) return null;
    const entry = GLOSSARY_BY_ID.get(anchor.termId);
    if (!entry) return null;
    const viz = getViz(entry.simulator);
    const simulatorId = entry.simulator;

    return createPortal(
        <div
            ref={cardRef}

```

```

    })

    {viz && simulatorId && (
      <button
        onClick={() => onOpenSimulator(simulatorId)}
        className="w-full flex items-center justify-center text-sm font-medium rounded-lg py-2 transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    )}
  </div>
</div>
</div>,
document.body
);
};

```

ФАЙЛ 54/96: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
  useVizStepSync(step, setStep, MAX_STEP, (vars) => {
    const edge = `${vizString(vars.u)} ?? ""}${vizString(vars.v)} ?? ""`;
    return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
  });

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
  ];

```

```

        if (step === 4) return "stroke-amber-400 stroke-emerald-500";
        if (step > 4) return "stroke-emerald-500";
    }
    if (e.id === 'BC') {
        if (step === 5) return "stroke-amber-400 stroke-emerald-500";
        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-emerald-500";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-slate)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-slate-dim)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
    if (e.u === 'S') return e.weight;

    if (step < 4) return e.weight;

```

```

        </div>
    );
    case 1: return (
        <div>
            <b>Шаг 1: Точка отсчета.</b><br/>
            Добавляем фиктивную вершину S. Соединяем ее с остальными вершинами.
            Это наша база для расчета потенциалов.
        </div>
    );
    case 2: return (
        <div>
            <b>Шаг 2: Ищем потенциалы  $h(v)$ .</b><br/>
            Запускаем медленный, но мощный алгоритм для всех вершин:
            <span className="text-pink-300 ml-1">...</span>
        </div>
    );
    case 3: return (
        <div>
            <b>Шаг 3: Магическая формула.</b><br/>
            Всем ребрам графа назначаем новый вес.
            <div className="text-center font-monospace">...</div>
            Это повышает или понижает вес ребра.
        </div>
    );
    case 4: return (
        <div>
            <b>Шаг 3.1: Пересчет ребра A → B.</b><br/>
            Старый вес: <span className="text-green-300">...</span>
            Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-pink-300">...</span>
        </div>
    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра B → C.</b><br/>
            Старый вес: 1. <br/>
            Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-pink-300">...</span>
        </div>
    );

```

```

    uto-start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:#f00;stroke-dasharray: 5 5;" />
    </marker>
    <marker id="arrow-amber" viewBox="0 0 10 10" />
    start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:#f00;stroke-dasharray: 5 5;" />
    </marker>
    <marker id="arrow-pink" viewBox="0 0 10 10" />
    tart-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:#f00;stroke-dasharray: 5 5;" />
    </marker>
    <marker id="arrow-emerald" viewBox="0 0 10 10" />
    o-start-reverse">
        <path d="M 0 0 L 10 5 L 10 10" style="stroke-width:2px;stroke:#f00;stroke-dasharray: 5 5;" />
    </marker>
</defs>

{/* Edges */}
edges.map((e, i) => {
    const u = nodes.find(n => n.id === e.u);
    const v = nodes.find(n => n.id === e.v);

    const isS = e.u === 'S';
    if (isS && (step === 0 || step === 1)) {
        return null;
    }

    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getEdgeColorClass(e);
    const textHigh = isEdgeTextHigh(e);
    const textEmer = isEdgeTextEmer(e);

    return (
        <g key={i}>
            <line
                x1={u.x} y1={u.y}
                x2={v.x} y2={v.y}
                className={`stroke-${edgeColorClass}`}
                markerEnd={getMarkerEnd(e)}
            />
            {textHigh && textEmer ? (
                <text
                    x={midX} y={midY}
                    text={e.label}
                    className={`text-${edgeColorClass} ${textHigh ? 'high' : ''} ${textEmer ? 'emer' : ''}`}
                />
            ) : null}
        </g>
    );
});

```

```

fade-in zoom-in duration-500">
<rect x="-1
k-500 stroke-1" />
<text x="0"
font-bold fill-pink-400">
{pot}
</text>
</g>
})
</g>
)
}}
</svg>
</div>

```

```

<div className="w-full md:w-[320px] flex
  {/* Log Box */}
  <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
  {getDesc()}
</div>

```

```

{/* Controls */}
<div className="flex bg-slate-900 b
  <button onClick={() => setStep(1
    className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
    <RotateCcw strokeWidth={3}
  </button>
  <button onClick={() => setStep(1
    className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
    <Undo strokeWidth={3} size=
  </button>
  <button onClick={() => setStep(
    className={`flex-1 font-bol
      ${step === MAX_STEP
        ? 'bg-emerald-600/50

```

```

    label: string;
  }

interface GEdge {
  u: number;
  v: number;
}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {

```

```

        : [...initialEdges],
    });
};

recordStep(
  "init",
  "Инициализация алгоритма Косарайю. Начинаем первый шаг",
  null,
  false,
  -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);
  recordStep(
    "dfs1",
    `DFS 1: зашли в вершину ${u}, помечаем посещение`,
    u,
    false,
    -1,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs1(to);
    }
  }

  order.push(u);
  recordStep(
    "dfs1",
    `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
    u,
    true,
    -1,
  );
};

```



```

        currentScc,
    );

    for (const to of adjT[u]) {
        if (!visited.has(to)) {
            dfs2(to, currentScc);
        }
    }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
    const u = order[i];
    if (!visited.has(u)) {
        sccId++;
        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека`,
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в ${sccId}`,
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
    null,

```

```

const getSccColor = (id: number) => {
  if (id === 1) return "fill-emerald-600 stroke-emera
  if (id === 2) return "fill-indigo-600 stroke-indigo
  return "fill-slate-800 stroke-slate-500";
};

return (
  <div className="bg-slate-900 rounded-xl border bord
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Layers className="w-5 h-5 text-emerald-400"
          Визуализатор Алгоритма Косарайю (Поиск SCC)
        </h3>

      <div className="flex items-center gap-2 bg-slate
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            d-500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          className="flex items-center justify-center
            ors"
        >
          <RotateCcw className="w-4 h-4" />
        </button>
      </div>
    </div>
  )

```

```

        </marker>
    </defs>

    { /* Edges */
    {initialEdges.map((edge, idx) => {
        const u = initialNodes.find((n) => n.id === edge.u);
        const v = initialNodes.find((n) => n.id === edge.v);

        // If transposed, draw reversed coordinates
        const x1 = step.transposed ? v.x : u.x;
        const y1 = step.transposed ? v.y : u.y;
        const x2 = step.transposed ? u.x : v.x;
        const y2 = step.transposed ? u.y : v.y;

        const isSccEdge =
            step.sccMap[edge.u] &&
            step.sccMap[edge.v] &&
            step.sccMap[edge.u] === step.sccMap[edge.v];
        let stroke = "#475569";
        let markerId = "arrow";

        if (step.transposed) {
            stroke = isSccEdge ? "#818cf8" : "#4f46e5";
            markerId = "arrow-transposed";
        } else {
            if (
                step.currentNode === edge.u ||
                step.currentNode === edge.v
            ) {
                stroke = "#10b981";
                markerId = "arrow-active";
            }
        }
    })

    // Adjust slightly for bidirectional link
    const isBidi =
        (edge.u === 3 && edge.v === 4) ||
        (edge.u === 4 && edge.v === 3);

```

```

        cy={node.y}
        r="20"
        className={`transition-all duration
        strokeWidth="3"
    />
    <text
        x={node.x}
        y={node.y}
        dy=".3em"
        textAnchor="middle"
        fill="white"
        fontSize="13px"
        fontWeight="bold"
        className="select-none"
    >
        {node.label}
    </text>

    {sccId && (
        <text
            x={node.x}
            y={node.y - 28}
            textAnchor="middle"
            fill="#10b981"
            fontSize="10px"
            fontWeight="bold"
        >
            SCC #{sccId}
        </text>
    )}
    </g>
    );
    }}}
</svg>

{step.transposed && (
    <div className="absolute top-4 left-4 bg-in
    1 rounded">

```

```

        </div>
      </div>

      {/* List of actions log */}
      <div>
        <span className="text-[10px] text-slate-500">
          ЛОГ ОПЕРАЦИЙ:
        </span>
        <div className="space-y-1.5 max-h-[150px]">
          {steps
            .slice(0, currentStepIdx + 1)
            .reverse()
            .slice(0, 5)
            .map(({s, idx}) => {
              <div
                key={idx}
                className={`text-[11px] p-1.5 rounded`
              >
                {s.desc}
              </div>
            )}}
          </div>
        </div>
      </div>

      <div className="mt-4 pt-3 border-t border-slate-200">
        <span>
          Шаг {currentStepIdx + 1} из {steps.length}
        </span>
        <div className="flex gap-1.5">
          <button
            disabled={currentStepIdx === 0}
            onClick={() => setCurrentStepIdx((prev) => prev - 1)}
            className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
          >
            Назад
          </button>
          <button
            disabled={currentStepIdx === steps.length - 1}
            onClick={() => setCurrentStepIdx((prev) => prev + 1)}
            className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
          >
            Далее
          </button>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    status: 'pending' | 'inspecting' | 'included' | 'reje
    color: string;
}

```

```

interface StepLog {
  title: string;
  description: string;
  type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
  { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
  { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
  { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
  { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
  { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
  { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
  { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
  { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
  { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

```

```

        овы начинать!',
        type: 'info',
    },
  ]));

// --- KRUSKAL STEPS ---
const kruskalSteps = [
  // Step 1: Inspect A-B (2)
  () => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в
      description: 'Оно соединяет две бесцветные верш
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Include A-B
  () => {
    setVertices(prev => prev.map(v => v.id === 0 || v
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро A-B в осто́ве',
      description: 'Вершины A and B окрашены в красны
      type: 'success'
    }, ...prev]));
  },
  // Step 3: Inspect D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
      description: 'Оно соединяет две другие бесцветн
      type: 'info'
    }, ...prev]));
  },
  // Step 4: Include D-E
  () => {
    setVertices(prev => prev.map(v => v.id === 3 || v

```

```

setVertices(prev => prev.map(v => v.color === 'blue' ? v : {id: 5, color: 'blue'}));
setEdges(prev => prev.map(e => e.status === 'included' ? e : {id: '1-4', status: 'included'}));
setLogs(prev => [{
  title: 'Слияние кланов: Ребро A-D добавлено',
  description: 'Мы объединили красный и синий кланы',
  type: 'success'
}], ...prev));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {id: '1-4', status: 'rejected'} : e));
  setLogs(prev => [{
    title: 'Шаг 5: Берем ребро B-E (вес 6)',
    description: 'НО: это ребро соединяет вершины B и E, которые уже соединены!',
    type: 'warning'
  }], ...prev));
},
// Step 10: Reject B-E
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {id: '1-4', status: 'rejected'} : e));
  setLogs(prev => [{
    title: 'Отказ: Цикл обнаружен!',
    description: 'Мы выбрасываем ребро B-E. Иначе получится цикл A-B-E-D-A',
    type: 'error'
  }], ...prev));
},
// Step 11: Inspect E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {id: '4-5', status: 'included'} : e));
  setLogs(prev => [{
    title: 'Шаг 6: Берем ребро E-F (вес 7)',
    description: 'Оно соединяет фиолетовую вершину E и синюю вершину F',
    type: 'info'
  }], ...prev));
},
// Step 12: Include E-F
() => {
  setVertices(prev => prev.map(v => v.id === 5 ? {id: 5, color: 'blue'} : v));

```



```

    setLogs(prev => [{
      title: 'Отказ: Цикл обнаружен!',
      description: 'Выбрасываем ребро C-F.',
      type: 'error'
    }, ...prev]));
  },
  // Step 17: Inspect G-H (10)
  () => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
      title: 'Шаг 9: Берем ребро G-H (вес 10)',
      description: 'Соединяет фиолетовую G и бесцветную H.',
      type: 'info'
    }, ...prev]));
  },
  // Step 18: Include G-H
  () => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро G-H в осто́ве',
      description: 'Вершина H перекрашена в фиолетовую.',
      type: 'success'
    }, ...prev]));
  },
  // Step 19: Inspect E-H (11) - Cycle!
  () => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{
      title: 'Шаг 10: Берем ребро E-H (вес 11)',
      description: 'Оба конца E и H уже фиолетовые. Соединение образует цикл!',
      type: 'warning'
    }, ...prev]));
  },
  // Step 20: Reject E-H
  () => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setLogs(prev => [{

```

```

        type: 'info'
    }, ...prev]);
},
// Step 2: Pop D-E (3)
() => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
        title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
        description: 'Плесень тянется по ребру D-E к вершине E',
        type: 'info'
    }, ...prev]);
},
// Step 3: Include D-E, add D's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
        ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-F (8)']);
    setLogs(prev => [{
        title: 'Успех: Плесень захватила вершину D',
        description: 'В кучу добавлены новые пути из D: A-D, B-E, E-F, D-F',
        type: 'success'
    }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
        description: 'Плесень расширяется в сторону вершины A',
        type: 'info'
    }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
    setVertices(prev => prev.map(v => v.id === 0 ? {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
        eap' } : e));

```

```

() => {
  setVertices(prev => prev.map(v => v.id === 2 ? {
  setEdges(prev => prev.map(e => e.id === '1-2' ? {
    eap' } : e));
  setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
  setLogs(prev => [{
    title: 'Успех: Вершина C захвачена',
    description: 'Ребро C-F(9) добавлено в кучу.',
    type: 'success'
  }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setLogs(prev => [{
    title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
    description: 'ВНИМАНИЕ! Плесень проверяет вершину B',
    type: 'warning'
  }, ...prev]));
},
// Step 11: Reject B-E
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
  setLogs(prev => [{
    title: 'Отказ: Игнорируем внутреннее ребро',
    description: 'Ребро B-E отброшено. Плесень не р',
    type: 'error'
  }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
  setEdges(prev => prev.map(e => e.id === '4-5' ? {
  setLogs(prev => [{
    title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
    description: 'Плесень из центра Токио (E) тянет',
    type: 'info'
  }, ...prev]));
},

```

```

        title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
        description: 'Вершины C и F уже заражены плесенью',
        type: 'warning'
    }, ...prev]);
},
// Step 17: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Отказ: Игнорируем ребро C-F',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
        description: 'Плесень ползет к вершине H.',
        type: 'info'
    }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
        eap' } : e));
    setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Успех: Вершина H захвачена',
        description: 'В кучу добавлено H-I(12).',
        type: 'success'
    }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {

```

```

    ];

    // --- BORUVKA STEPS ---
    const boruvkaSteps = [
      // Step 1: Five-Year Plan 1 (Inspecting outgoing)
      () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
        // Node 6(G) -> D-G(8)
        // Node 7(H) -> G-H(10)
        // Node 8(I) -> H-I(12)
        setEdges(prev => prev.map(e =>
          ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8']
        ));
        setLogs(prev => [{
          title: 'Первая пятилетка: Каждая деревня выбирает',
          description: 'Каждая из 9 деревень одновременно выбирает',
          text: 'А выбирает A-B(2), D выбирает D-E(3), а G выбирает D-G(8).',
          type: 'info'
        }, ...prev]);
      },
      // Step 2: Concurrently merge into Kolkhozes (Components)
      () => {
        // Concurrently build chosen roads. Merge into:
        // Component 1: {A, B, C} (colored red)
        // Component 2: {D, E, F, G, H, I} (colored gold/olive)
        setVertices(prev => prev.map(v =>
          [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
        ));
        setEdges(prev => prev.map(e => {
          if (['0-1', '1-2'].includes(e.id)) {
            return { ...e, status: 'included', color: 'red' };
          }
          return e;
        }));
      }
    ];
  }
}

```

```

    },
    // Step 5: Check remaining edges & mark as rejected
    () => {
      setEdges(prev => prev.map(e => e.status === 'pend
      setLogs(prev => [{
        title: '    Успех: MST построен Борувкой за 2 шага
        description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
          вес: 51. Благодаря параллельности, мы потрати
        type: 'success'
      }, ...prev]));
    }
  ];

const currentSteps =
  activeAlgo === 'kruskal' ? kruskalSteps :
  activeAlgo === 'prim' ? primSteps :
  boruvkaSteps;

const handleNextStep = () => {
  if (stepIndex < currentSteps.length) {
    currentSteps[stepIndex]();
    setStepIndex(stepIndex + 1);
  }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
  setActiveAlgo(algo);
  setVertices(initialVertices);
  setEdges(initialEdges);
  setStepIndex(0);
  setHeapQueue([]);
  if (algo === 'kruskal') {
    setLogs([
      {
        title: 'Введение в раскраску (Краскал)',
        description: 'Представь, что все 9 вершин гра
        type: 'info',
      },
    ],
  },

```

```

    if (status === 'rejected') return '#ef4444';
    if (status === 'in_heap') return '#0284c7';
    if (status === 'included') {
      if (color === 'red') return '#ef4444';
      if (color === 'blue') return '#3b82f6';
      if (color === 'purple') return '#a855f7';
      if (color === 'mold') return '#10b981';
    }
    return '#475569';
  };

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-slate-800" >
      <div className="flex flex-col lg:flex-row justify-between" >
        <div>
          <div className="flex items-center gap-2 mb-2" >
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-weight-bold uppercase tracking-wider" >
              Сунер-Интерактив 3 в 1
            </span>
            <h3 className="text-2xl font-bold text-white" >
              Алгоритмы
            </h3>
          </div>
          <p className="text-slate-400 text-sm" >
            Наблюдай за работой Краскала (Раскраска),
          </p>
        </div>

        {/* Algorithm Tabs */}
        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center bg-slate-800/50 p-2" >
            <button
              onClick={() => handleReset('kruskal')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              Краскала
            </button>
          </div>
          <div className="flex items-center bg-slate-800/50 p-2" >
            <button
              onClick={() => handleReset('prim')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              Прима
            </button>
          </div>
          <div className="flex items-center bg-slate-800/50 p-2" >
            <button
              onClick={() => handleReset('mst')}
              className={
                "flex items-center gap-1.5 px-3 py-2"
              }
            >
              MST
            </button>
          </div>
        </div>
      </div>
    </div>
  );

```

```

        className="flex items-center justify-center gap-2
        -200 font-semibold text-xs rounded-xl border-1
      >
        <RotateCcw className="w-3.5 h-3.5" />
        <span>Сбросить</span>
      </button>

    <button
      onClick={handleNextStep}
      disabled={stepIndex >= currentSteps.length}
      className={
        "flex items-center justify-center gap-2
        initial cursor-pointer " +
        (stepIndex >= currentSteps.length
          ? "bg-slate-800 text-slate-500 border-1
            : activeAlgo === 'kruskal'
          ? "bg-indigo-600 hover:bg-indigo-500
            : activeAlgo === 'prim'
          ? "bg-emerald-600 hover:bg-emerald-500
            : "bg-rose-600 hover:bg-rose-500 text
        )
      }
    >
      <SkipForward className="w-4 h-4" />
      <span>{stepIndex === 0 ? 'Начать' : stepIndex + 1}</span>
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
  /* Graph Canvas */
  <div className="lg:col-span-2 bg-slate-950 rounded-2xl
    active min-h-[480px] overflow-hidden">

    /* Legend */
    <div className="absolute top-4 left-4 bg-slate-900
      -center gap-3 text-xs">
      <div className="flex items-center gap-1.5">
        <div className="w-3 h-3 rounded-full bg-
```



```

ext-slate-300">
  ⚬ar {stepIndex} / {currentSteps.length}
</div>

{compilerLinked && {
  <div className="absolute bottom-4 right-4
ont-mono text-emerald-300">
    Python: i={liveI ?? "-"}{liveU || liveW
    {liveW !== null ? `, w=${liveW}` : ""}
  </div>
}}

{/* SVG Edges */}
<svg className="absolute inset-0 w-full h-full
spectRatio="none">
  {liveEdges.map(edge => {
    const u = vertices[edge.u];
    const v = vertices[edge.v];
    const strokeColor = getEdgeStroke(edge);
    const isInspecting = edge.status === 'inspect';
    const isRejected = edge.status === 'rejected';
    const isInHeap = edge.status === 'in_heap';

    return (
      <g key={edge.id}>
        <line
          x1={u.x + "%"}
          y1={u.y + "%"}
          x2={v.x + "%"}
          y2={v.y + "%"}
          stroke={strokeColor}
          strokeWidth={isInspecting ? 4.5 : 1}
          strokeDasharray={isRejected ? '4,4' : ''}
          className={"transition-all duration-200"}
        />
        <text
          x={({u.x + v.x} / 2 + "%")}
          y={({u.y + v.y} / 2 + "%")}

```

```

        <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'}}>
            MST построено с помощью {activeAlgo}
        </p>
        <p className="text-slate-300 text-xs mt
            Общий вес минимального остова: 51. Вс
        </p>
    </div>
    })
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border
    <div className="p-4 bg-slate-900 border-b b
        <h4 className="font-bold text-slate-200 t
            Ход выполнения
        </h4>
    </div>

    {activeAlgo === 'prim' && {
        <div className="p-3 bg-slate-900/60 borde
            <p className="text-[11px] font-bold tex
                ⚡ Приоритетная очередь (Min-Heap):
            </p>
            <div className="flex flex-wrap gap-1.5"
                {heapQueue.length === 0 ? {
                    <span className="text-xs text-slate
                ) : {
                    heapQueue.map(({item, idx}) => {
                        <span key={idx} className={"px-2 p
0 shadow-md animate-pulse" : "bg-slate-800
                            {item}
                        </span>
                    })
                })
            </div>
        </div>
    })
}
```

```

<div className="flex bg-slate-950 p-1 rounded
  <button
    onClick={() => setActiveCheatTab('kruskal')}
    className={"px-3 py-1.5 rounded text-xs font-medium"}
    == 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-950 text-slate-400"
  >
    Краскал (Билет 18)
  </button>
  <button
    onClick={() => setActiveCheatTab('prim')}
    className={"px-3 py-1.5 rounded text-xs font-medium"}
    == 'prim' ? "bg-emerald-600 text-white" : "bg-slate-950 text-slate-400"
  >
    Прима (Билет 19)
  </button>
  <button
    onClick={() => setActiveCheatTab('boruvka')}
    className={"px-3 py-1.5 rounded text-xs font-medium"}
    == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-slate-400"
  >
    Борувка (Билет 20)
  </button>
</div>
</div>

```

```

{activeCheatTab === 'kruskal' && (
  <div className="grid grid-cols-1 lg:grid-cols-2"
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
          <Clock className="w-3.5 h-3.5 text-indigo-500" />
        </p>
        <p className="text-2xl font-black text-white">Краскал</p>
        <p className="text-slate-400 text-xs font-medium">Минимальное остовное дерево</p>
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">

```

```

        </pre>
      </div>
    </div>
  </div>
})

```

```

{activeCheatTab === 'prim' && {
  <div className="grid grid-cols-1 lg:grid-cols-2">
    <div className="lg:col-span-1 space-y-4">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-mono">
          <Clock className="w-3.5 h-3.5 text-emerald-400"/>
        </p>
        <p className="text-2xl font-black text-white">Примитивы</p>
        <p className="text-slate-400 text-xs mt-2">Примитивы — это базовые типы данных, которые используются для хранения информации в памяти компьютера. Они являются фундаментом для построения более сложных структур данных и программ.
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-mono">
          <Award className="w-3.5 h-3.5 text-emerald-400"/>
        </p>
        <p className="text-2xl font-black text-white">Награды</p>
        <p className="text-slate-400 text-xs mt-2">Награды — это специальные значки или медали, которые используются для поощрения пользователей за определенные действия. Они могут быть получены за достижение определенных целей, за участие в соревнованиях или за предоставление ценной информации.
      </div>
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-mono">
          <Code className="w-3.5 h-3.5 text-emerald-400"/>
        </p>
        <p className="text-2xl font-black text-white">Код</p>
        <p className="text-slate-400 text-xs mt-2">Код — это набор инструкций, которые используются для управления компьютером. Он может быть написан на любом из множества существующих языков программирования. Код является основой для создания любых программ и приложений.
      </div>
    </div>
    <div className="lg:col-span-2">
      <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-mono">
          <Code className="w-3.5 h-3.5 text-emerald-400"/>
        </p>
        <pre className="text-xs text-emerald-400">
          80 flex-1">

```

```
{`import heapq
```



```

    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: '  Все ко Все (Флойд)',
    desc: 'Матрица + три цикла (',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '). Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-500/30',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover grid" />
          </div>
          <div className="p-4">
            <h3 className={`text-lg font-bold mb-1 ${c.highlightColor}`>{c.title}</h3>
            <p className="text-slate-400 text-sm">{c.desc} <span className={`font-bold ${c.highlightColor}`>{c.highlight}</span>
            </p>
            <div className={`mt-3 text-xs font-mono px-1`>{c.complexity}</div>
          </div>
        </div>
      ))}
    </div>
  );
};

```

```
return () => window.removeEventListener("keydown",
}, []);
```

```
return (
```

```
  <
```

```
    <div className="lg:hidden sticky top-[57px] z-40"
```

```
      <button
```

```
        onClick={() => setOpen(true)}
```

```
        className="w-full flex items-center gap-2.5 p-2"
```

```
        aria-label="Открыть содержание"
```

```
      >
```

```
        <span className="p-1.5 rounded-lg bg-indigo-600"
```

```
          <List className="w-4 h-4" />
```

```
        </span>
```

```
        <span className="min-w-0 flex-1">
```

```
          <span className="block text-[10px] uppercase"
```

```
            Содержание • {index + 1} из {total}
```

```
          </span>
```

```
          <span className="block text-[13px] font-bol"
```

```
        </span>
```

```
      </button>
```

```
    </div>
```

```
{open && (
```

```
  <div className="lg:hidden fixed inset-0 z-[90]"
```

```
    <div className="absolute inset-0 bg-slate-950"
```

```
    <div className="absolute inset-y-0 left-0 w-[100%]"
```

```
      te-[tocIn_.18s_ease-out]">
```

```
      <div className="flex items-center justify-b"
```

```
        <span className="font-bold text-white tex"
```

```
        <button
```

```
          onClick={() => setOpen(false)}
```

```
          className="p-2 -mr-2 text-slate-400 hov"
```

```
          aria-label="Закрыть содержание"
```

```
        >
```

```
          <X className="w-5 h-5" />
```

```
        </button>
```

```
      </div>
```

```

    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-xl font-extrabold text-white">
              Универсальное пособие
            </h1>
            <p className="text-xs text-indigo-400 font-medium">
              Подготовка к экзаменам: Алгоритмы, Структуры данных
            </p>
          </div>
        </div>
      </div>

      <div className="flex items-center w-full lg:w-auto">
        <button
          id="tab-guide-btn"
          onClick={() => window.scrollTo({ top: 0, behavior: 'smooth' })}
          className="hidden sm:flex flex-1 lg:flex-none text-white border border-white/20 rounded-md px-4 py-2"
          title="Содержание пособия"
        >
          <BookOpen className="w-4 h-4" /> Учебное пособие
        </button>
        <button
          id="tab-compiler-btn"
          onClick={onToggleCompiler}

```



```
      href={` ${import.meta.env.BASE_URL}export/full_
      download="full_code_all_pages.txt"
      target="_blank"
      rel="noreferrer"
      className="flex items-center justify-center
      :bg-sky-600/20 border border-sky-500/30 tra
      title="Скачать полный TXT файл со всем кодом
    >
      <FileText className="w-4 h-4" /> TXT
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center justify-center
      er:bg-amber-600/20 border border-amber-500/
      title="Скачать всё пособие одним автономным
    >
      {busy ? <Loader2 className="w-4 h-4 animate
    </button>
  </div>
</div>
</header>
);
};
```

ФАЙЛ 60/96: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотри
```

```

    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x}
            stroke={xing ? "#f43f5e" : "#4f46e5"} />);
    }
    const labels = ["A","B","C","D","E"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
            <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
        </g>);
    }
    return <>{lines}{circles}</>;
  })())
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" style={{
  background: 'white',
  padding: '5px',
  border: '1px solid #ccc',
  z-index: 20,
  display: 'flex',
  justify-content: space-between,
  align-items: center,
  font-family: 'monospace',
  font-size: 1.2em,
  color: 'black'
}}>
  V=5, E=10 { '>' } 3V-6 = 9 { '- ->' } { '\u274C' }
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border border-slate-800" style={{
  width: '100%',
  height: '100%',
  position: 'relative',
  background: 'linear-gradient(to top right, #1a202c 49%, #2d3748 49% 51%, #2d3748 51% 53%, #1a202c 53%)',
  backgroundSize: '300% 300%',
  backgroundPosition: 'top left, top right, bottom left, bottom right',
  backgroundRepeat: 'no-repeat',
  animation: 'gradient 10s ease infinite',
  color: 'white',
  font-family: 'monospace',
  font-size: 1.2em
}}>
  <div className="absolute top-2 left-2 bg-slate-900 p-2" style={{
    width: '100%',
    height: '100%',
    position: 'relative',
    background: 'linear-gradient(to top right, #1a202c 49%, #2d3748 49% 51%, #2d3748 51% 53%, #1a202c 53%)',
    backgroundSize: '300% 300%',
    backgroundPosition: 'top left, top right, bottom left, bottom right',
    backgroundRepeat: 'no-repeat',
    animation: 'gradient 10s ease infinite',
    color: 'white',
    font-family: 'monospace',
    font-size: 1.2em
  }}>K3,3 – НЕПЛАНАРЕН</div>
  <svg className="w-full h-full absolute inset-0" style={{
    width: '100%',
    height: '100%',
    position: 'relative',
    background: 'linear-gradient(to top right, #1a202c 49%, #2d3748 49% 51%, #2d3748 51% 53%, #1a202c 53%)',
    backgroundSize: '300% 300%',
    backgroundPosition: 'top left, top right, bottom left, bottom right',
    backgroundRepeat: 'no-repeat',
    animation: 'gradient 10s ease infinite',
    color: 'white',
    font-family: 'monospace',
    font-size: 1.2em
  }}>
    {(() => {
      const pts = [
        {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
        {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
      ];
      const edges = [
        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
      ];
    })()}
  </svg>
</div>

```

```

        V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
    </div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950">
    ▲ Запомни: K5 и K3,3 – два кита непланарности.
    гомеоморфный K5 или K3,3 – он непланарен. Теорема
</div>
</div>
);
}
```

ФАЙЛ 61/96: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
  ArrowRightLeft,
  Check,
  CheckCircle2,
  ChevronLeft,
  ChevronRight,
  Copy,
  Eye,
  HelpCircle,
  Link2,
  Loader2,
  Pause,
  Play,
  Plus,
  RotateCcw,
  SkipBack,
  SkipForward,
  Square,
}
```

```

-----
type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => Promise<string>; }) => Promise<string>;
  setStderr: (options: { batched: (message: string) => Promise<string>; }) => Promise<string>;
  setStdin: (options: { stdin: () => string | number | null }) => Promise<string>;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_PATH)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_PATH }));
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}

/* -----
/*  Шаблон синхронизации: короткая шапка + минимальный скелет
/*  скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.html)
/*  -----

/* -----
/*  Шпаргалка Python: команды сортировки, циклы, структуры данных
/*  «Вставить» кладёт сниппет в редактор под курсор.
/*  -----

interface Snippet {

```

```

        tip: "Индекс и элемент одновременно: i — позици
        code: "for i, x in enumerate(['a\", \"b\", \"c\"
    },
],
},
{
    group: "Структуры",
    items: [
        {
            label: "Стек (список)",
            tip: "LIFO, как рекурсия DFS: append = push, pop
            code: 'st = []\nst.append("A")    # push\nst.ap
        },
        {
            label: "Очередь (deque)",
            tip: "FIFO для BFS: popleft() за O(1). list.pop
            code: 'from collections import deque\nq = deque
                t(q))\n',
        },
        {
            label: "Куча (heapq)",
            tip: "Мин-куча: (вес, вершина) — как очередь пр
            code: 'import heapq\nh = []\nheapq.heappush(h,
                w, v)\n',
        },
        {
            label: "Матрица n × m",
            tip: "Таблица расстояний: dist[i][j], бесконечн
            code: 'n, m = 3, 4\ndist = [[float("inf")] * m
        },
        {
            label: "Множество visited",
            tip: "Посещённые вершины: in/add за O(1).",
            code: 'visited = set()\nvisited.add("A")\nprint
        },
    ],
},
{

```

```

    if (!savedEditor) return initTemplate;
    if (savedEditor.code === savedEditor.template || sa
    return savedEditor.code;
  });
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Код выполнится .
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState(
  /** Принудительный повтор (глаз / Ctrl+Enter), даже е
  const [traceRequest, setTraceRequest] = useState(0);
  const [copied, setCopied] = useState(false);
  const [stripTab, setStripTab] = useState<"vars" | "hi
  const [outputOpen, setOutputOpen] = useState(true);
  const editorRef = useRef<HTMLTextAreaElement>(null);

  const [desyncedFrom, setDesyncedFrom] = useState<stri
    savedEditor &&
    savedEditor.chapterId !== chapterId &&
    savedEditor.code !== savedEditor.template &&
    savedEditor.code.trim() !== "" &&
    savedEditor.code !== template &&
    savedEditor.code !== initTemplate
      ? chapterId
      : null
  );
  const prevTplRef = useRef({ base: initTemplate, full:

  /** Активная трасса. input нужен, чтобы изменение std
  const [debug, setDebug] = useState<{ result: DebugRes
  /** Esc оставляет неизменённый текст в редакторе, пок
  const [editHold, setEditHold] = useState<string | nul
  /** Автопроход включается пользователем; первый шаг п
  const [playing, setPlaying] = useState(false);
  const [playbackSpeed, setPlaybackSpeed] = useState<num
    if (typeof window === "undefined") return 1;
    const saved = Number(window.localStorage.getItem(PL
    return PLAYBACK SPEEDS.includes(saved as (typeof PL

```

```

    setStripTab("vars");
    setPlaying(false);
    setEditHold(null);
    setDebug(null);
    setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
  try {
    await navigator.clipboard.writeText(code);
    setCopied(true);
    setTimeout(() => setCopied(false), 1600);
  } catch {
    setRuntimeMessage("Не получилось скопировать – бу
  }
}, [code]);

/* — Пошаговый отладчик —————

/** Связь больше не зависит от эталонных строк: совпа
const hasViz = sync.variables.length > 0;

/**
 * Снимок ПОСЛЕ выделенной строки. Контейнеры берём и
 * (там уже виден append/st[i][j]=...), но индексы цикл
 * события: после st[i][j]= Python уже мог увеличить
 */
const snapshotAt = useCallback(({result: DebugResult,
  const step = result.steps[idx];
  const next = result.steps[idx + 1];
  const afterLocals = next?.locals ?? result.finalLoca
  const afterValues = next?.values ?? result.finalVal
  const beforeValues = step?.values ?? {};
  const line = step?.line ? source.split("\n")[step.l
  // Нас интересуют только простые скалярные цели (`i
  // `i +=`). Контейнеры и так всегда берутся из afte
  // внутри print(f"i={i}") не будет ошибочно принят
  const assignment = line.match(

```

```

// визуализации согласованный снимок, без side-effect
useEffect(() => {
  if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
  (nextIdx: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
  []
);

const stepBy = useCallback(
  (delta: number) => {
    setDebug((d) => {
      if (!d || d.result.steps.length === 0) return d
      const idx = Math.max(0, Math.min(d.result.steps
      return { ...d, idx };
    });
  },
  []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {
  setPlaying(false);
  setEditHold(code);
  setDebug(null);
  clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранитс
useEffect(() => {

```



```

        setPlaying(autoPlay && result.steps.length > 1);
        const printed = [...stdout, ...stderr].join("");
        setOutput(printed || "Программа ничего не вывела");
        setRuntimeMessage(
            result.truncated
            ? `Выполнение остановлено после ${DEBUG_STEP} шага`
            : "Трасса готова автоматически. ← → — шаги, пауза — пробел";
        );
    } catch (error) {
        const message = errorText(error);
        setDebug({ result: { steps: [], error: message, truncated: true } });
        setOutput(`Ошибка:\n${message}`);
        setRuntimeMessage("Не удалось выполнить код — произошла ошибка");
    } finally {
        setRunning(false);
    }
}, [chapterId, code, stdin, traceRequest, running, runtimeReady]);

// Код выполняется сам: debounce даёт спокойно допечать вывод
// изменился во время загрузки Pyodide, после завершения загрузки
useEffect(() => {
    if (running || editHold === code) return;
    if (debug?.src === code && debug.input === stdin && debug.truncated) {
        const timer = window.setTimeout(() => void debugRun(), 100);
        return () => window.clearTimeout(timer);
    }
}, [code, stdin, traceRequest, running, runtimeReady]);

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки)
useEffect(() => {
    if (!debug) return;
    const onKey = (e: KeyboardEvent) => {
        const target = e.target as HTMLElement | null;
        if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) {
            if (e.metaKey || e.ctrlKey || e.altKey) return;
            if (e.key === "ArrowRight") {
                e.stopPropagation();
                e.preventDefault();
                setPlaying(false);
            }
        }
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
}, [debug]);

```

```
-----
const debugChanged = curDebug?.step ? changedVars(curDebug) : {}
const debugOrdered = curDebug?.step ? orderVars(shownDebug) : []
```

```
/** Одна и та же кнопка запуска всегда видна: из редактора
 * проигрывает её, в отладчике переключает play/pause
```

```
const runOrTogglePlayback = useCallback(() => {
  if (running) return;
  if (!debug || debug.result.steps.length === 0) {
    void debugRun(true);
    return;
  }
  if (playing) {
    setPlaying(false);
    return;
  }
  if (debug.idx >= debug.result.steps.length - 1) stepToNext();
  setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);
```

```
/** Вставка сниппета из шпаргалки в позицию курсора.
```

```
const insertSnippet = useCallback(({snippet: string}) => {
  const ta = editorRef.current;
  setCode((current) => {
    if (!ta) return current + (current.endsWith("\n") ? "" : "\n");
    const start = ta.selectionStart ?? current.length;
    const end = ta.selectionEnd ?? start;
    const padded = (start > 0 && !current.slice(0, start).endsWith("\n")) ? "\n" : "";
    const next = current.slice(0, start) + padded + snippet + current.slice(end, current.length);
    requestAnimationFrame(() => {
      ta.focus();
      ta.selectionStart = ta.selectionEnd = start + padded.length;
    });
    return next;
  });
}, []);
```

```
return (
  <aside
```

```

        digo-400 transition-colors" />
    </div>
  })

  {/* Левый край меняет ширину на десктопе. */}
  {onWidthChange && (
    <div
      role="separator"
      aria-orientation="vertical"
      aria-label="Потяните, чтобы изменить ширину п
      title="Потяните, чтобы изменить ширину панели
      className="hidden lg:block absolute top-0 bot
      onPointerDown={(e) => {
        const el = e.currentTarget;
        el.setPointerCapture(e.pointerId);
        const startX = e.clientX;
        const startW = width ?? 560;
        const onMove = (ev: PointerEvent) => {
          const next = Math.round(startW + (startX
            onWidthChange(Math.max(380, Math.min(next
          });
          const onUp = () => {
            el.removeEventListener("pointermove", onM
            el.removeEventListener("pointerup", onUp
          };
          el.addEventListener("pointermove", onMove a
          el.addEventListener("pointerup", onUp as Ev
        })
      >
        <div className="absolute left-1/2 top-1/2 -tr
          digo-500 transition-colors" />
        </div>
      })
    {/* — Заголовок панели
    <div className="shrink-0 flex items-center gap-2
      <span className={`w-2 h-2 rounded-full shrink-0
      <Terminal className="w-4 h-4 text-emerald-400 s
      <span className="text-[13px] font-bold text-whi

```

```

        onClick={resync}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Сбросить к шаблону страницы"
      >
        <RotateCcw className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip side="bottom" content="Скрыть панель"
      <button
        type="button"
        onClick={onClose}
        className="p-1.5 rounded-lg text-slate-400"
        aria-label="Закрыть компилятор"
      >
        <X className="w-4 h-4" />
      </button>
    </Tooltip>
    <Tooltip
      side="bottom"
      content="Вставить полный эталонный код этого"
    >
      <button
        type="button"
        onClick={() => {
          setCode(template);
          setDesyncedFrom(null);
          setPlaying(false);
          setEditHold(null);
          setDebug(null);
          setTraceRequest((n) => n + 1);
        }}
        className="p-1.5 rounded-lg text-indigo-300"
        aria-label="Вставить эталонный код"
      >
        <Eye className="w-4 h-4" />
      </button>
    </Tooltip>
    <label

```

```

    }` }
  >
    <HelpCircle className="w-3.5 h-3.5" /> Подс
  </button>
  <div className="ml-auto pr-1">
    <Tooltip
      side="bottom"
      content={
        hasViz
          ? "Связь идёт по именам ниже: реальны
а не сравнивается."
          : "На выбранной странице нет интеракт
        }
      >
        <span
          tabIndex={0}
          className={`cursor-help inline-flex item
            hasViz ? "text-emerald-400" : "text-s
          }` }
        >
          <Link2 className="w-3 h-3" />
          {hasViz ? "синхронизировано" : "нет виз
        </span>
      </Tooltip>
    </div>
  </div>

{stripTab === "vars" ? {
  <div className="px-3 pb-2.5 space-y-2 max-h-3
    <div className="flex items-center gap-2 min
      <Tooltip side="bottom" content="Открыть э
        <button
          type="button"
          onClick={onOpenGuide}
          className="inline-flex items-center g
        700 text-slate-300 hover:text-white hover:b
      >
        <span className="truncate">{chapterTi

```

```

        className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald-300
        >
        {v.name}
      </code>
    </Tooltip>
  )))
  {sync.stepNote && {
    <Tooltip side="bottom" content={`War
    <span
      tabIndex={0}
      className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
    >
      <ArrowRightLeft className="w-3 h-3
      <span className="truncate max-w-5
    </span>
    </Tooltip>
  }}
</div>
)}
</div>
) : (
  <div className="px-3 pb-2.5 max-h-36 overflow
    {SNIPPETS.map(({gItem}) => {
      <div key={gItem.group}>
        <div className="text-[9px] font-bold up
        <div className="flex flex-wrap gap-1.5"
          {gItem.items.map(({s}) => {
            <Tooltip key={s.label} side="bottom
            <button
              type="button"
              onClick={() => insertSnippet(s.
              className="inline-flex items-ce
            -[10.5px] text-indigo-300 hover:text-white
              title=""
            >
              <Plus className="w-3 h-3 text-s

```

```

<Tooltip content="Шаг назад (←) – автопро
  <button type="button" onClick={() => {
    -md text-slate-400 hover:text-white hover:b
    <ChevronLeft className="w-4 h-4" />
  } />
</button>
</Tooltip>
<Tooltip content="Шаг вперёд (→) – автопро
  <button type="button" onClick={() => {
    sult.steps.length - 1} className="p-1 round
    tion-colors">
    <ChevronRight className="w-4 h-4" />
  } />
</button>
</Tooltip>
<Tooltip content="В конец трассы">
  <button type="button" onClick={() => {
    tep || debug.idx >= debug.result.steps.leng
    0 disabled:opacity-40 transition-colors">
    <SkipForward className="w-4 h-4" />
  } />
</button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-
  {curDebug?.step ? (
    <>
      шаг {debug.idx + 1}/{debug.result.s
      {curDebug.step.func !== "<module>"
    } />
  ) : (
    "шагов нет – исправьте ошибку и начини
  )}
</span>
{debug.result.truncated && (
  <Tooltip content={`Выполнение остановлен
    <span tabIndex={0} className="cursor-l
    rounded-full px-1.5 py-0.5">
      ЛИМИТ
    </span>
  } />
</Tooltip>
)}

```

```

        {active ? "▶" : ln}
      </span>
      <span>{line || " "}</span>
    </div>
  );
  }}}
</div>

{/* панель переменных – «Variables», как в VS Code */}
<div className="shrink-0 max-h-[34%] overflow-hidden" >
  <div className="flex items-center gap-1.5" >
    <Variable className="w-3 h-3 text-indigo-500" />
    Переменные на этом шаге
    <span className="normal-case tracking-normal" />
  </div>
  {debug.result.error && (
    <pre className="mt-1 whitespace-pre-wrap" />
  )}
  {debugOrdered.length === 0 && !debug.result.error && (
    <div className="mt-1 text-[11px] text-slate-500" />
  )}
</div>
  )}
  <div className="mt-1 flex flex-wrap gap-1" >
    {debugOrdered.map(([name, value]) => {
      const changed = debugChanged[name];
      const synced = syncVarSet.has(name);
      return (
        <span
          key={name}
          className={`inline-flex items-baseline gap-1.5`
            + `border-1px rounded-sm`
            + `padding-2px`
            + `font-family: monospace`
            + `font-size: 0.9em`
            + `text-slate-700`
            + `width: 100px`
            + `height: 1.2em`
            + `vertical-align: middle`
            + `display: inline-block`
            + `text-align: left`
            + `white-space: nowrap`
            + `overflow: hidden`
            + `text-overflow: ellipsis`
            + `border-color: ${changed ? "border-amber-500/50 bg-amber-500/10" : "border-emerald-500/40 bg-emerald-500/10"}`
            + `border-width: 1px`
            + `border-radius: 4px`
            + `background-color: ${synced ? "border-slate-700 bg-slate-700/10" : "border-slate-700 bg-slate-700/10"}`
          >
            <span className={changed ? "text-amber-500" : "text-emerald-500"} />
            {value}
          </span>
        </div>
      )
    })}
  </div>
  <div className="mt-1 flex flex-wrap gap-1" >
    {debugOrdered.map(([name, value]) => {
      const changed = debugChanged[name];
      const synced = syncVarSet.has(name);
      return (
        <span
          key={name}
          className={`inline-flex items-baseline gap-1.5`
            + `border-1px rounded-sm`
            + `padding-2px`
            + `font-family: monospace`
            + `font-size: 0.9em`
            + `text-slate-700`
            + `width: 100px`
            + `height: 1.2em`
            + `vertical-align: middle`
            + `display: inline-block`
            + `text-align: left`
            + `white-space: nowrap`
            + `overflow: hidden`
            + `text-overflow: ellipsis`
            + `border-color: ${changed ? "border-amber-500/50 bg-amber-500/10" : "border-emerald-500/40 bg-emerald-500/10"}`
            + `border-width: 1px`
            + `border-radius: 4px`
            + `background-color: ${synced ? "border-slate-700 bg-slate-700/10" : "border-slate-700 bg-slate-700/10"}`
          >
            <span className={changed ? "text-amber-500" : "text-emerald-500"} />
            {value}
          </span>
        </div>
      )
    })}
  </div>
</div>

```



```

        value={stdin}
        onChange={(event) => {
            setEditHold(null);
            setStdin(event.target.value);
        }}
        spellCheck={false}
        placeholder="строки ввода через ↵"
        className="w-full bg-transparent font-mono"
    />
</div>
</div>

```

```

{/* — Вывод ————— */
<div className="shrink-0 border-t border-slate-800"
  <button
    type="button"
    onClick={() => setOutputOpen((o) => !o)}
    className="flex items-center justify-between gap-2"
    colors"
  >
    <span className="inline-flex items-center gap-2"
      <Terminal className="w-3.5 h-3.5 text-emerald-500"
    />
    <span className="inline-flex items-center gap-2"
      {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500"
      <span className="text-slate-600">{outputOpen ? "Вывод" : "Вход"}
    />
    </span>
  </button>
  {outputOpen && (
    <pre className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
      {output}
    </pre>
  )}
  <div className="px-3 py-1.5 border-t border-slate-800"
</div>
</aside>
);

```



```

        setTimeout(() => {
          // Step 2: Guest leaves with happy face, queue sh
          setQueue(prev => prev.map((c, idx) => idx === 0 ?

          setServedHistory(prev => [`${headGuest.emoji} ${h

          setTimeout(() => {
            setQueue(prev => prev.slice(1));
            setIsServing(false);
            addLog(`  ${headGuest.name} поел и ушел сытым.
          }, 400);
        }, 900);
      }, [queue, isServing]));

const reset = () => {
  counter = 3;
  setQueue([
    { id: 'r1', name: 'Студент Вася', emoji: ' ', col
      т голода после матана!', animState: 'in' },
    { id: 'r2', name: 'Кодер Семён', emoji: ' ', col
      апишу ИИ за порцию борща!', animState: 'in' },
    { id: 'r3', name: 'Кот Борис', emoji: ' ', col
      без сметаны, плиз.', animState: 'in' },
  ]));
  setServedHistory([]);
  setIsServing(false);
  setLog([' Столовая сброшена. Снова голодная толпа!
  setTimeout(() => {
    setQueue(prev => prev.map(c => ({ ...c, animState
  }, 500);
};

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-indigo-950/40 border border-in
      <div className="text-3xl"> </div>
      <div>

```

```
</div>
```

```
{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
```

```
<div className="grid grid-cols-1 md:grid-cols-12">
```

```
  {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
```

```
  <div className={`md:col-span-9 bg-slate-900 rounded-3xl
    -[300px] overflow-hidden ${shakeEmpty ? 'animate-pulse'}`>
```

```
    <div className="absolute top-4 left-4 text-[1.2em] font-mono">
```

```
      Очередь голодных гостей (Буфер FIFO / BFS)
```

```
      {liveQ && <span className="ml-2 text-emerald-400">{liveQ.length}</span>}
```

```
    </div>
```

```
  <div className="flex-1 flex flex-col justify-between">
```

```
    <div className="flex items-end justify-between">
```

```
      {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
```

```
      <div className="flex flex-col items-center">
```

```
        <div className="relative">
```

```
          <div className="absolute -top-10 left-1/2 -translate-x-1/2">
```

```
            <div className="w-1.5 h-6 bg-slate-900 rounded-3xl">
```

```
              <div className="w-2 h-4 bg-slate-400 rounded-3xl">
```

```
            </div>
```

```
            <span className="text-5xl block animate-pulse">{liveQ.length}</span>
```

```
          </div>
```

```
        <div className="text-center">
```

```
          <span className="text-3xl block">{liveQ.length}</span>
```

```
          <span className="text-[10px] font-bold font-mono tracking-wider">
```

```
            Очередь голодных гостей (Буфер FIFO / BFS)
```

```
            {liveQ && <span>{liveQ.length}</span>}
```

```
          </span>
```

```
        </div>
```

```
      </div>
```

```
    {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
```

```
    <div className="flex flex-col items-center">
```

```
      <span className="text-2xl animate-pulse">{liveQ.length}</span>
```

```
      <span className="text-[8px] font-mono tracking-wider">
```

```

        w-14 h-14 rounded-2xl border-1
        relative
            ${customer.color}
            ${isHead ? 'ring-4 ring-amber-500' : ''}
        `}>
        <span className="text-3xl">{customer.name}</span>

        {/* Индексы HEAD/TAIL */}
        {(isHead || isTail) && (
            <span className={`
                absolute -bottom-2 text-[10px]
                ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
            `}>
                {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}
            </span>
        )}
        </div>
    </div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800">
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
    <div className="absolute top-4 left-4 text-[10px] font-mono">
        Сытые гости (Архив FIFO)
    </div>

    <div className="absolute top-4 right-4 flex items-center justify-between text-[10px] font-mono">
        order-emerald-800/80 text-[9px] font-mono">

```



```

        newNodes[curr].children[c] = count;
        newNodes[count] = {
          id: count,
          char: c,
          parent: curr,
          children: {},
          fail: 0,
          term_link: 0,
          is_terminal: false,
          words: [],
          depth: newNodes[curr].depth + 1
        };
      }
      curr = newNodes[curr].children[c];
    }
    if (!newNodes[curr].words.includes(word)) {
      newNodes[curr].words.push(word);
      newNodes[curr].is_terminal = true;
    }
  });

  let currentX = 0;
  const paddingX = 70;
  const paddingY = 80;

  const layoutDFS = (u: number, depth: number) => {
    const childKeys = Object.values(newNodes[u].children);
    newNodes[u].y = 40 + depth * paddingY;

    if (childKeys.length === 0) {
      newNodes[u].x = 40 + currentX * paddingX;
      currentX++;
    } else {
      let leftX = -1;
      let rightX = -1;
      childKeys.forEach(v => {
        layoutDFS(v, depth + 1);
        if (leftX === -1) leftX = newNodes[v].x!;
      });
      rightX = newNodes[childKeys[childKeys.length - 1]].x!;
      newNodes[u].x = (leftX + rightX) / 2;
    }
  };

  layoutDFS(0, 0);

```

```

    if (words.length <= 1) {
        setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
        return;
    }
    const updated = words.filter((w) => w !== wordToRemove);
    setWords(updated);
    buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
    const rootChildren = Object.values(nodes[0].children);
    const queue = [...rootChildren];
    const updatedNodes = { ...nodes };
    rootChildren.forEach((childId) => {
        updatedNodes[childId].fail = 0;
    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
    );
};

// Один шаг BFS
const stepBFS = () => {
    if (bfsQueue.length === 0) {
        setSimulationStep(2);
        setCurrentNode(null);
        setSpeechText(
            'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов! Можешь полюбиться таблице переходов!'
        );
        return;
    }
    const currentId = bfsQueue.shift();
    const current = nodes[currentId];
    const children = current.children;
    const newQueue = [...children];
    bfsQueue.push(...children);
    setBfsQueue(newQueue);
    setCurrentNode(currentId);
    setSimulationStep(1);
    setSpeechText(
        `Переходим к вершине с ID ${currentId}!`
    );
};

```



```

        <path d="M 165 110 Q 155 115 168 118" s
    }}

    {/* Улыбка / жабры */}
    <path d="M 130 80 Q 125 100 130 120" stro
    <path d="M 120 85 Q 115 100 120 115" stro
</svg>

    {/* Декоративные пузыри */}
    <div className="absolute -top-1 right-4 tex
    <div className="absolute top-8 right-1 text
</div>
    <span className="mt-3 bg-rose-600/30 border b
        Эхо-Лосось Сеня
    </span>
</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
    tween min-h-[140px]">
    {/* Треугольник баббла */}
    <div className="absolute -left-3 top-1/2 -tran
        slate-800 border-b-[12px] border-b-transpar

    <div className="flex items-center space-x-2 t
        <Volume2 className={`w-4 h-4 ${isTalking ?
        <span>Прямое вещание из аквариума:</span>
    </div>

    <p className="text-slate-200 text-base leading
        {speechText}
    </p>

    <div className="mt-4 pt-3 border-t border-sla
        <div className="text-xs text-slate-400 flex
            <Info className="w-3.5 h-3.5 text-blue-40
                Статус: {simulationStep === 0 ? 'Бор гото
                оен!' }

```

```
<h5 className="text-white font-bold mb-3 flex i
  <span> </span> Дерево и Суффиксные (fail) ссы
</h5>
```

```
{(() => {
  let maxX = 0;
  let maxY = 0;
  Object.values(nodes).forEach(n => {
    if (n.x && n.x > maxX) maxX = n.x;
    if (n.y && n.y > maxY) maxY = n.y;
  });
  const svgWidth = Math.max(maxX + 80, 400);
  const svgHeight = Math.max(maxY + 60, 200);

  return (
    <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
      <defs>
        <marker id="arrowhead" markerWidth="6"
          <polygon points="0 0, 6 3, 0 6" fill=
        </marker>
        <marker id="fail-arrow" markerWidth="6"
          <polygon points="0 0, 6 3, 0 6" fill=
        </marker>
      </defs>

    {/* Draw Edges */}
    {Object.values(nodes).map(node => {
      if (node.id === 0) return null;
      const parent = nodes[node.parent];
      if (!parent.x || !parent.y || !node.x ||

      const isCurrent = currentNode === node.

      return (
        <g key={`edge-${node.id}`}>
          <line
            x1={parent.x}
            y1={parent.y + 16}
```

```

        stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
        strokeWidth={isCurrent ? 2 : 1.5}
        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
      />
    );
  }}}

```

```

{/* Draw Nodes */}
Object.values(nodes).map(node => {
  if (!node.x || !node.y) return null;
  const isRoot = node.id === 0;
  const isCurrent = currentNode === node.id;
  const isInQueue = bfsQueue.includes(node.id);

```

```

  let fill = '#1e293b';
  let stroke = '#475569';

```

```

  if (isCurrent) {
    fill = '#2563eb';
    stroke = '#60a5fa';
  } else if (isInQueue) {
    fill = '#b45309';
    stroke = '#fbbf24';
  } else if (node.is_terminal) {
    fill = '#059669';
    stroke = '#34d399';
  }

```

```

  return (
    <g key={`node-${node.id}`}>
      <circle
        cx={node.x}
        cy={node.y}
        r="16"
        fill={fill}
        stroke={stroke}
        strokeWidth="2"

```

```

        <span
          key={w}
          className="bg-slate-800 border border-slate-900
            border-radius-10px px-2 py-1 gap-1 group"
        >
          {w}
          <button
            onClick={() => removeWord(w)}
            className="text-slate-500 hover:text-white"
            title="Удалить"
          >
            ×
          </button>
        </span>
      )})
    </div>
  </div>

  <form onSubmit={handleAddWord} className="flex flex-col gap-2" >
    <input
      type="text"
      value={newWord}
      onChange={(e) => setNewWord(e.target.value)}
      placeholder="НОВОЕ СЛОВО..."
      maxLength={8}
      className="bg-slate-800 border border-slate-900
        border-radius-10px px-2 py-1 outline-none focus:border-blue-500"
    />
    <button
      type="submit"
      className="bg-slate-700 hover:bg-slate-600 text-white
        border border-slate-800 border-radius-10px px-2 py-1
        transition-colors"
    >
      <Plus className="w-3.5 h-3.5" /> Добавить
    </button>
  </form>
</div>

```

```

    }` }
  >
  <td className="py-2.5 px-3 flex items-center" >
    <span
      className={`w-2 h-2 rounded-full
        node.id === 0
          ? 'bg-purple-500'
          : isCurrent
          ? 'bg-blue-400 animate-pulse'
          : isInQueue
          ? 'bg-amber-400'
          : 'bg-slate-600'
        }` }
    ></span>
    <span>#{node.id}</span>
    {node.id === 0 && <span className="font-size-2" >
  </td>
  <td className="py-2.5 px-3" >
    <span className="bg-slate-800 border-1px solid black" >
      {node.char}
    </span>
  </td>
  <td className="py-2.5 px-3 text-slate-500" >
    {node.id === 0 ? '-' : `#${node.id}`}
  </td>
  <td className="py-2.5 px-3" >
    {node.id === 0 ? (
      <span className="text-slate-500" >
        {node.term}
      ) : (
        <span className="bg-indigo-950 text-white" >
          #{node.fail}
        </span>
      )}
  </td>
  <td className="py-2.5 px-3" >
    {node.id === 0 || node.term_line === 0 ? (
      <span className="text-slate-500" >
        {node.term}
      ) : (

```

```

import React, { useState, useEffect, useCallback, useMe
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRi
import { useVizRuntime, vizNumber } from '../data/vizSt

// ===== TYPES =====
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>;           // node ind
  highlightNodes: number[];                   // current
  mergeChildren?: [number, number];           // pair be
  arrayHighlight?: [number, number];           // [L, R]
  result?: number;                            // partial
  lazyState?: Record<number, number>;         // node in
  promisePushed?: number;                     // узел, у
}

// ===== PURE LOGIC: build tree =====
export function buildTreeFull(arr: number[]): Record<num
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0)
  }
  go(1, 0, n - 1);
  return tree;
}

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

```

    }
    go(1, 0, n - 1);
    steps.push({ id: id++, desc: `  Дерево построено! Корень: ${arr[0]}
      Highlight: [0, n - 1] });
    return steps;
  }

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
  const n = arr.length;
  const tree = buildTreeFull(arr);
  const steps: Step[] = [];
  let id = 0;

  function go(v: number, l: number, r: number, ql: number, qr: number) {
    if (ql > r || qr < l) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 2, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
      return 0;
    }
    if (ql <= l && r <= qr) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
        codeLine: 4, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
      return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
      codeLine: 6, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`,
      codeLine: 8, treeState: tree, highlightNodes: [v], arrayHighlight: [l, r] });
    return res;
  }

  const ans = go(1, 0, n - 1, qL, qR);
  steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]) = ${ans}`,
    codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
  return steps;
}

```



```

    if (l < r) {
      steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}`,
        , arrayHighlight: [qL, qR], result: res });
    }
  }
  steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
    ighlightNodes: [1], arrayHighlight: [qL, qR], result: res });
  return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }

```

```

function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? null };
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
    treeState, v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
  "build(v, l, r) {",
  "  if (l === r) { tree[v] = A[l]; return; }",
  "  // -----",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  build(2*v, l, mid);",
  "  build(2*v+1, mid+1, r);",
  "  tree[v] = tree[2*v] + tree[2*v+1];",
  "}"
];

```

```

const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",

```

```

-----
export function genLazySteps(arr: number[], uL: number,
  const n = arr.length;
  const steps: Step[] = [];
  let id = 0;
  const tree: Record<number, number> = {};
  const lazy: Record<number, number> = {};
  for (const k of Object.keys(buildTreeFull(arr))) {
    tree[Number(k)] = buildTreeFull(arr)[Number(k)];
    lazy[Number(k)] = 0;
  }
  const full = buildTreeFull(arr);

  const snap = {desc: string, codeLine: number, hl: number,
    steps.push({
      id: id++,
      desc,
      codeLine,
      treeState: { ...tree },
      highlightNodes: [...hl],
      lazyState: { ...lazy },
      ...extra,
    });
  };
  snap(`Старт: дерево построено (корень = ${full[1]}).
  `);

  function pushDown(v: number, l: number, r: number) {
    if (lazy[v] !== 0) {
      const m = (l + r) >> 1;
      const d = lazy[v];
      snap(`push(${v}): обещание +${d} с узла ${v} уходит
        lazy.` , 8, [v, 2 * v, 2 * v + 1], { promisePush
      tree[2 * v] = (tree[2 * v] ?? 0) + d * (m - l + 1);
      lazy[2 * v] = (lazy[2 * v] ?? 0) + d;
      tree[2 * v + 1] = (tree[2 * v + 1] ?? 0) + d * (r - m);
      lazy[2 * v + 1] = (lazy[2 * v + 1] ?? 0) + d;
      lazy[v] = 0;
      snap(`Обещание ${v} исполнено и снято (lazy[${v}])

```

```

        snap(`query(v=${v}, [${l}..${r}]): целиком внутри
            sult: res`));
        return;
    }
    pushDown(v, l, r);
    const m = (l + r) >> 1;
    snap(`query(v=${v}, [${l}..${r}]): частичное пересече
        `);
    query(2 * v, l, m);
    query(2 * v + 1, m + 1, r);
}
res = 0;
snap(`Теперь запрос sum([${uL}..${uR}]) – проверим, ч
    query(1, 0, n - 1);
snap(`    sum([${uL}..${uR}]) = ${res} – совпадает с че
return steps;
}

```

// ===== PLAYER COMPONENT =====

```

function Player({ steps, color }: { steps: Step[]; colo
    const [idx, setIdx] = useState(0);
    const [playing, setPlaying] = useState(false);
    const [speed, setSpeed] = useState(700);

    useEffect(() => { setIdx(0); setPlaying(false); }, [s

    useEffect(() => {
        if (!playing || idx >= steps.length - 1) { if (idx :
            const t = setTimeout(() => setIdx(i => i + 1), speed
            return () => clearTimeout(t);
        }, [playing, idx, steps, speed]);

        const step = steps[idx] || steps[0];
        if (!step) return null;

        const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
            border-indigo-500 bg-indigo-600', childRing: 'ring-
            -900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e

```

```

        {promise !== 0 && {
          <span className="mt-0.5 px-1 rounded-full b
            o whitespace-nowrap">
              +{promise}
            </span>
          }}
        </div>
        {(nd.left || nd.right) && {
          <div className="flex flex-col items-center mt
            <div className="w-px h-2 bg-slate-700" />
            <div className="flex justify-around w-full l
              {nd.left && renderNode(nd.left)}
              {nd.right && renderNode(nd.right)}
            </div>
          </div>
        }}
      </div>
    );
  }, [step, clr, liveNode]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
    /* Header */
    <div className={`px-4 py-2.5 flex items-center ju
      -emerald-950/50' : 'bg-amber-950/50'} border-b l
      <h4 className="font-bold text-sm text-white fle
        <span className="text-[10px] font-mono text-sla
      </div>

    /* Array strip */
    <div className="px-3 py-2 bg-slate-950 border-b b
      {arr.map((v, i) => {
        const inRange = step.arrayHighlight && i >= s
        return (
          <div key={i} className={`flex flex-col item
            go' ? 'bg-indigo-950 border-indigo-500' : c
            er-500'} -translate-y-0.5` : 'bg-slate-900 l

```

```

    })
  </div>

  { /* Controls */
  <div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-slate-800" >
    <div className="flex items-center bg-slate-900 rounded" >
      <button onClick={() => { setPlaying(false); setSpeed(1200); }} title="Сброс" ><RotateCcw className="w-3 h-3" /></button>
      <button onClick={() => { setPlaying(false); setSpeed(700); }} title="Норма" ><RotateCcw className="w-3 h-3" /></button>
      <button onClick={() => setPlaying(!playing)} title={playing ? "Пауза" : "Воспроизведение"} ><Pause className="w-3 h-3" /></button>
      <button onClick={() => { setPlaying(false); setSpeed(120); }} title="Турбо" ><RotateCcw className="w-3 h-3" /></button>
    </div>
    <select value={speed} onChange={e => setSpeed(Number(e.target.value))} >
      <option value={1200}>Медл.</option>
      <option value={700}>Норма</option>
      <option value={350}>Быстро</option>
      <option value={120}>Турбо</option>
    </select>
  </div>

  { /* Progress bar */
  <div className="h-1 bg-slate-950" ><div className="bg-rose-500" style={{ width: ((currentSpeed + 1) / total) * 100 : 0}%` }} /></div>
</div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {

```

```

const effAdd = liveAdd ?? uAdd;
const lazySteps = useMemo(() => genLazySteps(arr, uL,

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border border-
    /* ---- TOP BAR: array editor + query range ----
  <div className="mb-6 space-y-4">
    <div className="flex flex-col lg:flex-row gap-4">
      /* Array input */
      <form onSubmit={handleApply} className="flex-
        <div className="flex items-center justify-b
          <label className="text-[10px] font-bold up
        label>
          <button type="button" onClick={randomize}
        "><RefreshCw className="w-3.5 h-3.5" /></bu
      </div>
      <div className="flex gap-2">
        <input
          value={customInput}
          onChange={e => setCustomInput(e.target.v
          className="flex-1 bg-slate-900 border bor
        one focus:border-indigo-500"
          placeholder="5, 8, 3, 12, 7, 2"
        />
        <button type="submit" className="bg-indigo
        ition-colors">Применить</button>
      </div>
      <div className="flex flex-wrap gap-1.5 pt-1">
        {arr.map((v, i) => (
          <span key={i} className={`border px-2 p
          ext-amber-200 ring-2 ring-amber-400/30" : "l
            <span className={liveI === i ? "text-
          </span>
        </span>
      )}}
      <span className="text-xs text-slate-500 f
    </div>

```

```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1"
  <button onClick={() => setView('build')} className="
    -colors ${view === 'build' ? 'border-indigo-500
    -slate-200'}`">
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className="
    on-colors ${view === 'queries' ? 'border-emerald-500
    er:text-slate-200'}`">
    <Search className="w-3.5 h-3.5" /> Запрос: св
  </button>
  <button onClick={() => setView('lazy')} className="
    colors ${view === 'lazy' ? 'border-amber-500
    e-200'}`" title="Массовое обновление на отрезке
    Обновление на отрезке (Lazy)
  </button>
  <button onClick={() => setView('theory')} className="
    n-colors ${view === 'theory' ? 'border-blue-500
    te-200'}`">
    <Info className="w-3.5 h-3.5" /> Почему / Сравн
  </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
  <SimPanel
    key={`build-${arr.join('-')}`}
    title="Построение дерева (рекурсия)"
    icon=""
    steps={buildSteps}
    code={BUILD_CODE}
    color="indigo"
    arr={arr}
  />
)}

```

```

{view === 'queries' && (

```

```

        border-slate-700 text-white text-xs font-mono
        l ? 'задана переменной add в Python-компиляторе'
    </div>
    <p className="text-[11px] text-slate-500 margin-top-1">
        Ко всем элементам отрезка прибавляем <b>сумму</b>
        емся только по границе, а внутри вешаем стили (суммы).
        {liveAdd !== null && <span className="ml-1">
        {liveV !== null && <span className="ml-2">
    </p>
</div>
<SimPanel
  key={`lazy-${arr.join('-')}-${uL}-${uR}-${effAdd}`}
  title={`update([${uL}..${uR}], +${effAdd})`}
  icon=""
  steps={lazySteps}
  code={LAZY_CODE}
  color="amber"
  arr={arr}
  liveNode={liveV}
/>
</div>
)}

{view === 'theory' && (
  <div className="space-y-5">
    /* Why it splits this way */
    <div className="bg-slate-950 p-5 rounded-xl border">
      <h4 className="text-base font-bold text-white">
        во разбилось именно так для N={arr.length}?
      <p className="text-sm text-slate-300 leading-relaxed">
        Каждый узел берёт <code className="bg-slate-900 px-1">
        к получает <code className="bg-slate-900 px-1">
        -slate-900 px-1 rounded font-mono text-emerald-400
        евая часть забирает на 1 больше (округление)
      </p>
      <div className="bg-slate-900 p-4 rounded-lg">
        <div className="text-slate-400">Корень: L

```



```
<div className="flex-1 overflow-y-auto px-3 pb-4" data-bbox="111 820 750 853">
  {groups.length === 0 && <p className="text-sm t
```

```

{groups.map(({category, items}) => (
  <div key={category}>
    <div className="text-[10px] font-bold uppercase">
      <div className="space-y-1">
        {items.map((chapter) => {
          const isSelected = chapter.id === selectedChapterId
          const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
          return (
            <button
              key={chapter.id}
              onClick={() => {
                setSelectedChapterId(chapter.id);
                onNavigate?.();
                window.scrollTo({ top: 0, behavior: 'smooth' });
              }}
              className={`w-full text-left px-3 py-2 border-1
                ${isSelected ? "bg-indigo-600/15 border-indigo-600/20" : "bg-slate-800/40 border-transparent"}
              `}
            >
              {chapter.title}
            </button>
          )
        })}
      <div
        className="flex justify-between items-center gap-2"
        style={hasViz && {
          height: 1.5,
          border: "1px solid #444",
          border-radius: 0.5,
          padding: 0.5,
          marginTop: 0.5,
        }}
      >
        <span
          className="w-1.5 h-1.5 rounded-full"
          style={hasViz ? { background: "#444" } : {}}
        />
        <span>Есть интерактивная визуализация</span>
      </div>
    </div>
  </div>
)}

```

```
-----
import { SplayRotationsViz } from "../SplayRotationsViz"
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 *    (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего дерева
 *    поворот – кадры «прицел → отстёгиваем среднее по высоте»
 * 3. «Собери сам» – песочница: подними узел кликами, разбей
 */
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
  const [tab, setTab] = useState<Tab>(defaultTab);

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border">
      <div className="flex flex-wrap items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-indigo-600 text-white">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-indigo-300">Один
            </p>
          </div>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5">
        {[
          { id: "cases" as Tab, label: "1 · Механика: Zig / Zig-Zig / Zig-Zag" },
          { id: "big" as Tab, label: "2 · Большое дерево" },
          { id: "play" as Tab, label: "3 · Собери сам" }
        ]}.map((t) => {
          <button

```

* Здесь ничего не подсказывается: дано настоящее дерево
* поднять отмеченный узел в корень. Клик по узлу = один шаг
* Порядок поворотов выбираешь сам; правильный ли он по сути
* только после того, как узел окажется наверху (или по сути)
*/

```
export interface TNode {  
  key: number;  
  left: TNode | null;  
  right: TNode | null;  
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null })
```

```
const clone = (t: TNode | null): TNode | null =>  
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null
```

/** Заготовки: линия (zig-zig), змейка (zig-zag) и просеивание

```
export const PRESETS: { name: string; hintCase: string; target: number; build: () => TNode } =  
{  
  {  
    name: "Линия влево",  
    hintCase: "Zig-Zig",  
    target: 1,  
    build: () => {  
      const n1 = leaf(1);  
      const n2: TNode = { key: 2, left: n1, right: leaf(3) };  
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };  
      return { key: 6, left: n4, right: leaf(7) };  
    },  
  },  
  {  
    name: "Змейка",  
    hintCase: "Zig-Zag",  
    target: 3,  
    build: () => {  
      const n3 = leaf(3);  
      const n2: TNode = { key: 2, left: leaf(1), right: n3 };  
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };  
      return { key: 6, left: n4, right: leaf(7) };  
    },  
  },  
}
```

```

    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number) {
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

    if (!g) return { node: x.key, kind: "Zig" };

    const xLeft = p.left === x;
    const pLeft = g.left === p;
    if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
    return { node: x.key, kind: "Zig-Zag (сначала нижняя поворот)" };
}

/* ----- раскладка -----

```

```

const preset = PRESETS[presetIdx];

const [tree, setTree] = useState<TNode>(() => preset.t
const [history, setHistory] = useState<TNode[]>([]);
const [moves, setMoves] = useState<{ node: number; co
const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
  (idx = presetIdx) => {
    setPresetIdx(idx);
    setTree(PRESETS[idx].build());
    setHistory([]);
    setMoves([]);
    setShowAnswer(false);
  },
  [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, tar
const expected = useMemo(() => (solved ? null : canon

const clickNode = (key: number) => {
  if (solved) return;
  const p = findPath(tree, key);
  if (p.length < 2) return; // корень крутить не с ке

  const want = canonicalNext(tree, target);
  setHistory((h) => [...h, clone(tree) as TNode]);
  setMoves((m) => [...m, { node: key, correct: want?.
  setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
  if (history.length === 0) return;

```

```

        </button>
      )})
    </div>

    <div className="bg-slate-900 rounded-xl border bo
      <svg
        viewBox={`0 0 ${width} ${height}`}
        className="h-auto block mx-auto"
        style={{ minWidth: Math.min(width * 1.5, 420)
        role="img"
      >
        {edges.map(([a, b], i) => {
          const na = nodes.find((n) => n.key === a);
          const nb = nodes.find((n) => n.key === b);
          if (!na || !nb) return null;
          const onPath = path.has(a) && path.has(b);
          return (
            <line
              key={i}
              x1={na.x}
              y1={na.y}
              x2={nb.x}
              y2={nb.y}
              stroke={onPath ? "#6366f1" : "#475569"}
              strokeWidth={onPath ? 3 : 1.8}
              style={{ transition: "all 700ms cubic-b
            />
          );
        })}
      )})

    {nodes.map((n) => {
      const isTarget = n.key === target;
      const clickable = !solved && n.depth > 0;
      return (
        <g
          key={n.key}
          onClick={() => clickable && clickNode(n)
          style={{

```



```

* сразу видно, что дерево остаётся деревом поиска;
* * под деревом печатается обход слева направо: он ОД
* это и есть доказательство, что поворот ничего не
* * кадр «прицел» ничего не двигает, только подсвечива
* * на кадре «результат» остаются призраки – пунктирн
* местах и стрелки «откуда → куда»;
* * по умолчанию анимация НЕ крутится сама: пользоват
* идёт в своём темпе. Автопрокрутка включается кноп
*/

```

```

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;

```

```

interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Папа, которую крутим на этом шаге: кадр-«прицел». */
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стрелки */
  moved?: NodeId[];
  ms: number;
}

```

```

interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах. */
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
  frames: Frame[];
}

```

```

pos: { x: [160, 50], A: [80, 130], p: [245, 130],
edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 – корень",
text: "40 стало правым сыном двадцатки. Поддеревов
      ева от 40 – это одно и то же место.",
moved: [{"x", "p", "B"}],
ms: RESULT_MS,
},
],
};

```

/* ----- Zig-Zig -----

```

const ZZ_S0 = {
pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZZ_S1 = {
pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZIG: Case = {
key: "zig-zig",
label: "Zig-Zig",
when: "x и p – дети с ОДНОЙ стороны (оба слева или об
rotations: "2 поворота, первым – верхний",
keys: { x: 20, p: 40, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
{
...ZZ_S0,
kind: "start",
title: "Было: прямая линия 60 → 40 → 20",
text: "Три узла выстроились в «бамбук»: каждый сл
      зглаживает.",

```

```
};
```

```
/* ----- Zig-Zag -----
```

```
const ZG_S0 = {
  pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
  edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZG_S1 = {
  pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
  edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZAG: Case = {
  key: "zig-zag",
  label: "Zig-Zag",
  when: "x и p – дети с РАЗНЫХ сторон (змейка)",
  rotations: "2 поворота, первым – нижний",
  keys: { x: 40, p: 20, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZG_S0,
      kind: "start",
      title: "Было: 60 → влево на 20 → вправо на 40",
      text: "Узлы идут «змейкой»: сначала шаг влево, по",
      ms: RESULT_MS,
    },
    {
      ...ZG_S0,
      kind: "aim",
      title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
      text: "Дед (60) стоит на месте и в этом повороте",
      focus: ["p", "x"],
      ms: AIM_MS,
    },
  ],
}
```

```

-----
const ROLE_RU: Partial<Record<NodeId, string>> = {
  x: "x — ищем его",
  p: "p — родитель",
  g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
  const { pos, edges, focus, moved } = frame;
  const dim = (id: NodeId) => (focus ? !focus.includes(
  const ghosts = frame.kind === "result" ? (moved ?? [])

  return (
    <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
      <defs>
        <marker id="splay-arrow" viewBox="0 0 10 10" re
          <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
        </marker>
      </defs>

    {/* призраки: где узел стоял до поворота */}
    {ghosts.map((id) => {
      const from = prev[id]!;
      const to = pos[id]!;
      const r = isSubtree(id) ? 15 : 20;
      const dx = to[0] - from[0];
      const dy = to[1] - from[1];
      const len = Math.hypot(dx, dy) || 1;
      const pad = r + 8;
      return (
        <g key={`ghost-${id}`} opacity={0.55}>
          <circle cx={from[0]} cy={from[1]} r={r} fil
          {len > pad * 2 + 6 && (
            <line
              x1={from[0] + (dx / len) * pad}
              y1={from[1] + (dy / len) * pad}
              x2={to[0] - (dx / len) * pad}

```

```

    return (
      <g
        key={id}
        style={{
          transform: `translate(${p[0]}px, ${p[1]}px)`,
          transition: MOVE,
          opacity: dim(id) ? 0.28 : 1,
        }}
      >
        {hot && <circle r={r + 7} fill="none" stroke="black" />}
        <circle r={r} fill={c.fill} stroke={c.stroke} />
        <text
          textAnchor="middle"
          dominantBaseline="central"
          fontSize={sub ? 12 : 15}
          fontWeight="700"
          fill={sub ? "#94a3b8" : "#fff"}
        >
          {sub ? id : keys[id]}
        </text>
        {!sub && (
          <text textAnchor="middle" y={-r - 7} font-size={12} font-weight="bold">
            {id}
          </text>
        )}
      </g>
    );
  }
}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const liveCase = vizString(vars?.case);
  const liveFrame = vizNumber(vars?.i);
  const [caseIdx, setCaseIdx] = useState(0);

```

```

        key={c.key}
        onClick={() => setCaseIdx(i)}
        className={`px-3 sm:px-4 py-2 rounded-lg text-
          i === caseIdx ? "bg-indigo-600 text-white
        }`}
      >
        {c.label}
      </button>
    )})
  </div>

  <div className="mb-3 text-xs sm:text-[13px] text-
    <span className="font-bold text-indigo-300">Кор
    <span className="text-slate-500">({active.rotat
  </div>

  {/* шаг + пояснение стоят НАД картинкой: сначала
  <div className="mb-3 rounded-xl border border-sla
    <div className="flex items-center gap-2 mb-1 fl
      <span
        className={`text-[10px] font-bold uppercase
          current.kind === "aim"
            ? "bg-amber-500/20 text-amber-300"
            : current.kind === "start"
              ? "bg-slate-700 text-slate-300"
              : "bg-indigo-500/20 text-indigo-300"
        }`}
      >
        {current.kind === "aim" ? "прицел" : curren
      </span>
      <span className="text-sm font-bold text-white
        Шаг {idx + 1} из {total}. {current.title}
      </span>
    </div>
    <p className="text-[13px] text-slate-400 leadin
  </div>

  <div className="bg-slate-900 rounded-xl border bo

```

```

        setPlaying(false);
        setFrame((f) => (f - 1 + total) % total);
    }}
    disabled={idx === 0}
    className="px-3 py-2 rounded-lg bg-slate-800 l
        d: hover:text-slate-300 text-xs font-bold tr
>
    ← Назад
</button>
<button
    onClick={() => {
        setPlaying(false);
        setFrame((f) => (f + 1) % total);
    }}
    className="px-4 py-2 rounded-lg bg-indigo-600
        w-indigo-900/40"
>
    {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
    onClick={() => setPlaying((p) => !p)}
    className="flex items-center gap-1.5 px-3 py-1
        ext-xs font-bold transition-colors"
>
    {playing ? <Pause className="w-3.5 h-3.5" />
    {playing ? "Стоп" : "Авто"}}
</button>
<button
    onClick={() => setSlow((s) => !s)}
    title="Автопрокрутка ещё медленнее"
    className={`flex items-center gap-1.5 px-3 py-1
        slow
        ? "bg-emerald-600/20 border-emerald-500 tr
        : "bg-slate-800 border-slate-700 text-sla
    }`}
>
    <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1
</button>

```

```

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот.
 *   aim – прицел на тройку g–p–x, назван случай;
 *   cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 *   rot – поворот: зелёным подсвечены рёбра, которых не было.
 * Рядом с логом всегда висит врезка «Механика одного поворота».
 * На каждом кадре – живая автопроверка in-order (сортировка).
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents).
 */

```

```

export interface SN {
  key: number;
  left: SN | null;
  right: SN | null;
  parent: SN | null;
}

```

```

export function cloneWithParents(t: SN | null): SN | null {
  if (!t) return null;
  const n: SN = { key: t.key, left: null, right: null, parent: null };
  const l = t.left ? cloneWithParents(t.left) : null;
  const r = t.right ? cloneWithParents(t.right) : null;
  if (l) {
    n.left = l;
    l.parent = n;
  }
  if (r) {
    n.right = r;
    r.parent = n;
  }
  return n;
}

```

```

export function bstInsert(root: SN | null, key: number): SN | null {
  if (!root) return { key, left: null, right: null, parent: null };

```



```

    else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
    const es: string[] = [];
    const walk = (n: SN | null) => {
        if (!n) return;
        if (n.left) es.push(`${n.key}→${n.left.key}`);
        if (n.right) es.push(`${n.key}→${n.right.key}`);
        walk(n.left);
        walk(n.right);
    };
    walk(t);
    return es.sort();
}

const diffEdges = (before: string[], after: string[]):
    const rm = before.filter((e) => !after.includes(e));
    const ad = after.filter((e) => !before.includes(e));
    const parts: string[] = [];
    for (const a of ad) {
        const [p, c] = a.split("→");
        if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
        else parts.push(`поддерево ${c} переехало к ${p}`);
    }
    return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
    kind: SplayStepKind;
    root: SN | null;
    aimEdge?: [number, number];
    /** Кадр cut: ребро, по которому отстёгивается среднее */
    cutEdge?: [number, number];
    cutLabel?: string;
    /** Куда β переедет: сторона у верхнего узла. */

```

```

        ? `Вставили ${key} обычным BST-спуском. Теперь по
        : found
        ? `Спуск по ключу ${key}: путь ${pathKeys.join(
        : `Ключа ${key} нет. Спуск уперся в ${last.key}
    rotCount: 0,
  });

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
  while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string,
  steps.push({
    kind: "aim",
    root: cloneWithParents(root)!,
    aimEdge,
    caseLabel: label,
    note: aimNote,
    rotCount,
  });
const p = child.parent!;
const mid = p.left === child ? child.right : child.left;
steps.push({
  kind: "cut",
  root: cloneWithParents(root)!,
  cutEdge: [child.key, mid ? mid.key : child.key],
  cutLabel: mid ? `β = поддереву ${mid.key}` : "β пусто",
  cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : "",
  caseLabel: label,
  note: mid
    ? `Отстёгиваем среднее поддерево β = ${mid.key}
      й у них общий – после поворота переедет ${p.left ? p.left.key : p.right.key}
      : `Среднего поддерева нет (β пусто): ${child.key}
  rotCount,
});

```

```

        kind: "aim",
        root: cloneWithParents(root)!,
        aimEdge: [target.parent.key, target.key],
        caseLabel: "Zig-Zig",
        note: `Теперь обычный зиг: ребро ${target.parent.key} над ${target.key}`,
        rotCount,
    ));
    emitRotation(
        target,
        "Zig-Zig",
        [target.parent.key, target.key],
        `Доводим: ${target.key} над ${target.parent.key}`,
    );
} else {
    const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}`;
    emitRotation(
        target,
        "Zig-Zag",
        [p.key, target.key],
        `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: ${p.key} над ${target.key}`,
    );
    steps.push({
        kind: "aim",
        root: cloneWithParents(root)!,
        aimEdge: [target.parent.key, target.key],
        caseLabel: "Zig-Zag",
        note: `Второе ребро: ${target.parent.key}—${target.key}`,
        rotCount,
    ));
    emitRotation(
        target,
        "Zig-Zag",
        [target.parent.key, target.key],
        `Доводим: ${target.key} над ${target.parent.key}`,
    );
}
}
}

```

```

{ line: 6, text: "          rotate(p over g)          # B
{ line: 7, text: "          rotate(x over p)" },
{ line: 8, text: "      else:                          # Z
{ line: 9, text: "          rotate(x over p)          # H
{ line: 10, text: "          rotate(x over g)" },
];

```

/* ----- врезка «механика одного по

```

const MechanismInset: React.FC<{ step: SplayStep | null
const isCut = step?.kind === "cut";
const isRot = step?.kind === "rot";
return (
  <div className="bg-slate-950/80 rounded-lg border b
    <h4 className="text-xs font-bold text-slate-300 m
    <p className="text-[10px] text-slate-500 mb-2">
      Как вправление вывиха: одна кость ходит вокруг
    </p>
    <svg viewBox="0 0 340 190" className="w-full">
      <g transform="translate(10,6)">
        <text y="10" fill="#94a3b8" fontSize="11" fon
        <line x1="55" y1="46" x2="30" y2="86" stroke=
        <line x1="55" y1="46" x2="80" y2="86" stroke=
        <line x1="30" y1="86" x2="12" y2="126" stroke=
        <line
          x1="30" y1="86" x2="48" y2="126"
          stroke={isCut ? "#f59e0b" : "#64748b"}
          strokeWidth={isCut ? 3 : 2}
          strokeDasharray={isCut ? "5,3" : "none"}
        />
        <circle cx="55" cy="46" r="15" fill="#1e293b"
        <text x="55" y="50" textAnchor="middle" fill=
        <circle cx="30" cy="86" r="15" fill="#78350f"
        <text x="30" y="90" textAnchor="middle" fill=
        <circle cx="12" cy="126" r="13" fill="#0f172a"
        <text x="12" y="130" textAnchor="middle" fill=
        <circle
          cx="48" cy="126" r="13"

```

```

    <text x="95" y="130" textAnchor="middle" fill="black">
    <text x="112" y="92" fill={isRot ? "#10b981" : "#64748b"}>
      {isRot ? "↓ β теперь при p" : "β при p"}
    </text>
    <text x="30" y="160" fill="#64748b" fontSize=14>
  </g>
</svg>
<p className="text-[10px] text-slate-500 leading-5">
  Обход слева-направо α x β p γ не меняется никогда,
  числа поворотов. Единственный, кто меняет хозяина — это β.
</p>
</div>
);
};

```

/* ----- рендер -----

```

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
  const pos = new Map<number, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: SN | null, d: number) => {
    if (!n) return;
    walk(n.left, d + 1);
    pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
    slot += 1;
    walk(n.right, d + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const SplayWalkViz: React.FC = () => {
  const initial = useMemo(buildInitial, []);
  const [steps, setSteps] = useState<SplayStep[] | null>();

```

```

        return;
    }
    timer.current = setTimeout(() => setIdx((v) => v +
    return () => {
        if (timer.current) clearTimeout(timer.current);
    });
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length) - 1] : null;
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot]);
const nodes = useMemo(() => [...pos.entries()].map(([key, node]) => node), [pos]);
const edges = useMemo(() => {
    const es: { a: number; b: number }[] = [];
    const walk = (n: SN | null) => {
        if (!n) return;
        if (n.left) es.push({ a: n.key, b: n.left.key });
        if (n.right) es.push({ a: n.key, b: n.right.key });
        walk(n.left);
        walk(n.right);
    };
    walk(viewRoot);
    return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.w));
const autoH = Math.max(300, ...nodes.map((n) => n.py));

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1].root) : 0), [steps]);
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1].root) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1].root) : 0), [steps]);
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
    if (!steps || !step || idx <= 0) return null;
    return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);

```

```

    return "#64748b";
  };

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border"
      {/* Wanka */}
      <div className="flex flex-wrap items-center justify-center"
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-indigo-600 text-white"
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">
              <p className="text-sm text-indigo-300">Клик
            </div>
          </div>
          <div className="flex flex-wrap items-center gap-3">
            <input
              value={input}
              onChange={(e) => setInput(e.target.value)}
              onKeyDown={(e) => {
                if (e.key === "Enter") {
                  const v = parseInt(input, 10);
                  if (!Number.isNaN(v)) {
                    const has = inorderKeys(baseRef.current);
                    if (has) runKey(v);
                    else doInsert();
                    setInput("");
                  }
                }
              }}
              placeholder="ключ + Enter"
              className="w-28 bg-slate-950 border border-indigo-500"
            />
            <button
              onClick={() => {
                baseRef.current = buildInitial();
              }}
            />
          </div>
        </div>
      </div>
    );
  );
}

```

```

<button onClick={() => setPlaying(!playing)}
  er-600" : "bg-emerald-600"}`}>
  {playing ? "  Пауза" : "▶ Пуск"}
</button>
<button onClick={() => { setPlaying(false); s
  lassName="px-3 py-1.5 rounded-lg text-xs fo
  War →
</button>
<span className="text-xs text-slate-400">
  War {idx + 1} из {steps.length} • поворотов
</span>
{step?.caseLabel && (
  <span className="px-2.5 py-1 rounded-full t
  00/60">
    {step.caseLabel}
  </span>
)}
</div>
)}

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
  {!steps && (
    <div className="absolute top-3 left-3 bg-indi
      ont-bold">
        Кликни по любому узлу – splay покадрово
      </div>
    )}
  <svg viewBox={`0 0 ${autoW} ${autoH}`} className
    {edges.map((e) => {
      const a = pos.get(e.a);
      const b = pos.get(e.b);
      if (!a || !b) return null;
      const isAim = step?.aimEdge && step.aimEdge
      const isCutEdge =
        step?.kind === "cut" && step.cutEdge && s
        .cutEdge[1];
      const isMoved = !isAim && !isCutEdge && pre

```



```

        />
        <text y={4} fill="#ffffff" fontSize="13
    >
        {n.key}
    </text>
  </g>
);
}}
</svg>

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-rot
  <div className="flex-1 bg-slate-950/80 p-4 rot
    <p className="font-semibold text-amber-400
      {step?.kind === "aim"
        ? "    Прицел:"
        : step?.kind === "cut"
        ? "∞    Отстёгиваем среднее поддерево:"
        : step?.kind === "rot"
        ? "    Поворот:"
        : step?.kind === "search"
        ? "    Спуск:"
        : "Статус:"}
    </p>
    <p className="min-h-[40px] flex items-cente
      {step ? step.note : "Клигни по узлу или в
ке справа – механика одного поворота."}
    </p>
    {steps && step && step.kind !== "search" &&
      <p className={`text-xs mt-2 ${checkOk ? "
        {checkOk ? " " : " "} проверка кадра: o
        {inorderKeys(viewRoot).length}
        {movedCount > 0 ? ` · переехало рёбер:
      </p>
    }}
  </div>
  <div className="w-full lg:w-[300px] shrink-0"
    <MechanismInset step={step} />

```

```

        const za1 = step?.caseLabel === "Zig-Zag"
        const za2 = step?.caseLabel === "Zig-Zag"
        const active = zig || zz1 || zz2 || za1 ||
        return (
          <div key={l.line} className={`py-1 px-2
ber-400" : "text-slate-400"}`>
            <span className="text-slate-600 select
            {l.text}
          </div>
        );
      }
    }
  </div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 70/96: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSte

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };

  setDirtyStack(prev => [...prev, newPlate]);
  addLog(` PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.color}`);

  setTimeout(() => {
    setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
  }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    return;
  }

  const topPlate = dirtyStack[dirtyStack.length - 1];
  addLog(` POP: Помыли ${topPlate.emoji} с ${topPlate.color}`);
  setDirtyStack(prev => prev.slice(0, -1));
}, [dirtyStack, isWashing]);

```

```

        mState: 'in' },
    });
    setCleanRack([]);
    setIsWashing(false);
    setShowSoap(false);
    setLog([' Стол сброшен. Посуда снова грязная!']);
  });

  const topIndex = displayStack.length - 1;

  return (
    <div className="flex flex-col gap-6">
      { /* МНЕМОНИКА / ПРАВИЛО */ }
      <div className="bg-blue-950/40 border border-blue-950/40">
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-blue-400 text-2xl">
            ПРАВИЛО
          </h3>
          <p className="text-xs text-slate-300 mt-1 leading-tight">
            Ты складываешь грязные тарелки друг на друга.
            А когда приходит время мыть, ты берёшь именно
            последнюю. Попытаешься вытащить нижнюю – всё рухнет!
          </p>
        </div>
      </div>

      { /* УПРАВЛЕНИЕ */ }
      <div className="flex items-center gap-3 flex-wrap">
        <button
          onClick={pushPlate}
          disabled={isWashing}
          className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
            disabled:cursor-not-allowed text-white font-medium text-sm py-2.5
            border border-blue-950/40 transition-all cursor-pointer flex items-center justify-center">
          <span> Положить грязную (PUSH)</span>
        </button>

        <button

```

```

        <div className="absolute w-6 h-6 rounded-1
"></div>
        <div className="absolute w-4 h-4 rounded-1
"></div>
        <div className="absolute w-5 h-5 rounded-1
"></div>
        <div className="absolute text-4xl anim-cr
</div>
    }}

{displayStack.length === 0 ? (
    <div className="flex-1 flex flex-col items-
        <span className="text-6xl mb-3 animate-pu
        <p className="text-white font-black text-
        <p className="text-slate-500 text-xs mt-1
    </div>
) : (
    <div className={`flex-1 flex flex-col justifi

        {/* Указатель TOP */}
        <div
            className="absolute right-8 flex items-
            style={{ bottom: `${24 + topIndex * 44}}
        >
            <span className="text-xs font-mono text
er gap-1">
                <ArrowUp className="w-3.5 h-3.5" />
                <span>ВЕРШИНА (TOP)</span>
            </span>
            <span className="text-blue-400 animate-p
        </div>

        {/* Тарелки (отрисовываем в обратном поряд
        {displayStack.slice().reverse().map(({plate
            const idx = displayStack.length - 1 - r
            const isTop = idx === topIndex;
            return (
                <div

```

```

<div className="flex-1 flex flex-col justify-
  {cleanRack.length === 0 ? {
    <div className="flex-1 flex flex-col item
      <span className="text-5xl mb-2"> </span>
      <p className="text-xs font-bold font-mo
      <p className="text-[10px] mt-1">Здесь 6
    </div>
  ) : {
    <div className="flex flex-col gap-2 w-full
      {cleanRack.map(plate => {
        <div
          key={plate.id}
          className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-
        >
          <span className="text-sm"> {plate.i
          <span className="text-[10px] text-e
        </div>
      }}}
    </div>
  })
</div>

{/* Столешница сушилки */}
<div className="w-full h-4 bg-gradient-to-r f
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => {
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-blue-500

```

```

        if (s[i] === s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
                ска увеличивается.` });
            j++;
        } else {
            steps.push({ i, j, pi: [...pi], highlight:
                овпадений нет.` });
        }

        pi[i] = j;
        steps.push({ i, j, pi: [...pi], desc: `Записывае

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Вни
                !` });
            z[i] = Math.min(r - i + 1, z[i - l]);
            steps.push({ i, l, r, z: [...z], desc: `Пре

```

```

const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPa
  if (mode === "kmp") return generateKmpSteps(safeP
  else return generateZSteps(safePattern, safeText
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const runtimePattern = vizString(runtime?.variables
const s = runtimeString ?? demoString;
// Длина образца: приоритет – переменная pattern из
// разделителя "#" в склейке pattern#text; иначе вс
const patternLength = runtimePattern
  ? runtimePattern.length
  : runtimeString
    ? (runtimeString.includes("#") ? runtimeString
    : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    }, 1200);
  }

```



```

        </button>
        <button onClick={() => setMode("z")}
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

      <div className="flex items-center gap-2"
        <div className="bg-slate-800 p-1 fl
          <span className="text-[10px] te
          <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
        </div>
        <div className="bg-slate-800 p-1 fl
          <span className="text-[10px] te
          <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
        </div>
      </div>
    </div>

    <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
      <div className="flex gap-1.5 px-4 h-ful
        {s.split("").map((char, index) => {
          const isPattern = index < patte
          const isSeparator = index === p

          let borderColor = "border-slate
          let bgColor = "bg-slate-900";
          let textColor = isSeparator ? "

          // KMP Logic
          if (mode === "kmp") {
            if (step.highlight?.include
              borderColor = "border-a
              bgColor = step.match ==

```

```

    slate-700/50 font-mono font-bold">
        {{mode == "kmp" ?
    </div>

    {/* KMP fingers */}
    {mode == "kmp" && step
        <div className="abs

z-20">
            <span className=
        </div>
    }}
    {mode == "kmp" && step
        <div className="abs
            <span className=
        </div>
    }}
    {mode == "kmp" && step
        <div className="abs

20">
            <span className=
        </div>
    }}

    {/* Z fingers */}
    {mode == "z" && step.i
        <div className="abs

e z-20">
            <span className=
        </div>
    }}
    {mode == "z" && step.z
        <div className="abs
            <span className=
        </div>
    }}
    {mode == "z" && step.z
        <div className="abs
            <span className=

```



```

-----
* Появляется по наведению курсора и по фокусу с клавиатуры
*/
export const Tooltip: React.FC<TooltipProps> = ({ content }) => {
  const [open, setOpen] = useState(false);

  const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

  return (
    <span
      className={`relative inline-flex ${className ?? ""}
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
        onFocus={() => setOpen(true)}
        onBlur={() => setOpen(false)}
      >
      {children}
      <span
        role="tooltip"
        className={`pointer-events-none absolute left-1/2 top-1/2
          bg-slate-900 px-3 py-2 text-left text-[11px]
          transition-all duration-150 ${
            side === "top" ? "bottom-full mb-2" : "top-full mt-2"
          } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
        >
        {content}
        <span
          className={`absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
            ${side === "top" ? "-bottom-[5px] border-b border-slate-800" : ""}
          >
        </span>
      </span>
    >
  );
};

```

```
    { u: 3, v: 4 },
  ];
```

```
interface TStep {
  desc: string;
  visited: Set<number>;
  fullyProcessed: Set<number>;
  currentNode: number | null;
  topoList: number[];
  dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  useVizStepSync(currentStepIdx, setCurrentStepIdx, steps, (vars) => {
    const v = vizNumber(vars.v);
    const orderLength = vizArray(vars.order)?.length;
    if (v === null && orderLength === undefined) return null;
    const candidates = steps
      .map((step, index) => ({ step, index }))
      .filter(({ step }) => (v === null || step.currentNode !== v));
    return candidates.at(-1)?.index ?? null;
  });
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: fullyProcessed,
        currentNode,
        topoList,
        dfsStack
      });
    };
  });
}
```

```

        topoList.push(u); // prepend behavior: we write it to the beginning
        recordStep(
            `DFS: закончили обработку '${dagNodes[u].name}'
            ,
            u,
        );
    };
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
    if (!visited.has(i)) {
        recordStep(
            `Запускаем новый обход DFS от нетронутой вершины
            null,
        );
        dfs(i);
    }
}

recordStep(
    "Топологическая сортировка завершена! Полученный список:
    null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
    desc: "Запуск...",
    visited: new Set(),
    fullyProcessed: new Set(),
    currentNode: null,
    topoList: [],
    dfsStack: [],
};

useEffect(() => {
    let timer: NodeJS.Timeout;

```

```

        <Pause className="w-4 h-4 ml-1" />
      ) : {
        <Play className="w-4 h-4 ml-1" />
      }}
      {isPlaying ? "Пауза " : "Старт "}
    </button>

    <button
      onClick={reset}
      className="flex items-center justify-center
      ors"
    >
      <RotateCcw className="w-4 h-4" />
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
  { /* Playfield SVG */}
  <div className="lg:col-span-8 bg-[#0B1120] relative" >
    <svg className="w-full h-full max-w-[650px]" >
      <defs>
        <marker
          id="dag-arrow"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"
          markerWidth="6"
          markerHeight="6"
          orient="auto-start-reverse"
        >
          <path d="M 0 1 L 10 5 L 0 9 z" fill="#4" />
        </marker>
        <marker
          id="dag-arrow-active"
          viewBox="0 0 10 10"
          refX="24"
          refY="5"

```

```

let fill = "#1e293b";
let stroke = "#334155";
let textColor = "text-slate-400";
void textColor;

if (isCurrent) {
  fill = "#312e81";
  stroke = "#818cf8";
  textColor = "text-white font-bold";
} else if (isProcessed) {
  fill = "#022c22";
  stroke = "#059669";
  textColor = "text-emerald-300";
} else if (isVisited) {
  fill = "#3730a3";
  stroke = "#6366f1";
  textColor = "text-indigo-200";
}

return (
  <g key={node.id}>
    <rect
      x={node.x - 55}
      y={node.y - 22}
      width="110"
      height="44"
      rx="8"
      fill={fill}
      stroke={stroke}
      strokeWidth="2"
      className="transition-all duration-!
    />
    <text
      x={node.x}
      y={node.y - 4}
      textAnchor="middle"
      fill="white"
      fontSize="11px"

```


РЕЗУЛЬТАТ (МАТРУТ ОБУЧЕНИЯ):

```

</span>
<div className="bg-slate-900/60 p-2.5 rounded
pr-1 text-slate-300">
  {step.topoList.length === 0 ? (
    <span className="text-slate-600 text-
      ожидаем обработку узлов...
    </span>
  ) : (
    getTopoListString()
  )}
</div>
</div>

{/* Color key */}
<div className="space-y-1">
  <span className="text-[10px] text-slate-500">
    ВЕРШИНЫ В DFS:
  </span>
  <div className="flex items-center gap-2">
    <span className="w-3.5 h-3.5 rounded bg-
    <span className="text-slate-400 text-[10px]">
      Серые (зашли)
    </span>
  </div>
  <div className="flex items-center gap-2">
    <span className="w-3.5 h-3.5 rounded bg-
    <span className="text-slate-400 text-[10px]">
      Черные (готово)
    </span>
  </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-
  <span>
    {currentStepIdx + 1} из {steps.length}
  </span>

```

```

*          чек-лист инвариантов (автопроверка, как у
*  split   – разрез по x с двумя корзинами (грамотная
*  merge   – склейка двух деревьев по y.
*  erase   – удаление ПРОИЗВОЛЬНОЙ точки: найти по x,
*  layers  – миф «дети в массиве по  $2k/2k+1$ »: работает
*  search  – «отсортирую бинарным поиском»: поиск не с
*          сортировка платит  $\geq n \cdot \log n$  сравнений, со
*
* Канонический набор: все x и все y различны  $\Rightarrow$  дерево
* Узел идентифицируется парой  $id = "x:y"$ .
*/

```

```

export interface Pair {
  key: number;
  grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
  id: NodeId;
  key: number;
  grade: number;
  left: TNode | null;
  right: TNode | null;
}

export const POINTS: Pair[] = [
  { key: 45, grade: 8 },
  { key: 3, grade: 6 },
  { key: 6, grade: 5 },
  { key: 2, grade: 4 },
  { key: 9, grade: 3 },
  { key: 7, grade: 2 },
  { key: 12, grade: 0 },
  { key: 1, grade: -4 },
];

```

```

const trace: SplitResult["trace"] = [];
const go = (t: TNode | null): { l: TNode | null; r: TNode | null } => {
  if (!t) return { l: null, r: null };
  if (t.key <= x) {
    const sub = go(t.right);
    trace.push({ atId: t.id, goLeft: true, note: `(${t.key} <= ${x})  
о правое поддереву.` });
    t.right = sub.l;
    return { l: t, r: sub.r };
  }
  const sub = go(t.left);
  trace.push({ atId: t.id, goLeft: false, note: `(${t.key} > ${x})  
о левое поддереву.` });
  t.left = sub.r;
  return { l: sub.l, r: t };
};
const res = go(root);
return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
  root: TNode | null;
  trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; }[];
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
  const trace: MergeResult["trace"] = [];
  const go = (a: TNode | null, b: TNode | null): TNode | null => {
    if (!a || !b) {
      trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: `один из узлов целиком.` });
      return a ?? b;
    }
    if (a.grade > b.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `выбран узел ${a.id} с ${a.key} и ${a.grade},  
в него вставлено правое поддереву с (${b.key}, ${b.grade})` });
      a.right = go(a.right, b);
      return a;
    }
    if (b.grade > a.grade) {
      trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `выбран узел ${b.id} с ${b.key} и ${b.grade},  
в него вставлено левое поддереву с (${a.key}, ${a.grade})` });
      b.left = go(a, b.left);
      return b;
    }
    if (a.key < b.key) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `выбран узел ${a.id} с ${a.key} и ${a.grade},  
в него вставлено левое поддереву с (${b.key}, ${b.grade})` });
      a.left = go(a.left, b);
      return a;
    }
    if (b.key < a.key) {
      trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `выбран узел ${b.id} с ${b.key} и ${b.grade},  
в него вставлено левое поддереву с (${a.key}, ${a.grade})` });
      b.left = go(a, b.left);
      return b;
    }
    if (a.key > b.key) {
      trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `выбран узел ${b.id} с ${b.key} и ${b.grade},  
в него вставлено левое поддереву с (${a.key}, ${a.grade})` });
      b.left = go(a, b.left);
      return b;
    }
    if (b.key > a.key) {
      trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `выбран узел ${a.id} с ${a.key} и ${a.grade},  
в него вставлено левое поддереву с (${b.key}, ${b.grade})` });
      a.left = go(a.left, b);
      return a;
    }
  };
  const res = go(a, b);
  return { root: res, trace };
}

```

```

let heap = true;
let prev: number | null = null;
const walk = (n: TNode | null, min: number, max: number) => {
  if (!n) return;
  if (n.key <= min || n.key > max) bst = false;
  if (parentGrade !== null && n.grade > parentGrade) bst = false;
  walk(n.left, min, n.key, n.grade);
  if (prev !== null && n.key < prev) bst = false;
  prev = n.key;
  walk(n.right, n.key, max, n.grade);
};
walk(root, -Infinity, Infinity, null);
return { bst, heap };
}

```

/* ----- раскладка сцены дерева ----- */

```

const layoutTree = (root: TNode | null) => {
  const pos = new Map<NodeId, { px: number; py: number }>();
  let slot = 0;
  const walk = (n: TNode | null, depth: number) => {
    if (!n) return;
    walk(n.left, depth + 1);
    pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 92 });
    slot += 1;
    walk(n.right, depth + 1);
  };
  walk(root, 0);
  return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
  const es: { from: NodeId; to: NodeId }[] = [];
  const walk = (n: TNode | null) => {
    if (!n) return;
    if (n.left) { es.push({ from: n.id, to: n.left.id }); }
    if (n.right) { es.push({ from: n.id, to: n.right.id }); }
  };
  walk(root);
  return es;
};

```

```

        strokeDasharray={edgeDash && edgeDash(e.f
        className="transition-all duration-500"
      />
    );
  }}}
  {nodes.map((n) => {
    const p = pairParts(n.id);
    const pulse = pulseId === n.id;
    return (
      <g key={`tn-${n.id}`} transform={`translate
        <circle r={pulse ? 24 : 20} fill={nodeColo
        } strokeWidth={pulse ? 4 : 2} className="tr
        <text y={-2} fill="#fff" fontSize="13" for
        <text y={13} fill="#cbd5e1" fontSize="10"
      </g>
    );
  }}}
</svg>
{subtitle && <p className="text-xs text-slate-400
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
  const [a, b] = id.split(":").map(Number);
  return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
  points: Pair[];
  drawnEdges: { a: Pair; b: Pair }[];

```

```

    const b = byPair(e.b);
    const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.a.grade === e.b.grade) || (badEdge.a.key === e.b.key && badEdge.a.grade === e.a.grade));
    return <line key={`pe-${i}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y} strokeDasharray={isBad ? "4,3" : "none"} />
  )}

  {/* временная линия игры */}
  {interactive && selected && cursor && (
    <line
      x1={PX(selected.key)} y1={PY(selected.grade)}
      stroke="#818cf8" strokeWidth={2.5} strokeDasharray={isBad ? "4,3" : "none"} />
  )}

  {points.map((p) => {
    const done = drawnEdges.some((e) => (e.a.key === p.key && e.b.key === p.key));
    const isSel = selected && selected.key === p.key;
    return (
      <g
        key={`pp-${p.key}:${p.grade}`}
        className="transition-all duration-500"
        onPointerDown={interactive && onPointDown ? (e) => onPointDown(e, p) : null}
        onPointerUp={interactive && onPointUp ? () => onPointUp(p) : null}
        style={interactive ? { cursor: "pointer" } : null} />
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={10} stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"} strokeWidth={2} />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-size={12} font-weight="bold">{p.key}</text>
    </g>
  )};
  )}
}

{pending && (
  <g className="transition-all duration-500">
    <circle cx={PX(pending.key)} cy={PY(pending.grade)} r={10} stroke="#ccc" strokeWidth={2} />
    <text x={PX(pending.key)} y={PY(pending.grade) + 3.5} font-size={12} font-weight="bold">{pending.key}</text>
  </g>
)}

```



```

    }
    const xs = pairs.map((p) => p.key);
    const ys = pairs.map((p) => p.grade);
    return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
  a: Pair;
  b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
  edges: GameEdge[],
  x: Pair,
  y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
  if (x.key === y.key && x.grade === y.grade) return { ok: true };
  for (const e of edges) {
    if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
        (pid(e.a) === pid(y) && pid(e.b) === pid(x)))
      return { ok: false, reason: "такая линия уже проведена" };
  }
  if (x.grade === y.grade) return { ok: false, reason: "одинаковые уровни" };
  const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
  if (edges.some((e) => pid(e.b) === pid(child)))
    return { ok: false, reason: `у точки ${child.key} уже есть ребёнок` };
  // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)
  const childComp = new Set<string>([pid(child)]);
  let grew = true;
  while (grew) {
    grew = false;
    for (const e of edges) {

```



```

}
const comps = new Set(points.map((p) => find(pid(p)))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
    const seen = new Set<string>();
    const inorder: number[] = [];
    const walk = (p: Pair): void => {
        if (seen.has(pid(p))) {
            bstOk = false; // цикл
            return;
        }
        seen.add(pid(p));
        const kids = children.get(pid(p)) ?? [];
        if (kids.length > 2) {
            bstOk = false;
            return;
        }
        const sorted = [...kids].sort((u, v) => u.key - v
        let left: Pair | null = null;
        let right: Pair | null = null;
        if (sorted.length === 2) {
            if (sorted[0].key === sorted[1].key || sorted[0
                bstOk = false; // двое детей с равным x или с
                return;
            }
            left = sorted[0];
            right = sorted[1];
        } else if (sorted.length === 1) {
            // одиночный ребёнок: сторона важна – определяет
            if (sorted[0].key <= p.key) left = sorted[0];
            else right = sorted[0];
        }
    }
}

```

```

    if (right[0]) node.right = build(right[0]);
    return node;
  };
  return build(rootPair);
}

const BuildMode: React.FC = () => {
  const [raw, setRaw] = useState(DEMO_RAW);
  const [phase, setPhase] = useState<BuildPhase>("sort");
  const [algo, setAlgo] = useState<SortAlgo>("none");
  const [order, setOrder] = useState<ConnectOrder>("y-d");
  const [sortIdx, setSortIdx] = useState(0);
  const [connIdx, setConnIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [sortCost, setSortCost] = useState<string | null>();
  const live = useTreapLive();

  const parsed = useMemo(() => parsePairs(raw), [raw]);
  const userPairs = parsed.errors.length ? [] : parsed.pairs;

  const n = userPairs.length;

  const userTree = useMemo(() => (userPairs.length >= 1 ? buildTree(userPairs) : null), [userPairs]);
  const canonEdges = useMemo(() => {
    const es: { a: Pair; b: Pair }[] = [];
    const walk = (node: TNode | null) => {
      if (!node) return;
      if (node.left) { es.push({ a: { key: node.key, group: node.group, left: true }, b: node.left }); }
      if (node.right) { es.push({ a: { key: node.key, group: node.group, right: true }, b: node.right }); }
    };
    walk(userTree);
    return es;
  }, [userTree]);
  const leftRightEdges = useMemo(() => [...canonEdges], [canonEdges]);

  useEffect(() => {

```

```
const [gameMsg, setGameMsg] = useState<string | null>
const [badEdge, setBadEdge] = useState<GameEdge | null>
const [hintUsed, setHintUsed] = useState(false);
```

```
const resetGame = () => {
  setGameEdges([]);
  setSel(null);
  setCursor(null);
  setGameMsg(null);
  setBadEdge(null);
  setHintUsed(false);
};
```

```
const startGame = () => {
  resetGame();
  setPhase("game");
  setPlaying(false);
};
```

```
const verdict = useMemo(() => checkGameState(userPair)
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree
```

```
const attemptConnect = (x: Pair, y: Pair) => {
  const r = canConnect(gameEdges, x, y);
  if (r.ok && r.edge) {
    setGameEdges((prev) => [...prev, r.edge!]);
    setGameMsg(r.reason ?? null);
    setBadEdge(null);
  } else {
    setGameMsg(r.reason ?? "так нельзя");
    setBadEdge({ a: x, b: y });
    window.setTimeout(() => setBadEdge(null), 900);
  }
};
```

```
const pointDown = (p: Pair) => {
  if (phase !== "game") return;
  if (!sel) {
    setSel(p);
  }
};
```

```

<h4 className="text-sm font-bold text-slate-300">
  <button onClick={() => { setRaw(DEMO_RAW); setPy(1); }}>
    демо-набор
  </button>
</div>
<textarea
  value={raw}
  onChange={(e) => { setRaw(e.target.value); setRows(2); }}
  className="w-full bg-slate-950 border border-slate-500"
  placeholder="3 5, 45 8, 1 -4 ..."
/>
{parsed.errors.length > 0 && (
  <div className="mt-2 text-xs text-rose-300 bg-slate-950 border border-slate-500 p-2">
    {parsed.errors.map((e, i) => <div key={i}>{e}</div>)}
  </div>
)}
{parsed.errors.length === 0 && (parsed.equalX || parsed.equalY ? (
  <div className="mt-2 text-xs text-amber-300 bg-slate-950 border border-slate-500 p-2">
    {parsed.equalY && <div>⚡ Есть <b>равные y</b> у тех, кто <b>раньше</b> в списке (сортировка стабильная) </div>
    {parsed.equalX && <div>⚡ Есть <b>равные x</b> у тех, у кого <b>раньше</b> де. key → left).</div>}
  </div>
)}
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4">
  <div className="flex flex-wrap items-center justify-center">
    <h4 className="text-base font-bold text-emerald-400">Координатная плоскость – НАВЕРХУ
    </h4>
    <div className="flex items-center gap-2">
      {phase === "sort" ? (
        <div>
          <span className="text-xs text-slate-400">Сортировка
          {(["quicksort", "counting"] as const).map((method) => (
            <button
              key={method}
              className="text-xs text-slate-400 border border-slate-500 rounded-sm px-2 py-1"
              onClick={() => { setPhase(method); }}
            >{method}
            </button>
          ))}
        </div>
      ) : (
        <div>
          <span className="text-xs text-slate-400">Фазы
          {(["quicksort", "counting"] as const).map((method) => (
            <button
              key={method}
              className="text-xs text-slate-400 border border-slate-500 rounded-sm px-2 py-1"
              onClick={() => { setPhase(method); }}
            >{method}
            </button>
          ))}
        </div>
      )}
    </div>
  </div>
</div>

```

```

    })
  </div>
</div>
<Plane
  points={userPairs}
  drawnEdges={phase === "game" ? gameEdges : ph
  pending={null}
  interactive={phase === "game"}
  selected={sel}
  cursor={cursor}
  badEdge={badEdge}
  onPointDown={pointDown}
  onPointUp={pointUp}
  onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
  {phase === "sort" ? (
    algo === "none" ? (
      <>Ввод приходит <b>как попало</b> ({n} пар
      ть</b>.</>
    ) : sortIdx < 3 ? (
      algo === "quicksort" ? (
        <>⚡ quicksort: счётчик сравнений: <b>cl
        {comparisons}</b> ( $n \cdot \log_2 n$ ).</>
      ) : (
        <> counting: точки в корзины по значени
        , потому что у – маленькие целые.</>
      )
    ) : (
      <>Отсортировано ({sortCost ?? "способ не
      <b>Игру </b></>
    )
  ) : phase === "game" ? (
    gameWon ? (
      <> Готово! {hintUsed ? "Решение подсказан
      ево: линии вниз по y, обход по x отсортиров
    ) : gameMsg ? (
      <>⚠ {gameMsg}</>
    )
  )

```

```

        ate-700 text-white disabled:opacity-30"> C
        <button onClick={showSolution} disabled={ga
        te disabled:opacity-30"> Показать решение<
    </div>
  })
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
  <div className="bg-emerald-950/20 rounded-xl p-
    <h4 className="text-sm font-bold text-emerald
    <TreeView root={gameTree} subtitle="Слева-нап
      ществует." />
  </div>
)}

{/* Дерево */}
{order === "y-desc" && userTree && (
  <div className="bg-slate-900/40 rounded-xl p-4
    <h4 className="text-sm font-bold text-slate-3
    <TreeView root={userTree} subtitle="Слева-нап
  </div>
)}
</div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SplitMode: React.FC = () => {
  const [x0, setX0] = useState(7);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [])
  const split = useMemo(() => splitTree(buildTree(POINT
  const steps = split.trace;
  const live = useTreapLive();
  // Компилятор: x – ключ разреза, i – номер шага спуска
  useEffect(() => { if (live.x !== null) setX0(live.x);

```

```

        {steps[idx]?.note}
      </div>

      {done && (
        <div className="grid grid-cols-1 md:grid-cols-2"
          <div className="bg-sky-950/20 rounded-xl p-3"
            <h5 className="text-sm font-bold text-sky-300"
              <TreeView root={lTree} width={480} height={480} />
            </div>
            <div className="bg-rose-950/20 rounded-xl p-3"
              <h5 className="text-sm font-bold text-rose-300"
                <TreeView root={rTree} width={480} height={480} />
              </div>
            </div>
          </div>
        )}

    <div className="text-xs text-slate-400 bg-slate-900 p-3"
      <LiveChip live={live} />{' '}
      <b>Как это работает на самом деле:</b> идём с
      деревом), отдаётся <b>вместе с веткой</b> – д
    </div>
  </div>
);
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
  const [stepIdx, setStepIdx] = useState(0);
  const live = useTreapLive();
  // Компилятор: i – номер шага слияния.
  useEffect(() => { if (live.i !== null) setStepIdx(live.i); }, [live.i]);
  const { l, r } = useMemo(() => splitTree(buildTree(P0, live.i)), [live.i]);
  const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
  const steps = merged.trace;
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;
  const cur = steps[idx];

```

```

        <div className="text-xs text-slate-400 bg-slate-900" style={bgStyle}>
          <b>Грамотно про «слияние»:</b> у корней сравнения
          в (то, что сохраняет порядок x) отправляется,
        </div>
      </div>
    );
  };
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
  const [key, setKey] = useState(9);
  const [stepIdx, setStepIdx] = useState(0);
  const fullTree = useMemo(() => buildTree(POINTS), [POINTS]);
  const res = useMemo(() => eraseNode(buildTree(POINTS), key), [key]);
  const steps = res.trace;
  const live = useTreapLive();
  // Компилятор: x — ключ удаляемой вершины, i — шаг слияния
  useEffect(() => { if (live.x !== null && POINTS.some((v) => v.x === live.x)) setKey(Math.min(stepIdx, steps.length - 1)); }, [live.x, stepIdx]);
  useEffect(() => { if (live.i !== null) setStepIdx(Math.min(stepIdx, steps.length - 1)); }, [live.i, stepIdx]);
  const idx = Math.min(stepIdx, steps.length - 1);
  const done = idx >= steps.length - 1;

  const pathIds = useMemo(() => {
    const ids: NodeId[] = [];
    for (const s of steps.slice(0, idx + 1)) if (s.atId !== null) ids.push(s.atId);
    return ids;
  }, [steps, idx]);
  const pathSet = new Set(pathIds);

  return (
    <div className="space-y-4">
      <div className="flex flex-wrap items-center gap-3">
        <span className="text-sm text-slate-300">Удалить</span>
        <LiveChip live={live} />
        <select value={key} onChange={(e) => { setKey(Number(e.target.value)); }}>
          {Array.from({ length: 10 }, (_, i) => i).map((i) => <option value={i}>{i}</option>)}
        </select>
      </div>
      <div className="text-sm text-slate-700 rounded-lg px-3 py-1.5 text-sm font-mono">
        {done ? "Готово" : "Шаг " + (idx + 1) + " из " + steps.length}
      </div>
    </div>
  );
};

```



```

    });
  };

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
  const [pick, setPick] = useState<"heap" | "treap">("heap");
  // Полное дерево из 7 узлов (куча): индексы 1..7
  const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
  const full = useMemo(() => buildTree(POINTS), []);
  // Честная слотовая разметка treap: индексы как в куче
  const slots = useMemo(() => {
    const arr: (string | null)[] = Array(16).fill(null);
    const walk = (n: TNode | null, i: number) => {
      if (!n || i >= 16) return;
      arr[i] = n.id;
      walk(n.left, 2 * i);
      walk(n.right, 2 * i + 1);
    };
    walk(full, 1);
    return arr;
  }, [full]);
  const nodeCount = slots.filter(Boolean).length;

  return (
    <div className="space-y-4">
      <div className="flex gap-2">
        <button onClick={() => setPick("heap")} className="text-
          ext-white" : "bg-slate-700 text-slate-300">heap</button>
        <button onClick={() => setPick("treap")} className="text-
          text-white" : "bg-slate-700 text-slate-300">treap</button>
      </div>

      {pick === "heap" ? (
        <div className="bg-slate-900/40 rounded-xl p-4">
          <h4 className="text-sm font-bold text-slate-300">
            4>
          <div className="flex gap-2 overflow-x-auto mb-4">

```



```

    });
  })}
</div>

<div className="text-xs text-slate-300 bg-slate-900" >
  {stage === 0 && <>Массив пар <b>не отсортирован</b>
    а беспорядок.</>}
  {stage === 1 && <> Бинарный поиск посмотрел на
    ю половину... <b>в которой лежала цель</b>. На
    реставил</b>: массив как был в беспорядке, та
  {stage === 2 && bidx < bsSteps.length - 1 && <>
  {stage === 2 && bidx >= bsSteps.length - 1 && <>
    сравнения. Но чтобы поиск вообще заработал,
    сравнения на <b>всех</b>. «Отсортирую бинарни
  </div>

  <div className="text-xs text-slate-400 bg-slate-900" >
    <b>Есть ли разница, чем сортировать?</b> Сравн
    ня граница, каждое сравнение даёт 1 бит. Но
    ля маленьких целых у (вкладка «Собрать», кноп
    задать порядок соединения.
  </div>
</div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
  const [edges, setEdges] = useState<GEdge[]>([]);
  const [sel, setSel] = useState<Pair | null>(null);
  const [hover, setHover] = useState<Pair | null>(null);
  const [msg, setMsg] = useState<string>("Кликни точку-1");

  const v = gameVerdict(edges, POINTS);

  const click = (p: Pair) => {

```

```

<defs>
  <pattern id="gm-grid" width="14" height="14">
    <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1" />
  </pattern>
</defs>
<rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1" />
<line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1" />
<line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(1)} stroke="black" stroke-width="1" />

{edges.map((e, i) => {
  <line
    key={`ge-${i}`}
    x1={PX(e.a.key)} y1={PY(e.a.grade)}
    x2={PX(e.b.key)} y2={PY(e.b.grade)}
    stroke={i === edges.length - 1 ? "#34d399" : "black"}
    strokeWidth={i === edges.length - 1 ? 3.5 : 1}
    className="transition-all duration-300"
  />
})}

{POINTS.map((p) => {
  const isSel = sel && sameP(sel, p);
  const isHover = hover && sameP(hover, p) && !isSel;
  return (
    <g key={`gp-${p.key}:${p.grade}`} onClick={p.onClick}
    className="cursor-pointer">
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={12} fill="white" stroke="black" stroke-width="1" />
      <circle cx={PX(p.key)} cy={PY(p.grade)} r={12} fill={isHover ? "#94a3b8" : "#a5b4fc"} stroke="black" stroke-width="1" />
      <text x={PX(p.key)} y={PY(p.grade) + 3.5} fill="black" font-size="12" font-weight="bold">{p.key}</text>
      <text x={PX(p.key)} y={PY(p.grade) + 24} fill="black" font-size="12" font-weight="bold">{p.grade}</text>
    </g>
  );
})}
</svg>

```

```

/** Значения из Python-компилятора: x — ключ, y — приоритет
function useTreapLive(): { x: number | null; y: number | null; i: number } {
  const runtime = useVizRuntime();
  const vars = runtime?.variables;
  const x = vizNumber(vars?.x);
  const y = vizNumber(vars?.y);
  const i = vizNumber(vars?.i);
  return { x, y, i, linked: x !== null || y !== null || i !== null };
}

const LiveChip: React.FC<{ live: { x: number | null; y: number | null; i: number }; linked?: boolean > = ({ live, linked }) => {
  linked ? (
    <span className="inline-flex items-center gap-1 rounded-md px-2 py-1 text-emerald-300">
      Python: x={live.x ?? "-"} · y={live.y ?? "-"} · i={live.i ?? "-"}
    </span>
  ) : null;
}

export const TreapBuildViz = () => {
  const [tab, setTab] = useState<Tab>("build");

  return (
    <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
      <div className="flex flex-wrap items-center justify-between">
        <div className="flex items-center gap-3">
          <div className="p-2.5 bg-emerald-600 text-white rounded-md">
            <span className="text-2xl"> </span>
          </div>
          <div>
            <h3 className="text-2xl font-bold text-white">Treap
            <p className="text-sm text-emerald-300">Память
          </div>
        </div>
        <div>
          <h3 className="text-2xl font-bold text-white">Treap
          <p className="text-sm text-emerald-300">Память
        </div>
      </div>

      <div className="flex flex-wrap gap-2 mb-5">
        {TABS.map((t) => {

```

```

-----
import { GraphTraversalViz } from "./GraphTraversalViz"
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { ColoringViz } from "./ColoringViz";
import { ComponentsViz } from "./ComponentsViz";
import { ComplexityClassesViz } from "./ComplexityClassesViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import ChapterImage from "./ChapterImage";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {

```

```

    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": {
    title: "Splay: механика → большое дерево → собери с",
    hint: "Три вкладки: покадровая механика поворотов в  
песочница «подними узел сам» с проверкой порядка.",
    Component: SplayPlayground,
  },
  "splay-rotations": {
    title: "Splay: механика → большое дерево → собери с",
    hint: "Три вкладки: покадровая механика поворотов в  
песочница «подними узел сам» с проверкой порядка.",
    Component: SplayPlayground,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: ... },
  "top-sort": { title: "Топологическая сортировка", Component: ... },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: ... },
  "graph-components": {
    title: "Компоненты связности: BFS-разметка",
    hint: "Каждый запуск обхода красит один «остров»; ч",
    Component: ComponentsViz,
  },
  "graph-articulation": { title: "Точки сочленения", Component: ... },
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: ... },
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: ... },
  "planarity-euler-formula": {
    title: "Планарность: K5/K3,3 + раскраска",
    hint: "Слева – непланарные графы с подсветкой пересеч",
    Component: () => (
      <div className="space-y-10">
        <PlanarityDemo />
        <ColoringViz />
      </div>
    )
  }
}

```

```
"string-z-func": { title: "Z-функция", Component: () => {
"aho-corasick": {
  title: "Ахо-Корасик",
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
  Component: () => {
    <div className="space-y-10">
      <WaterfallAnimationWidget />
      <SalmonAutomatonWidget />
    </div>
  },
},
"complexity-classes": {
  title: "Классы сложности: матрёшка  $P \subseteq NP \subseteq PSPACE$ ",
  hint: "Кликни класс или задачу; стрелки  $\leq$ р показывают вложения",
  Component: ComplexityClassesViz,
},
intro: { title: "Мнемокарточки", Component: MnemonicCards },
};
```

```
export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 76/96: src/components/WaterfallAnimationWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from 'react';
import { useVizRuntime, vizNumber, vizString } from 'viz-runtime';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
```

```
// Структура заранее подготовленного красивого бора для Waterfall
```

```
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {
```

```
  id: number;
```

```
  label: string; // путь от корня
```

```
  char: string;
```

```
  x: number; // Координаты для SVG водопада
```



```

export const WaterfallAnimationWidget: React.FC = () => {
  // Переключатель режимов: 'training' (Полный BFS обхо
  const [activeMode, setActiveMode] = useState<'trainin

  // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
  const [textToScan, setTextToScan] = useState<string>(
  const [currentIndex, setCurrentIndex] = useState<numb
  const [currentNode, setCurrentNode] = useState<number>
  const [isSearchDone, setIsSearchDone] = useState<bool
  const [searchLog, setSearchLog] = useState<string>(
    'Режим поиска: Введите текст и жмите «Следующий шаг
  );
  const [jumpPath, setJumpPath] = useState<{ from: numb
  const [foundMatches, setFoundMatches] = useState<{ in
  const [activeSearchCodeLine, setActiveSearchCodeLine]

  // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
  // Шаги обучения от 0 до 8 (по числу вершин в боре, и
  const [trainStep, setTrainStep] = useState<number>(0)

  // — Синхронизация с Python-компилятором: s/текст, i
  // v — текущая вершина автомата. Влияет на режим поис
  const runtime = useVizRuntime();
  const liveText = vizString(runtime?.variables?.s) ??
  const liveI = vizNumber(runtime?.variables?.i);
  const liveNode = vizNumber(runtime?.variables?.v) ??
  useEffect(() => {
    if (liveText) {
      setTextToScan(liveText.toUpperCase().replace(/[^A
      resetSearchSimulation();
    }
  }, [liveText]);
  useEffect(() => {
    if (liveI !== null) setCurrentIndex(Math.max(0, Mat
  }, [liveI, textToScan.length]);
  useEffect(() => {
    if (liveNode !== null && WATERFALL_NODES[liveNode])

```

```

        setJumpPath({ from: curr, to: nextNode, type: 'no
curr = nextNode;
setActiveSearchCodeLine(3);
} else {
setActiveSearchCodeLine(2);
let jumps = 0;
while (curr !== 0 && TRIE_TRANSITIONS[curr][char])
    curr = WATERFALL_NODES[curr].fail;
    jumps++;
}
const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
if (originalCurr === 0) {
    logMsg += `В корне нет перехода по '${char}'. Л
    setJumpPath(null);
} else {
    logMsg += `Поток по '${char}' у вершины #${orig
        рыжок по fail-ссылке в #${curr} (${WATERFALL_I
        . `;
    setJumpPath({ from: originalCurr, to: nextNode,
    }
    curr = nextNode;
}

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
    if (WATERFALL_NODES[temp].is_terminal) {
        WATERFALL_NODES[temp].words.forEach((word) :
            currentMatches.push({ index: currentInd
            matchedWordsForLog.push(word);
        });
    }
    temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {

```

```

    title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
    desc: 'Обрабатываем дочернюю вершину #7 (S) на Depth 1. Родитель: "S".
    а-разведчик плывет в ROOT и ищет ветку по "E". У корня нет ветки "E".
    Суффикс "S"!',
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'S',
    checkingBranchesFrom: null,
},
{
    targetNodeId: 2,
    title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
    desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель: "S".
    а-разведчик плывет в ROOT и ищет ветку по "E". У корня нет ветки "E".
    Суффикс "S"!',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'E',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 5,
    title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
    desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель: "S".
    "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT. Суффикс "S"!',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'I',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) – КАК СТАТЬ ШЕФ-ОХОТНИКОМ?',
    desc: 'ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на Depth 2. Родитель: "S".
    Рыба плывет в ROOT и ищет ветку "H". У корня нет ветки "H".
    Суффикс "H"!',
    codeLine: 4,

```

```

    {
      targetNodeId: 4,
      title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
      desc: 'Финальная вершина #4 (HERS) на Depth 4. Роль  
#7 (S). fail[HERS] = #7 (S). Автомат полностью  
codeLine: 4,
      scoutPos: 0,
      inspectedParentFail: 0,
      checkTargetChar: 'S',
      checkingBranchesFrom: 0,
    },
  ];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainStep];

```

```

return (
  <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700" >
    {/* Шапка и переключатель режимов */}
    <div className="flex flex-wrap items-center justify-between" >
      <div className="flex items-center space-x-3" >
        <span className="text-3xl" > </span>
        <div>
          <h4 className="text-2xl font-bold text-white" >
            Анимация водопада: Полное обучение автомата
          </h4>
          <p className="text-sm text-slate-400" >
            Визуализация пошагового построения всех функций
          </p>
        </div>
      </div>
    </div>

    {/* Переключатель режимов */}
    <div className="bg-slate-900 p-1.5 rounded-xl border-1px border-slate-800" >
      <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p-1`}
      >

```

```

        <button
          key={idx}
          onClick={() => setTrainStep(idx)}
          className={`w-full text-left px-3 p-2 ${
            idx === trainStep
              ? 'bg-indigo-600 text-white font-medium'
              : 'bg-slate-800 text-slate-400'
          }`}
        >
          <span className="line-clamp-1">{stepInfo[idx]}
          {idx === trainStep && <span className="font-medium">
            </button>
          </div>
        </div>

    <div className="flex gap-2">
      <button
        onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
        disabled={trainStep === 0}
        className="flex-1 bg-slate-700 hover:bg-slate-600 text-white font-medium transition-all"
      >
        Назад
      </button>
      <button
        onClick={() => setTrainStep((prev) => Math.min(TRAINING_STEPS_INFO.length - 1, prev + 1))}
        disabled={trainStep === TRAINING_STEPS_INFO.length - 1}
        className="flex-1 bg-indigo-600 hover:bg-indigo-500 text-white font-medium transition-all shadow-lg shadow-indigo-900/50"
      >
        Далее
      </button>
    </div>
  </div>

  /* Блок с кодом построения fail-ссылки и опи

```

```

        <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
        <span>5. else fail[u] = root; // Bero
        {currentTrainInfo.codeLine === 5 && <
root</span>}
        </div>
    </div>
</div>

<div>
    <h5 className="text-slate-200 font-bold m
    <Lightbulb className="w-4 h-4 text-amber
    </h5>
    <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
        {currentTrainInfo.desc}
    </div>
</div>
</div>
) : (
    /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
    <div className="grid grid-cols-1 lg:grid-cols-3
    <div className="bg-slate-900/60 p-5 rounded-x
    <div>
        <h5 className="text-white font-bold mb-2
        <span>» </span> Текст для сканирования
        </h5>
        <p className="text-xs text-slate-400 mb-4
        Введи текст, чтобы лосось просканировал
        </p>
        <input
            type="text"
            value={textToScan}
            onChange={handleTextChange}
            maxLength={15}
            placeholder="ВВЕДИТЕ ТЕКСТ..."
            className="w-full bg-slate-800 border b

```

```

        <span>{currentIndex === 0 ? 'Начать пла
        <ArrowRight className="w-4 h-4" />
      </button>
    </div>
  </div>

<div className="lg:col-span-2 bg-slate-900/60" >
  <div>
    <h5 className="text-white font-bold mb-3" >
      <span>❌</span> Текущее действие в коде
    </h5>
    <div className="bg-slate-950 p-4 rounded-1" >
      <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold` :
        <span>1. curr = state; char = text[i]
        {activeSearchCodeLine === 1 && <span <
инициализация</span>}
      </div>
      <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
        <span>2. while (curr !== root && !trie
        {activeSearchCodeLine === 2 && <span <
рыжок по fail!</span>}
      </div>
      <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
        <span>3. curr = trie[curr].get(char)
        {activeSearchCodeLine === 3 && <span <
ем по течению</span>}
      </div>
      <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
        <span>4. if (is_terminal[curr]) repor
        {activeSearchCodeLine === 4 && <span <
овпадение!</span>}
      </div>
    </div>
  </div>
</div>

```

```
</div>
```

```
{/* Само SVG дерево водопада */}
```

```
<div className="w-full overflow-x-auto custom-s
```

```
  <svg viewBox="0 0 800 520" className="w-full
```

```
  {/* Горизонтальные линии и подписи уровней
```

```
  {[
```

```
    { depth: 0, y: 60, label: 'Depth 0 (Корень
```

```
    { depth: 1, y: 160, label: 'Depth 1 (H, S
```

```
    { depth: 2, y: 260, label: 'Depth 2 (HE, I
```

```
    { depth: 3, y: 360, label: 'Depth 3 (HER,
```

```
    { depth: 4, y: 460, label: 'Depth 4 (HERS
```

```
  ].map((level) => {
```

```
    <g key={`depth-line-${level.depth}`}>
```

```
      <line
```

```
        x1="20"
```

```
        y1={level.y}
```

```
        x2="780"
```

```
        y2={level.y}
```

```
        stroke="#334155"
```

```
        strokeWidth="1"
```

```
        strokeDasharray="4,4"
```

```
        opacity="0.4"
```

```
      />
```

```
      <rect x="25" y={level.y - 12} width="14"
```

```
    />
```

```
    <text x="32" y={level.y + 4} fill="#94a
```

```
      {level.label}
```

```
    </text>
```

```
  </g>
```

```
  ])]
```

```
  {/* Рендер прямых переходов (потоков воды)
```

```
  {Object.entries(TRIE_TRANSITIONS).map(([from
```

```
    const fromNode = WATERFALL_NODES[Number(f
```

```
    return Object.entries(children).map(([cha
```

```
      const toNode = WATERFALL_NODES[toId];
```



```

        .y) / 2}}`}>
            <rect x="-15" y="-12" width="30
strokeWidth="2" />
            <text x="0" y="5" fontSize="14"
                {isMatchBranch ? ' ' : ' '}>
            </text>
        </g>
    }}
</g>
);
});
}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
    if (node.id === 0) return null;
    const targetNode = WATERFALL_NODES[node.failId];

```

```

    // В режиме обучения показываем fail-ссылку
    // Найдем индекс шага, на котором обработали
    const stepIdxForNode = TRAINING_STEPS_INFO[node.id].stepIdx;
    if (activeMode === 'training' && trainStep === stepIdxForNode)

```

```

        let isHighlighted = activeMode === 'search' || activeMode === 'fail';
        let isJustEstablished = activeMode === 'training' && trainStep === stepIdxForNode;
        // Если это fail-ссылка

```

```

        // Вычисляем изогнутую кривую для красоты
        const dx = targetNode.x - node.x;
        const dy = targetNode.y - node.y;
        const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
        const cy = node.y + dy / 2 - 30;

```

```

        return (
            <g key={`fail-${node.id}`}>
                <path
                    d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
                />
            </g>
        );
    }
}

```

```

        fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : '#f8d7da'}
        stroke={isCurrent ? '#ffffff' : isTargetChild ? '#f8d7da' : '#f8d7da'}
        strokeWidth={isCurrent || isTargetChild ? 2 : 1}
        className="transition-all duration-300"
    />

    { /* Текст внутри вершины */ }
    <text
      x={node.x}
      y={node.y + 5}
      fill={isCurrent || isTargetChild ? '#f8d7da' : '#f8d7da'}
      fontSize={isCurrent || isTargetChild ? 12 : 10}
      fontWeight="bold"
      textAnchor="middle"
    >
      {node.label}
    </text>

    { /* Метка искомой вершины в режиме обхода */ }
    {isTargetChild && (
      <g transform={`translate(${node.x}, ${node.y})`}
        <rect x="-35" y="-12" width="70" height="24" fill="#f8d7da" stroke="#f8d7da"
        <text x="0" y="2" fontSize="10" fill="#f8d7da"
          ИЩЕМ FAIL
        </text>
      </g>
    )}

    { /* Индикатор словарных слов */ }
    {hasWords && (
      <g transform={`translate(${node.x} + 10, ${node.y} + 10)`}
        <circle cx="0" cy="0" r="9" fill="#f8d7da" stroke="#f8d7da"
        <text x="0" y="3" fontSize="10" fill="#f8d7da"
          *
        </text>
      </g>
    )}
  </g>

```

```

</h5>
{foundMatches.length === 0 ? (
  <p className="text-sm text-slate-400 italic">
    Пока ничего не поймали. Запустите шаги слева
  </p>
) : (
  <div className="flex flex-wrap gap-2">
    {foundMatches.map((match, idx) => (
      <div
        key={idx}
        className="bg-emerald-950 border border-
-sm font-bold shadow animate-[scaleUp_0.3s_
>
        <CheckCircle2 className="w-4 h-4 text
        <span>{match.word}</span>
        <span className="text-[10px] bg-emera
        Индекс {match.index}
        </span>
      </div>
    ))}
  </div>
)}
</div>
)}
</div>
);
};

```

=====

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

=====

```

    if (chapterId) emitVizDemo(chapterId, mode === "2d"
    }, [chapterId, mode]);

    return (
      <div className="w-full bg-slate-950 rounded-2xl border
        <div className="flex gap-2 mb-4 overflow-x-auto no
          {{
            [
              ["1d", "1. Одномерные суммы"],
              ["2d", "2. Двумерные суммы"],
            ] as const
          }.map(([key, label]) => {
            <button
              key={key}
              onClick={() => setMode(key)}
              className={`px-3 py-2 rounded-lg text-xs sm
                mode === key ? "bg-indigo-600 text-white"
              }`
            >
              {label}
            </button>
          }}}
        </div>
        {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
      </div>
    );
  };
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {
  const pref = useMemo(
    () => A1.reduce<number[]>((acc, v) => [...acc, acc[
    []
  );

  /** Что сейчас под курсором: префикс P[i] или элемент
  const [hover, setHover] = useState<{ kind: "pref" | "
  const [range, setRange] = useState<{ l: number; r: nu

```

```

<div className="w-[46px] shrink-0 text-right"
  {A1.map((v, i) => {
    const inCover = covered && i >= covered.f
    const inRange = i >= range.l && i <= range.r
    const isDrivenAdd = shownDriven !== null
    return (
      <div
        key={i}
        onMouseEnter={() => setHover({ kind: 'add' })}
        onMouseLeave={() => setHover(null)}
        onClick={() => setRange((r) => (i < r.l ? r : { l: i, r: r.r }))}
        className={` ${cellBase} cursor-pointer`
          isDrivenAdd
            ? "bg-amber-500 text-black ring-2"
            : inCover
              ? "bg-indigo-500 text-white ring-2"
              : inRange
                ? "bg-indigo-900/70 text-indigo-300"
                : "bg-slate-800 text-slate-300"
          }`
        title={`a[${i}] = ${v}`}
      >
        {v}
      </div>
    );
  })}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
  <div className="w-[46px] shrink-0 text-right"
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" && i === range.l;
      const isL = i === range.l;
      const isR = i === range.r + 1;
      const liveValue = liveP?.[i];
      const blank = hasLiveP && (liveValue === null || liveValue === 0);
      const masked = !hasLiveP && shownDriven !== null;
    })}
  </div>

```

```

    <span className="font-mono text-emerald-300"
    {hasLiveP && <> = <span className="font-mono
    <span className="ml-1 text-slate-500">Пусто
  </p>
) : hover?.kind === "pref" ? {
  hover.i === 0 ? {
    <p className="text-slate-300">
      <b className="text-emerald-400">P[0] = 0<
      отрезка, начинающегося с нуля.
    </p>
  ) : {
    <p className="text-slate-300">
      <b className="text-emerald-400">
        P[{hover.i}] = {pref[hover.i]}
      </b>{" "}
      – это сумма всего, что набежало от начала
    <span className="font-mono text-indigo-300"
      a[0..{hover.i - 1}] = {A1.slice(0, hove
    </span>
    </p>
  )
) : hover?.kind === "a" ? {
  <p className="text-slate-300">
    <b className="text-indigo-400">
      a[{hover.i}] = {A1[hover.i]}
    </b>{" "}
    входит во все префиксы, начиная с{" "}
    <span className="font-mono text-emerald-300"
    подвинуть границу запроса.
  </p>
) : {
  <p className="text-slate-500">
    Наведите курсор на любое число: у <b classN
    у <b className="text-indigo-400">a</b> – ку
  </p>
}}
</div>

```

```

    }, [n, m]));

const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

useEffect(() => {
    const r1 = vizNumber(vars?.r1);
    const c1 = vizNumber(vars?.c1);
    const r2 = vizNumber(vars?.r2);
    const c2 = vizNumber(vars?.c2);
    if (r1 === null || c1 === null || r2 === null || c2 === null) return;
    // Python-шаблон (и глава) используют 1-индексные координаты.
    // клетка (1, 1) – левый верхний элемент матрицы. В JS же 0-индексные.
    // 0-индексные, поэтому переводим: (1,1) → (0,0).
    const to0 = (v: number, hi: number) => Math.max(0, v - 1);
    setRect({
        r1: to0(r1, n - 1),
        c1: to0(c1, m - 1),
        r2: to0(r2, n - 1),
        c2: to0(c2, m - 1),
    });
}, [vars, n, m]);

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
    if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
    if (i === rect.r1 && j === rect.c2 + 1) return "B";
    if (i === rect.r2 + 1 && j === rect.c1) return "C";
    if (i === rect.r1 && j === rect.c1) return "D";
    return null;
};

return (

```

```

const liveRow = vizArray(liveS?.[i]);
const liveValue = liveRow?.[j];
const blank = hasLiveS && (liveValue === 0);
const shownValue = hasLiveS ? liveValue : 0;
const corner = cornerOf(i, j);
const isHover = hover?.i === i && hover?.j === j;
const isCompilerCell = liveI === i && liveJ === j;
const cornerColor =
  corner === "A"
    ? "bg-emerald-600 text-white"
    : corner === "D"
    ? "bg-sky-600 text-white"
    : corner === "C"
    ? "bg-rose-600 text-white"
    : "bg-slate-900 border border-slate-800";
return (
  <div
    key={j}
    onMouseEnter={() => setHover({ i, j })}
    onMouseLeave={() => setHover(null)}
    className={` ${cellBase} cursor-pointer ${
      isCompilerCell
        ? "bg-emerald-600 text-white"
        : isHover
        ? "bg-amber-500 text-white"
        : blank
        ? "bg-slate-950 text-slate-400"
        : cornerColor
      }`}
    title={hasLiveS ? `S[${i}][${j}]` : ""}
  >
    {blank ? "." : String(shownValue)}
    {corner && !isHover && (
      <span className="absolute -top-px -right-px"
        style={{border: 1px solid black, width: 10px, height: 10px, display: flex, align-items: center, justify-content: center; font-size: 8px; font-weight: bold; line-height: 10px; text-align: center; color: cornerColor; background-color: white; border-radius: 5px; opacity: 0.5;}}
      >
        {corner}
      </span>
    )}
  </div>
)

```



```
import React, { useContext, useEffect, useState } from
import {
  emitVizDemo,
  useVizRuntime,
  vizArray,
  vizNumber,
  vizRecord,
  VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () =>
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 <<
  }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
```



```

    ? { i: liveI, j: liveJ }
    : null;

```

```

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
  for (let i = 0; i + (1 << j) <= N; i++) {
    computeSteps.push({ i, j });
  }
}

```

```

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

```

```

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-600">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$ 
        </p>
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}
          className="p-2 bg-slate-800 hover:bg-slate-700"
          disabled={step === 0}
        >
          <ChevronLeft size={20} />
        </button>
        <button
          onClick={() => setStep(Math.min(maxStep, step + 1))}
          className="p-2 bg-slate-800 hover:bg-slate-700"
          disabled={step === maxStep}
        >
          <ChevronRight size={20} />
        </button>
      </div>
    </div>
    <div>
      <table>
        <caption>Состояние Sparse Table (Min)</caption>
        <thead>
          <tr>
            <th>i \ j</th>
            <th>1</th>
            <th>2</th>
            <th>3</th>
            <th>4</th>
            <th>5</th>
            <th>6</th>
            <th>7</th>
            <th>8</th>
            <th>9</th>
            <th>10</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <th>0</th>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
            <td>0</td>
          </tr>
          <tr>
            <th>1</th>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
            <td>1</td>
          </tr>
          <tr>
            <th>2</th>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
            <td>2</td>
          </tr>
          <tr>
            <th>3</th>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
            <td>3</td>
          </tr>
          <tr>
            <th>4</th>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
            <td>4</td>
          </tr>
          <tr>
            <th>5</th>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
            <td>5</td>
          </tr>
          <tr>
            <th>6</th>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
            <td>6</td>
          </tr>
          <tr>
            <th>7</th>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
            <td>7</td>
          </tr>
          <tr>
            <th>8</th>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
            <td>8</td>
          </tr>
          <tr>
            <th>9</th>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
            <td>9</td>
          </tr>
          <tr>
            <th>10</th>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
            <td>10</td>
          </tr>
        </tbody>
      </table>
    </div>
  </div>
)

```

```

</div>
<div className="mt-2 text-xs min-h-[36px]">
  {hover && covered ? (
    <p className="text-slate-300">
      <b className="text-sky-400">
        ST[{hover.i}][{hover.j}] = {stID[hover.i]}
      </b>{" "}
      – это минимум на отрезке{" "}
      <span className="font-mono text-sky-300">
        [{covered.from}, {covered.to}]
      </span>{" "}
      длины  $2^{\{hover.j\}} = \{1 \ll \{hover.j\} : \{ \}$ 
      <span className="font-mono text-slate-400">
        min({arrID.slice(covered.from, covered.to)}
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор (или тапните) на любое чи
    </p>
  )}
</div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-50">
    <div className="grid grid-cols-[auto_repeat(4,1fr)]">
      { /* Headers */ }
      <div className="text-slate-500 font-mono text-sm">
        i \ j
      </div>
      <div className="text-center font-bold text-sm">
         $2^0$  (L=1)
      </div>
      <div className="text-center font-bold text-sm">
         $2^1$  (L=2)
      </div>
    </div>
  </div>
</div>

```

```

        const currentStep = computeSteps[stepIndex]
        isActive = currentStep.i === i && currentStep.j === j
        if (currentStep.j - 1 === j) {
            if (currentStep.i === i) isSrc1 = true
            if (currentStep.i + (1 << (currentStep.j - 1)) === i) isSrc2 = true
        }
    }

    // Готовые вычисленные уровни демо не нужны
    // уровень j=0 всегда берём из уже существующих
    const shownValue = compilerLinked ? compilerLinked : undefined
    const displayValue =
        shownValue === undefined || shownValue === null
        ? "."
        : typeof shownValue === "object"
        ? JSON.stringify(shownValue)
        : String(shownValue);

    return (
        <div key={j} className="relative flex flex-direction: column align-items: center"
        {!isValid ? (
            <div className="w-[clamp(30px,9vw,17px)] font-normal text-center"
        ) : (
            <motion.div
                onMouseEnter={() => (compilerLinked ? compilerLinked : undefined)}
                onMouseLeave={() => setHover(i, j)}
                onClick={() => (compilerLinked ? compilerLinked : undefined)}
            >
                {displayValue}
            </motion.div>
        )}
    )

    initial={{ opacity: 0, scale: 1 }}
    animate={{
        // При st = [] каркас уже существует
        opacity: compilerLinked ? 1 : 0,
        scale: compilerLinked || isSrc1 || isSrc2 ? 1 : 1.1,
    }}
    className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200
    clamp(11px,3vw,17px)] font-bold transition-colors duration-200
    hover && hover.i === i && hover.j === j
    ? "bg-sky-500 text-white`

```

```

        </motion.svg>
      })
    </div>
  );
}
</React.Fragment>
)})
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
  {compilerLinked && liveTarget ? (
    <p className="text-base text-slate-300">
      <span className="text-emerald-400 font-bold">
        <span className="font-mono text-white">i =
        – активна ячейка <span className="font-mono
        {(hasLiveTable || hasLiveSource) && <> = <sp
        ?.[liveTarget.j] ?? (liveTarget.j === 0 ? l
      </p>
    ) : step > 0 && step <= maxStep ? (
      (() => {
        const c = computeSteps[step - 1];
        const half = 1 << (c.j - 1);
        return (
          <p className="text-lg text-slate-300">
            <span className="text-white font-bold">
              ST[{c.i}][{c.j}]
            </span>{" "}
            = min(
              <span className="text-emerald-400 font-l
                ST[{c.i}][{c.j - 1}]
              </span>
            ,
            <span className="text-amber-400 font-bo
              ST[{c.i + half}][{c.j - 1}]
            </span>

```

```

    ? Math.max(0, Math.min(3, Math.trunc(liveK)))
    : hasLiveSource
      ? 0
      : k;

```

```

// Уровень и клетка берутся прямо из k/r/c; номер строки
useEffect(() => {
  setHover(null);
  setLocked(null);
}, [k]);

```

```

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

```

```

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900 p-3 sm:p-4">
      <h3 className="text-xl font-bold text-indigo-400">Задача
      <p className="text-sm text-slate-400 mb-6 text-indent-1">
        Для наглядности показываем только <b className="text-white">
          <br/>
          <span className="text-rose-300 animate-pulse">
            иц!</span>
        </p>
      <div className="flex bg-slate-950 p-1 rounded-lg">
        {[0, 1, 2, 3].map(step => (
          <button
            key={step}
            onClick={() => { setK(step); setHover(null); }}
            className={`px-4 py-2 rounded-md text-sm text-slate-400
              hover:text-white hover:bg-slate-900`}
          >
            {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
          </button>
        ))}
      </div>
    </div>
  </div>

```

```

const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

    if (isActive) {
        if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
        } else {
            const prevL = 1 << (shownK
            const isTop = r < activeCel
            const isLeft = c < activeCe

                if (isTop && isLeft) {
                    highlightClass = 'bg-ros
md:scale-[1.08]';
                } else if (isTop && !isLeft
                    highlightClass = 'bg-blur
] md:scale-[1.08]';
                } else if (!isTop && isLeft
                    highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
                } else {
                    highlightClass = 'bg-amb
3)] md:scale-[1.08]';
                }
            zIndex = 'z-10';
        }
    }

    return (
        <div
            key={idx}
            onMouseEnter={() => { if(is
            onMouseLeave={() => { if(is
            onClick={() => {
                if(isCurr) {
                    if(locked && locked.

```



```

        </>
        <p className="mb-4">При <span classN
драты имеют размер <span className="font-bo
        <div className="bg-[#0a0fle] rounded
            <span className="text-slate-500">/
            <span className="text-indigo-400">
        </div>
        <p className="mt-6 italic text-indig
            <Play size={14} className="fill-in
a.
        </p>
        </>
    )}
</div>
) : (
    <div className="flex flex-col gap-4 animat
        <div className="bg-[#0a0fle] rounded-xl l
ate-300 shadow-inner overflow-x-auto">
            <span className="text-slate-500">// Те
            <span className="text-purple-400">int<
="text-amber-300">1</span>}<br/><br/>
            <span className="text-slate-500">// Кв
            <span className="text-indigo-400">ST</
            &nbsp;&nbsp;&nbsp;<span className="text-rose
e="text-amber-300">1</span>],          <sp
            &nbsp;&nbsp;&nbsp;<span className="text-blue
ext-amber-300">1</span>][k-<span className=
pan><br/>
            &nbsp;&nbsp;&nbsp;<span className="text-emer
="text-amber-300">1</span>][k-<span classNa
span><br/>
            &nbsp;&nbsp;&nbsp;<span className="text-ambe
white">c + pL</span>][k-<span className="te
ame="text-slate-500">// Правый Низ</span><b
        {'}}'));
    </div>

    {activeCell ? (

```

```

        shadow-[0_0_8px_#f59e0b]"></div> ST[{activeCell.r, activeCell.c} =>
            <span className="text-white">{ST2D[activeCell.r][activeCell.c][shownK]}</span>
        </div>
    </>
  )
  })()}
</div>

  <div className="pt-3 mt-3 border-top border-left border-right border-bottom"
    > <span className="text-white">{ST2D[activeCell.r][activeCell.c][shownK]}</span>
  </div>
  <p className="mt-4 text-[12px] font-mono">
    список матриц (слева) вниз, чтобы увидеть, как они меняются.
  </p>
  ) : {
    <div className="bg-slate-950/40 border border-slate-700 p-2 flex items-center justify-center animate-pulse">
      Наведите курсор на матрицу {L}x{L}
    </div>
  )}
</div>
})
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);
  const runtime = useVizRuntime();

```

// Координаты запроса – те же захардкоженные имена,

```

        <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
        {val2D[r][c]}
        </div>
    )
    }}}

```

```

    <div
      className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
      style={{
        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
      }}
    />

```

```

    /* Показываем 4 угла в самой матрице
    <div className="absolute pointer-e
      shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
    <div className="absolute pointer-e
      shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`,
    <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
    <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
    </div>

```

```

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset 0 2px 10px rgba(

```

```

        <span className="text-slate-500">// 1.
        <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
        <span className="text-purple-400">int<
ext-amber-300">1</span>); <span className="
        <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
        <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b>

        <span className="text-slate-500">// 2.
        <span className="text-indigo-400">int<
        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-900">
r1</span>][<span className="text-white bg-r

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
во, чтобы правый край уперся в c2)</span><b>
        &nbsp;&nbsp;&nbsp;<span className="text-blue-500">
r1</span>][<span className="text-white bg-b

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
рх, чтобы нижний край уперся в r2)</span><b>
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">
nded">r2 - Lx + 1</span>][<span className="

        &nbsp;&nbsp;&nbsp;<span className="text-slate-500">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-500">
">r2 - Lx + 1</span>][<span className="text
        {'}}''));
    </div>

```

```

<div className="w-full flex justify-center"
  <div className="flex flex-col items-center"
    <h4 className="text-slate-400 font-bo

```

Откуда берутся эти 4 числа: матриц

```

        <div className="mt-auto bg-slate-950 font
    )]">
        <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
        <span className="text-slate-400 font
            <span className="text-rose-400">
            <span className="text-blue-400 m
n>
            <span className="text-emerald-40
span>
            <span className="text-amber-400 r
            &nbsp;<span className="text-slate-4
            <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
        </div>
    </div>

    </div>
</div>
)
}
```

=====

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

=====

ФАЙЛ 79/96: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 137 | РАЗМЕР

```

        </div>
    </main>
</div>
```

```
<script>
```

```

        // Just some visual feedback on buttons
        document.querySelectorAll('[id^="theme-"]')
            .forEach(btn => {
                btn.classList.remove('outline', 'outline-1');
            });

        if (theme === 'dark') document.getElementById('dark')
        if (theme === 'light') document.getElementById('light')
        if (theme === 'terminal') document.getElementById('terminal')

        document.getElementById('btn-theme-dark').classList.add('bg-white', 'text-black', 'transition-colors', 'duration-300');
        document.getElementById('btn-theme-light').classList.add('bg-black', 'text-white', 'transition-colors', 'duration-300');
        document.getElementById('btn-theme-notebook').classList.add('bg-black', 'text-white', 'transition-colors', 'duration-300');
    }

    // Initialize theme and mode
    setTheme('dark');

    // Setup intersection observer for TOC highlighting
    document.addEventListener('DOMContentLoaded', () => {
        const mobileMenuBtn = document.getElementById('mobile-menu');
        const mobileDrawer = document.getElementById('mobile-drawer');
        const closeDrawerBtn = document.getElementById('close-drawer');
        const drawerOverlay = document.getElementById('drawer-overlay');

        function toggleDrawer() {
            const isClosed = mobileDrawer.classList.contains('hidden');
            if (isClosed) {
                mobileDrawer.classList.remove('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.remove('hidden');
                    // Timeout allows display:block to take effect
                }
            } else {
                mobileDrawer.classList.add('hidden');
                if (drawerOverlay) {
                    drawerOverlay.classList.add('hidden');
                }
            }
        }

        mobileMenuBtn.addEventListener('click', toggleDrawer);
        closeDrawerBtn.addEventListener('click', toggleDrawer);
        drawerOverlay.addEventListener('click', toggleDrawer);
    });

```

```
        const observer = new IntersectionObserver((
          entries.forEach(entry => {
            if (entry.isIntersecting && !document
              // Highlight active TOC link lo
              const id = entry.target.id;
              document.querySelectorAll('aside
                if (link.getAttribute('href
                  link.classList.add('bg-
                } else {
                  link.classList.remove('
                }
              });
            }
          });
        }, observerOptions);

        document.querySelectorAll('section[id^="que
          observer.observe(section);
        });
      });
    </script>
  </body>
</html>
```

ФАЙЛ 80/96: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1000

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
```

```
.formula-scroll::-webkit-scrollbar {
  display: none;
}

mjax-container {
  max-width: 100% !important;
}
mjax-container[display="true"] {
  overflow-x: auto;
  overflow-y: hidden;
  max-width: 100%;
  scrollbar-width: none;
  -ms-overflow-style: none;
  -webkit-overflow-scrolling: touch;
}
mjax-container[display="true"]::-webkit-scrollbar
  display: none;
}

@media (max-width: 640px) {
  html { font-size: 13px; }

  .uno-card { width: 40px; height: 60px; font-
  .uno-card::before, .uno-card::after { font-
  .uno-card::before { top: 1px; left: 2px; }
  .uno-card::after { bottom: 1px; right: 2px; }

  .uno-card.mini { width: 28px !important; he
  us: 4px; }
  .uno-card.mini::before, .uno-card.mini::aft

  .uno-equation .text-2xl { font-size: 1.2rem
  .uno-equation.gap-4 { gap: 0.5rem !important

  /* Allow horizontal scroll for equations in
  .formula-scroll { flex-wrap: nowrap !import
  px; }
}
```



```
body.theme-notebook {
  background-color: #fcf8eb !important; /* light beige */
  color: #3b2818 !important; /* brown ink */
  background-image: repeating-linear-gradient(
    45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
  background-attachment: local !important;
  font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
  background-color: rgba(255, 255, 255, 0.7);
  border-color: #cbd5e1 !important;
  box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 {
  color: #cbd5e1 !important; }
body.theme-notebook .text-slate-400 { color: #546e7a !important; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-800 {
  border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace", "Courier New", "Serif"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
  0%, 95%, 98%, 100% { transform: scaleY(1); }
  96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
  transform-origin: center;
  transform-box: fill-box;
  animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
  transform-origin: center;
  transform-box: fill-box;
  animation: creepy-blink 5.2s infinite;
  animation-delay: 1.5s;
}
```

```

        <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-orange" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#f97316" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-pink" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#f472b6" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-cyan" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-indigo" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-lime" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="black" stroke-width="1" />
    </marker>
    <marker id="arrow-green" markerWidth="6" markerHeight="6" orient="auto" />
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="black" stroke-width="1" />
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20" fill="#fff2cc" stroke="black" stroke-width="1" />
        <rect x="6" y="-5" width="8" height="6" fill="#fff2cc" stroke="black" stroke-width="1" />
        <rect x="26" y="-5" width="8" height="6" fill="#fff2cc" stroke="black" stroke-width="1" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20" fill="#ffe0b2" stroke="black" stroke-width="1" />
        <rect x="6" y="-5" width="8" height="6" fill="#ffe0b2" stroke="black" stroke-width="1" />
        <rect x="26" y="-5" width="8" height="6" fill="#ffe0b2" stroke="black" stroke-width="1" />
        <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10" fill="#fff2cc" stroke="black" stroke-width="1" />
        <rect x="6" y="-5" width="8" height="6" fill="#fff2cc" stroke="black" stroke-width="1" />
        <rect x="26" y="-5" width="8" height="6" fill="#fff2cc" stroke="black" stroke-width="1" />
    </g>

```

```
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-
```

```
<!-- Левое меню (Оглавление) -->
```

```
<aside id="mobile-drawer" class="fixed inset-y-0 left-0
  nslate-x-full transition-transform duration-300 ease-in-out
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto">
```

```
  <div class="h-full flex flex-col p-6 overflow-y-auto
    lg:rounded-2xl lg:border lg:border-slate-700">
    <div class="flex justify-between items-center">
      <h3 class="font-black text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-blue-300">
        Оглавление
      </h3>
      <button id="close-drawer-btn" class="text-slate-400 hover:text-slate-500">
        <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
          <path stroke-linecap="round" stroke-width="2" d="M6 18 18 6" />
        </svg>
      </button>
    </div>
    <nav class="space-y-3 text-sm font-medium">
      <div>
        <details class="group" open>
          <summary class="list-none cursor-pointer">
            <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
              1. Алгебраические структуры
            </a>
          </summary>
          <div class="pl-6 space-y-2 text-sm">
            <a href="#section-1-1" class="text-slate-400">1.1. Группы
            <a href="#section-1-2" class="text-slate-400">1.2. Кольца
            <a href="#section-1-3" class="text-slate-400">1.3. Модули
            <a href="#section-1-4" class="text-slate-400">1.4. Идеалы
          </div>
        </details>
      </div>
      <div>
        <details class="group">
          <summary class="list-none cursor-pointer">
            <a href="#question-2" class="text-blue-500 pl-3 font-bold text-blue-300">
              2. Линейная алгебра
            </a>
          </summary>
          <div class="pl-6 space-y-2 text-sm">
            <a href="#section-2-1" class="text-slate-400">2.1. Векторные пространства
            <a href="#section-2-2" class="text-slate-400">2.2. Линейные отображения
            <a href="#section-2-3" class="text-slate-400">2.3. Матрицы
            <a href="#section-2-4" class="text-slate-400">2.4. Характеристический многочлен
          </div>
        </details>
      </div>
    </nav>
  </div>
</aside>
```

```

        <a href="#q4-count" class="block">
</div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">
        <a href="#question-5" class="block">
5. Группа корней из единицы
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-lime-500">
        <a href="#q5-def" class="block">
        <a href="#q5-group" class="block">
        <a href="#q5-prim" class="block">
        <a href="#q5-crit" class="block">
        <a href="#q5-cyclic" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">
        <a href="#question-6" class="block">
6. Делимость, НОД и Евклид
        </a>
    </summary>
    <div class="pl-6 space-y-2 text-lime-500">
        <a href="#q6-1" class="block">
        <a href="#q6-2" class="block">
        <a href="#q6-3" class="block">
        <a href="#q6-4" class="block">
        <a href="#q6-5" class="block">
        <a href="#q6-6" class="block">
    </div>
</details>
```

9. Кольцо многочленов

```
    </a>
  </summary>
  <div class="pl-6 space-y-2 text-rose-500" >
    <a href="#q9-1" class="block text-rose-400">
    <a href="#q9-2" class="block text-rose-400">
    <a href="#q9-3" class="block text-rose-400">
    <a href="#q9-4" class="block text-rose-400">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
    10. Делимость и Схема Гильберта
  </summary>
  <div class="pl-6 space-y-2 text-rose-500" >
    <a href="#q10-1" class="block text-rose-400">
    <a href="#q10-2" class="block text-rose-400">
    <a href="#q10-3" class="block text-rose-400">
    <a href="#q10-4" class="block text-rose-400">
    <a href="#q10-5" class="block text-rose-400">
  </div>
</details>

<details class="group">
  <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
    11. НОД и НОК многочленов
  </summary>
  <div class="pl-6 space-y-2 text-cyan-500" >
    <a href="#q11-1" class="block text-cyan-400">
    <a href="#q11-2" class="block text-cyan-400">
```

```
        <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-400">
            14. Кратность корня. Корень
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q14-1" class="block">
        <a href="#q14-2" class="block">
        <a href="#q14-3" class="block">
            deg)</a>
        <a href="#q14-4" class="block border border-rose-500/30"> K
    </div>
</details>
```

```
<details class="group">
    <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
        15. Производная. Критерий
    </a>
</summary>
<div class="pl-6 mt-2 space-y-2">
    <a href="#q15-1" class="block">
    <a href="#q15-2" class="block">
    <a href="#q15-3" class="block">
    <a href="#q15-4" class="block">
</div>
</details>
```

```
<details class="group">
    <summary class="list-none cursor-pointer-red-500 pl-3 font-bold text-red-400">
        16. Неприводимые многочлены
    </a>
</summary>
```

```

</nav>
<script>
    document.querySelectorAll('details.'
        detail.addEventListener('toggle
            if (detail.open) {
                document.querySelectorA
                    if (other !== detail
                        other.removeAtt
                    }
            }));
        }
    });
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded')
    const sections = document.queryS
    = "q5-", div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('click
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting
                visibleId = entry.t

        // Highlight active
        document.querySelec

```

```
modal.style.height = '100vh';
modal.style.backgroundColor = 'white';
modal.style.zIndex = '99999';
modal.style.display = 'flex';
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = 'white';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за проблем с некоторыми браузерами, для загрузки файла необходимо открыть эту страницу в новой веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = 'white';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
```



```

        btnContainer.appendChild(closeB
    }
    btnContainer.appendChild(closeB
    box.appendChild(btnContainer);
    modal.appendChild(box);
    document.body.appendChild(modal
}

function downloadMD() {
    let clone = document.querySelec

    // Smart markdown parsing based

    // Smart markdown parsing based
    clone.querySelectorAll('h2').fo
    clone.querySelectorAll('h3').fo
    clone.querySelectorAll('.analog
    clone.querySelectorAll('li').fo
    clone.querySelectorAll('svg').f
        let caption = '';
        if (el.nextElementSibling &
            caption = el.nextElement
        }
        el.textContent = '\n\n[ Илл
    });
    clone.querySelectorAll('.img-ca

    let text = clone.innerText;
    text = text.replace(/\n{3,}/g,

    if (window !== window.top) {
        fallbackModal(text, 'Markdo
        return;
    }
    const blob = new Blob([text], {
    const url = URL.createObjectURL
    const link = document.createEle
    link.href = url;

```

```

/* Specific backgrounds
body.bg-white aside > d
body.bg-white [class*="
nt; }
body.bg-white [class*="
nt; }
body.bg-white [class*="
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cyar
body.bg-white .text-amb

```

```

        <div class="text-xs text-slate-
        <button onclick="downloadWord()
ont-bold flex items-center justify-center gap-
        <span> </span> Word
        </button>
        <a href="algebra.pdf" target="_b
-sm font-bold flex items-center justify-cen
        <span> </span> PDF
        </a>
        <button onclick="downloadMD()"
-b bold flex items-center justify-center gap-
        <span> </span> Markdown
        </button>
        <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-ce
        <span> </span> HTML
        </button>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-
        <div class="flex items-center gap-
            <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
            <button id="theme-light" on
r justify-center text-amber-50 hover:text-wh
">* </button>
                <button id="theme-terminal"
-center justify-center text-green-400 hover
терминала"> </button>
            </div>
        </div>
    </div>
</div>
</aside>

```

```

        </div>
    </header>

    <!-- ===== WTF CONTAINER =====>
    <div id="wtf-container" class="hidden">
        <h2 class="text-3xl font-black text-cen

        <div class="overflow-x-auto w-full pb-4"
            <div class="grid grid-cols-3 gap-4"
                <!-- Column headers -->
                <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
                <div class="bg-orange-900/40 bo
ольцо полиномов</div>
                <div class="bg-cyan-900/40 bord
сные числа</div>

                <!-- Row 1: База -->
                <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
                <div class="text-xs text-yellow
                <div class="font-bold text-slate
                </a>
                <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
                <div class="text-xs text-orange
                <div class="font-bold text-slate
                </a>
                <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
                <div class="text-xs text-cyan-5
                <div class="font-bold text-slate
                </a>

                <!-- Row 2: Базовые операции / Евкли
                <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
                <div class="text-xs text-yellow

```

```

<a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-500">
    <div class="font-bold text-slate-700">
</a>

<!-- Row 5: Корни / Уравнения -->
<div class="hidden md:block"></div>
<a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500">
    <div class="font-bold text-slate-700">
</a>
<a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
    <div class="text-xs text-cyan-500">
    <div class="font-bold text-slate-700">
</a>

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500">
    <div class="font-bold text-slate-700">
</a>
<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange-500">
    <div class="font-bold text-slate-700">
</a>
<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->

```

```
        <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
        <div class="text-xs text-red-500">
        <div class="font-bold text-slate-500">
        </a>
    </div>
</div>
```

```
    <!-- ЛИКБЕЗ SECTION INSERTED HERE -->
    <section id="proof-algorithms" class="m-0">
        <div class="flex items-center mb-6">
            <div class="bg-red-500/20 p-3 rounded-r-
                <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
                    <path stroke-linecap="round"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
                </svg>
            </div>
            <h2 class="text-3xl pr-4 border-r-2 border-slate-500">
        </div>
```

```
    <div class="prose prose-invert max-w-4xl">
        <p class="text-lg">
            Хватит тупить. Доказательств
        </p>
        <p>есть для того, чтобы жать на кнопки. Вот жес
```

```
    <h2 class="text-2xl font-bold border-bottom-2 border-slate-500">
    (Меметика)</h2>
```

```
    <!-- Прямое -->
    <div class="bg-slate-800/80 p-6">
        <h3 class="text-xl text-blue-400">
        <p class="text-sm text-slate-400">
        <ul class="list-disc pl-5">
            <li><span class="text-white">
k$).</li>
```

```
<li><span class="text-w
валось.</li>
<li><span class="text-w
Просто перепиши формулу с буквой $k$.</li>
<li><span class="text-w
усок $k$, замени его на жареного карася из
</ul>
</div>

<!-- Двойная делимость -->
<div class="bg-slate-800 p-6
  <h3 class="text-xl text-ora
  <p class="text-sm text-slate
  <ul class="list-disc pl-5 s
    <li><span class="text-w
      вверх по уравнениям).</li>
    <li><span class="text-w
      рху вниз, показывая, что $\delta \vdots r_i
    <li><span class="text-w
      х. Твой $d$ больше.</li>
  </ul>
</div>

<h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>

<div class="grid grid-cols-1 md
  <div class="bg-slate-800 p-
    <h4 class="text-red-400
    <p class="text-sm text-
    <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
</div>

<div class="bg-slate-800 p-
  <h4 class="text-red-400
  <p class="text-sm text-
```

```
<li class="bg-slate-800/50" style="border: 1px solid #000; padding: 10px; margin-bottom: 10px;">
  <span class="text-2xl" style="font-size: 1.5em; font-weight: bold; display: block; margin-bottom: 5px;">
    <div>
      <h4 class="text-cyan" style="margin: 0; font-size: 1.2em;">
        <p class="text-sm" style="margin: 0; font-size: 0.9em;">
          x двух, потом последних. <br>
          <code class="text-x" style="font-family: monospace; font-size: 0.8em;">
            </div>
          </li>

<li class="bg-slate-800/50" style="border: 1px solid #000; padding: 10px; margin-bottom: 10px;">
  <span class="text-2xl" style="font-size: 1.5em; font-weight: bold; display: block; margin-bottom: 5px;">
    <div>
      <h4 class="text-cyan" style="margin: 0; font-size: 1.2em;">
        <p class="text-sm" style="margin: 0; font-size: 0.9em;">
          есть сложение – есть ноль. Если есть умножение, то
          <code class="text-x" style="font-family: monospace; font-size: 0.8em;">
            </div>
          </li>

<li class="bg-slate-800/50" style="border: 1px solid #000; padding: 10px; margin-bottom: 10px;">
  <span class="text-2xl" style="font-size: 1.5em; font-weight: bold; display: block; margin-bottom: 5px;">
    <div>
      <h4 class="text-cyan" style="margin: 0; font-size: 1.2em;">
        <p class="text-sm" style="margin: 0; font-size: 0.9em;">
          авь их драться. Умножение всегда врывается и
          <code class="text-x" style="font-family: monospace; font-size: 0.8em;">
            </div>
          </li>
        </ul>
      </div>
    </section>
  </div>

<div id="questions-container">
  <!-- ===== ВОПРОС 1 =====>
```



```
-----  
/** Собирает CSS страницы: сначала пробуем правила, при  
async function collectCss(): Promise<string> {  
  const parts: string[] = [];  
  for (const sheet of Array.from(document.styleSheets))  
    try {  
      const rules = (sheet as CSSStyleSheet).cssRules;  
      parts.push(Array.from(rules).map((r) => r.cssText));  
    } catch {  
      const href = (sheet as CSSStyleSheet).href;  
      if (!href) continue;  
      try {  
        const res = await fetch(href);  
        if (res.ok) parts.push(await res.text());  
      } catch {  
        /* чужой домен – пропускаем */  
      }  
    }  
  }  
  return parts.join("\n");  
}
```

```
const EXTRA_CSS = `  
/* – правки для автономного файла – */  
html { color-scheme: dark; }  
body { background: #020617; color: #e2e8f0; font-family  
a.term-hint { text-decoration: none; }  
.export-shell { max-width: 62rem; margin: 0 auto; padding  
.export-head { border-bottom: 1px solid #1e293b; padding  
.export-note { background: #0f172a; border: 1px solid #  
  border-radius: .5rem; padding: .85rem 1rem; font-size  
.export-note a { color: #a5b4fc; }  
.export-gloss { margin-top: 3rem; border-top: 1px solid  
.export-gloss h2 { color: #fff; font-size: 1.25rem; fon  
.export-term { background: #0f172a; border: 1px solid #  
.export-term h3 { color: #c7d2fe; font-size: .95rem; fo  
.export-term p { margin: 0 0 .3rem; font-size: .85rem;  
.export-term .formal { color: #94a3b8; font-size: .8rem  
.export-term .cx { color: #6ee7b7; font-size: .78rem; f
```

```

    });

    // <details> в файле лучше раскрыть: иначе код и разб
    host.querySelectorAll("details").forEach((d) => d.set

    return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
  if (!termIds.length) return "";
  const items = termIds
    .map((id) => {
      const e = GLOSSARY_BY_ID.get(id);
      if (!e) return "";
      return `<div class="export-term" id="term-${escape
    <h3>${escapeHtml(e.title)}</h3>
    <p>${escapeHtml(e.short)}</p>
    ${e.formal ? `<p class="formal">${escapeHtml(e.formal
    ${e.complexity ? `<p class="cx">Сложность: ${escapeHt
    </div>`;
    })
    .join("\n");

    return `<section class="export-gloss">
    <h2>Словарик терминов из этой темы</h2>
    <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
    ны сюда.</p>
    ${items}
    </section>`;
  }

function wrap(opts: { title: string; subtitle: string;
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init

```

```

    a.remove();
    setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» — без кириллицы и
function safeName(title: string, id: string): string {
    const num = title.match(/^\\s*(\\d+)/)?.[1];
    const base = id.replace(/^[^a-z0-9-]+/gi, "-").toLowerCase();
    return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter): Promise<string> {
    const css = await collectCss();
    const { body, termIds } = prepareBody(chapter.content);
    const html = wrap({
        title: chapter.title,
        subtitle: chapter.category || "Универсальное пособие",
        css,
        body,
        gloss: glossarySection(termIds),
        origin: window.location.origin,
    });
    triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением.
export async function downloadBookHtml(chapters: Chapter[]): Promise<string> {
    const css = await collectCss();
    const allTerms: string[] = [];
    const bodies: string[] = [];

    const toc = chapters
        .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}>${c.title}</a>`
        .join("\\n");

    chapters.forEach((c) => {
        const { body, termIds } = prepareBody(c.content);

```

```

-----
*
*   node scripts/audit-coverage.mjs          # отчёт
*   node scripts/audit-coverage.mjs --md     # markd
*/
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")],
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/registry.json"), "utf8");
const registryBody = registry.slice(registry.indexOf("Vizualizatory"), registry.lastIndexOf("}"));
const vizIds = new Set([...registryBody.matchAll(/^\s{2}[a-zA-Z0-9_-]+/g)].map(s => s.slice(2)));

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1])
  };
});

```

```
-----  
    dump("НЕТ 2D (аналогия есть)", gapViz);  
    console.log(`Пропущенные номера билетов 1-24: ${missi  
    console.log(`Визуализаторы без главы: ${orphanViz.joi  
}
```

ФАЙЛ 84/96: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```
-----  
  
/**  
 * Проверка покрытия глав всплывающими подсказками:  
 * в каждой ли теме находятся термины из глоссария.  
 *  
 *   node scripts/audit-terms.mjs  
 */  
import { build } from "esbuild";  
import path from "node:path";  
  
const ROOT = process.cwd();  
await build({  
  entryPoints: ["src/data/content.ts", "src/data/glossa  
  bundle: true, format: "esm", platform: "node",  
  outdir: "tmp/audit-terms", outExtension: { ".js": ".m  
  external: ["react", "react-dom", "motion", "lucide-re  
});  
  
const { chapters } = await import(path.join(ROOT, "tmp/  
const { GLOSSARY_LABELS, GLOSSARY } = await import(path  
  
const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");  
const re = new RegExp(  
  `(?!<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e  
  "giu"  
);  
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label
```

```
-----  
console.log("Терминов в глоссарии:", GLOSSARY.length, "  
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

```
-----  
ФАЙЛ 85/96: scripts/export/build-guide-pdf.mjs
```

```
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОВОК: 28 | РАЗМЕР: 1.1 КБ  
-----
```

```
/**  
 * Оркестратор: бандлит TSX-генератор и запускает его.  
 * Использование: node scripts/export/build-guide-pdf.mjs  
 */  
import { build } from "esbuild";  
import { pathToFileURL } from "node:url";  
import path from "node:path";  
import fs from "node:fs";  
  
const root = path.resolve(import.meta.dirname, "../..");  
const outfile = path.join(root, "tmp/guide-pdf-bundle.cjs");  
  
await build({  
  entryPoints: [path.join(root, "scripts/export/guide-pdf.mjs")],  
  outfile,  
  bundle: true,  
  platform: "node",  
  format: "cjs",  
  target: "node20",  
  jsx: "automatic",  
  loader: { ".jpg": "dataurl", ".png": "dataurl", ".css": "text" },  
  external: ["pdfkit", "svg-to-pdfkit", "pdfkit/js/pdfkit"],  
  logLevel: "warning",  
});  
  
const outArg = process.argv[2];  
await import(pathToFileURL(outfile).href);
```

```
Title: "Универсальное пособие – полный исходный код"
Author: "CI: rebuild-pdf",
Subject: "Экспорт всего кода проекта (код -> txt)",
CreationDate: new Date(),
  },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
  doc.addPage({ size: [A4.width, A4.height], margin: 0 });
  doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
  stream.on("finish", resolve);
  stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

```
/** Порядок разделов в документе. */
const ORDER = [
  "Конфигурация проекта",
  "Ядро приложения",
  "Контент и данные пособия",
  "Билеты (учебный контент)",
  "Интерактивные компоненты и симуляторы",
  "Визуализаторы (вложенные)",
  "HTML-разметка (layout)",
  "Утилиты",
  "Скрипты сборки и экспорта",
  "CI / автоматизация",
  "Прочее",
];

function listFiles() {
  let files;
  try {
    files = execFileSync("git", ["ls-files", "--cached"], {
      cwd: ROOT,
      encoding: "utf8",
    })
      .split("\n")
      .filter(Boolean);
  } catch {
    files = [];
    const walk = (dir) => {
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
        const abs = path.join(dir, e.name);
        const rel = path.relative(ROOT, abs).split(path.sep);
        if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
        if (e.isDirectory()) walk(abs);
        else files.push(rel);
      }
    };
    walk(ROOT);
  }
}
```



```

ensureDir(OUT_DIR);

const out = [];
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { timeZoned: true });
const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

push(HR);
push("                УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
push("                ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ ИСКЛЮЧЕНЫ)");
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:          ${files.length}`);
push(`Суммарный объём кода:             ${humanSize(totalBytes)}`);
push(`Глав/билетов в пособии:           ${chapters.length}`);
push();

push(HR);
push("                ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
  push(`    ${String(i + 1).padStart(3, " ")} ${c.title}`);
});
push();

push(HR);
push("                ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
  const cat = categoryOf(rel);
  if (cat !== current) {
    current = cat;
    push();
    push(`    # ${cat}`);
  }
});

```

```
-----  
    console.log(`        выход:    ${path.relative(ROOT, TXT_PATH)}`  
  )  
}
```

```
main();  
  
-----
```

ФАЙЛ 88/96: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

```
-----  
/**  
 * Общие настройки и утилиты пайплайна экспорта.  
 *  
 * код   -> full_code_all_pages.txt    (collect-code.mjs)  
 * txt   -> page-XXXX.png              (render-pages.mjs)  
 * png   -> full_code_all_pages.pdf    (build-pdf.mjs)  
 */
```

```
import fs from "node:fs";  
import path from "node:path";  
import { fileURLToPath } from "node:url";  
import { execFileSync } from "node:child_process";
```

```
export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),  
  ..
```

```
/** Куда кладём финальные артефакты (эта папка отдаётся клиенту)  
export const OUT_DIR = path.join(ROOT, "public", "export");  
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают)  
export const PAGES_DIR = path.join(ROOT, "tmp", "export");
```

```
export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");  
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");
```

```
/** Параметры вёрстки страницы (подобраны под A4 @150dpi)  
export const PAGE = {
```

```
  /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100  
  density: Number(process.env.EXPORT_DENSITY ?? 150),  
  /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_COLOR=1  
  color: Boolean(process.env.EXPORT_COLOR ?? false),  
};
```

ФАЙЛ 89/96: scripts/export/guide-pdf-entry.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 673 | РАЗМЕР: 10.5 КБ

```
/**
 * Автосборка PDF-конспекта `guide_all_topics.pdf`.
 *
 * Что внутри:
 * * весь ТЕКСТ страниц пособия (больше текста, чем HTML)
 * * картинки визуализаций – React-компоненты рендерят
 *   (renderToStaticMarkup), из разметки берутся SVG-снимки
 * * НИКАКОГО кода компилятора и интерфейсных кнопок.
 *
 * Результат: public/export/guide_all_topics.pdf, ~40–70 КБ
 */
```

```
import * as fs from "node:fs";
import * as path from "node:path";
import { createElement } from "react";
import { renderToStaticMarkup } from "react-dom/server";
import PDFDocument from "pdfkit";
// @ts-ignore - у пакета нет типов
import SVGtoPDF from "svg-to-pdfkit/source.js";
import { chapters } from "../../src/data/content";
import { VizChapterContext } from "../../src/data/vizStamps";
import { VIZ_REGISTRY, getViz } from "../../src/components/vizRegistry";
```

```
// ----- шрифты -----
```

```
const FONT_DIRS = ["/usr/share/fonts/truetype/dejavu",
function findFont(file: string): string | null {
  for (const dir of FONT_DIRS) {
    const p = path.join(dir, file);
    if (fs.existsSync(p)) return p;
  }
}
```

```

        .replace(/<\/(p|div|li|h[1-6]|tr|summary)>/gi, "\n")
        .replace(/<[^>]+>/g, " ")
    )
    .replace(/[ \t]+/g, " ")
    .replace(/ *\\n */g, "\\n")
    .replace(/\\n{2,}/g, "\\n")
    .trim();
}

```

/** Найти закрывающий тег с учётом вложенности однотипных

```

function findClose(html: string, openEnd: number, tag: string): number {
    const re = new RegExp(`<\\/?${tag}(\\s[^\>]*)?>`, "gi");
    re.lastIndex = openEnd;
    let depth = 1;
    let m: RegExpExecArray | null;
    while ((m = re.exec(html))) {
        depth += m[0][1] === "/" ? -1 : 1;
        if (depth === 0) return m.index;
    }
    return html.length;
}

```

```

function findCloseAny(html: string, openEnd: number, tag: string): { end: number; inner: string } {
    const close = html.toLowerCase().indexOf(`</${tag}`);
    if (close < 0) return { end: html.length, inner: html.slice(openEnd, html.length) };
    const inner = html.slice(openEnd, close);
    const end = html.indexOf(">", close) + 1;
    return { end, inner };
}

```

/** Разобрать HTML главы на учебные блоки (текст → 90%,

```

export function htmlToBlocks(html: string): Block[] {
    const blocks: Block[] = [];
    let i = 0;
    const skipTo = ["<pre", "<h2", "<h3", "<h4", "<summary"];
    while (i < html.length) {
        const lower = html.toLowerCase();
        const nexts = skipTo.map((t) => lower.indexOf(t, i));
    }
}

```

```

        const detEnd = html.toLowerCase().indexOf("</details>");
        if (detEnd >= 0) {
            blocks.push(...htmlToBlocks(html.slice(end, detEnd));
            i = detEnd + "</details>".length;
        } else i = end;
        continue;
    }
    if (lower.startsWith("<li", at) || (liM && at === liM)) {
        const openEnd = html.indexOf(">", at) + 1;
        const close = html.toLowerCase().indexOf("</li>", at);
        const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
        if (text) blocks.push({ kind: "li", text });
        i = close < 0 ? html.length : close + 5;
        continue;
    }
    if (lower.startsWith("<tr", at)) {
        const openEnd = html.indexOf(">", at) + 1;
        const close = html.toLowerCase().indexOf("</tr>", at);
        const rowHtml = html.slice(openEnd, close < 0 ? html.length : close);
        const cells: string[] = [];
        const tdRe = /<(t[dh])(\s[^>]*)?>([\s\S]*?)<\/\1>/g;
        let tm: RegExpExecArray | null;
        while ((tm = tdRe.exec(rowHtml))) cells.push(stripTags(tm[3]));
        if (cells.some(Boolean)) blocks.push({ kind: "table", cells });
        i = close < 0 ? html.length : close + 5;
        continue;
    }
    // <p>
    const openEnd = html.indexOf(">", at) + 1;
    const close = html.toLowerCase().indexOf("</p>", at);
    const text = stripTags(html.slice(openEnd, close < 0 ? html.length : close));
    if (text) blocks.push({ kind: "p", text });
    i = close < 0 ? html.length : close + 4;
}
return blocks;
}

```



```

        if (F_REG) this.doc.registerFont("Body", F_REG);
        if (F_BOLD) this.doc.registerFont("BodyBold", F_BOLD);
        if (F_MONO) this.doc.registerFont("Mono", F_MONO);
        if (F_MONO) this.doc.registerFont("MonoBold", F_MONO);
    }
    bodyFont() { this.doc.font(F_REG ? "Body" : "Helvetica"); }
    boldFont() { this.doc.font(F_BOLD ? "BodyBold" : "HelveticaBold"); }
    monoFont() { this.doc.font(F_MONO ? "Mono" : "Courier"); }

    get y() { return this.doc.y; }
    set y(v: number) { this.doc.y = v; }
    get pageBottom() { return A4[1] - M.bottom; }
    ensureSpace(need: number) {
        if (this.y + need > this.pageBottom) this.doc.addPage();
    }
    footerAll(skipCover = true) {
        const range = this.doc.bufferedPageRange();
        const oldBottom = this.doc.page.margins.bottom;
        for (let p = range.start; p < range.start + range.count; p++) {
            if (skipCover && p === range.start) continue;
            this.doc.switchToPage(p);
            this.doc.page.margins.bottom = 0;
            this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(10)
                .text(String(p + 1), 0, A4[1] - 30, { width: A4[0] });
            this.doc.page.margins.bottom = oldBottom;
        }
    }
}

```

```

const CH_LINE = 13.4; // межстрочный в теле
const BODY_SIZE = 9.6;

```

```

function drawTextBlock(d: Doc, text: string, opts: { size?: number,
    boolean } = {}) {
    const size = opts.size ?? BODY_SIZE;
    const lead = size * 1.42;
    d.ensureSpace(lead * 2);
    d.doc.fontSize(size)

```

```

    drawPortion(from, to, yTop);
    from = to;
    if (from < lines.length) { d.doc.addPage(); yTop = 1; }
    d.y = yTop + Math.min(lines.length, Math.floor((d.page - 1) / 3));
    d.doc.moveDown(3);
}

```

```

function drawTableBlock(d: Doc, rows: string[]) {
    const size = 8.4;
    d.ensureSpace(size * 3);
    const text = rows.map((c) => c.replace(/\s*\n\s*/g, " "));
    d.doc.fontSize(size).fillColor("#374151").text(text, 1);
    d.doc.moveDown(2.2);
}

```

```

function drawGrid(d: Doc, rows: string[]) {
    const size = 8;
    const lead = size * 1.6;
    d.ensureSpace(rows.length * lead + 16);
    const top = d.y;
    d.monoFont().fontSize(size).fillColor("#111827");
    const cellW = Math.min(
        34,
        (CONTENT_W - 16) / Math.max(...rows.map((r) => r.split(" ").length))
    );
    rows.forEach((row, ri) => {
        const cells = row.split(" ");
        cells.forEach((cval, ci) => {
            const x = M.left + 6 + ci * cellW;
            const y = top + 4 + ri * lead;
            d.doc.save().rect(x - 2, y - 2, cellW - 4, lead - 2);
            d.doc.text(cval, x + 2, y + lead * 0.12, { width: cellW });
        });
    });
    d.y = top + 8 + rows.length * lead + 8;
    d.doc.moveDown(2);
}

```



```

    if (avail < 200) d.doc.addPage();
    try {
      d.doc.image(buf, M.left + (CONTENT_W - maxW) / 2, d
    } catch { return; }
    d.y = d.y + 10 + maxW * 0.6; // примерная высота фото
    d.doc.moveDown(2);
  }

// ----- сборка -----

const FRONT_TOC_LINES = 44; // строк содержания на стра

function tocEntries() {
  return chapters.map((c) => ({ id: c.id, title: c.titl
})

function tocPagesNeeded(): number {
  return Math.max(1, Math.ceil(tocEntries().length / FR
})

interface BuildOpts {
  outputPath: string;
  record: boolean;
  /** id → стартовая страница (для второго прохода) */
  pageMap?: Map<string, number>;
  frontPages: number;
}

function buildOne(opts: BuildOpts): Stats {
  return new Promise<Stats>((resolve, reject) => {
    const stream = opts.record ? fs.createWriteStream("
    const d = new Doc(stream as unknown as fs.WriteStre
    const doc = d.doc;

    const finish = (stats: Stats) => {
      d.footerAll(!opts.record);
      doc.end();
      stream.on("close", () => resolve(stats));
    }
  });
}

```

```

const pageNo2 = (opts.pageMap?.get(ch.id) ?? 0)
const isTopic = /^\\d/.test(ch.title);
doc.fontSize(isTopic ? 10.5 : 10).font(isTopic ?
    .fillColor(isTopic ? "#1f2937" : "#6b7280"));
// заголовок + точечная отбивка + номер страницы
const title = ch.title.length > 86 ? ch.title.s
const w = CONTENT_W - 40;
const tw = doc.widthOfString(title);
const pw = doc.widthOfString(String(pageNo2));
doc.text(title, M.left, d.y, { width: w, lineBr
const dots = ".".repeat(Math.max(3, Math.floor(
doc.fillColor("#d1d5db").text(dots, M.left + tw
doc.fillColor("#111827").text(String(pageNo2), M
d.y += 16.2;
line++;
});
// добиваем до расчётного числа страниц содержания
const used = Math.ceil(entries.length / FRONT_TOC
for (let p = used; p < opts.frontPages - 1; p++)
doc.addPage();
if (!opts.record) console.log(`TOC: ${entries.leng
}

// — главы —
const stats: Stats = { chapterStart: new Map(), tot

chapters.forEach((ch, ci) => {
  const startPageOf = () => {
    // текущая страница = pages (0-based) + 1
    return doc.bufferedPageRange().start + doc.buff
  };
  if (ci > 0) doc.addPage();
  const pageNow = opts.record ? doc.bufferedPageRan
  d.chapterStart.set(ch.id, pageNow);
  if (!opts.record && ci < 3) console.log(`глава ${
  d.bodyFont();
  doc.y = M.top;
  d.y = M.top;

```

```

        console.log(` viz ${ch.id}: svg=${art?.svgs.
            for ? " ОШИБКА: " + art.error : ""}`);
        if (n === 0) console.log(` ▲ ${ch.id}: визуа
    }
    if (art && (art.svgs.length || art.images.length
        d.ensureSpace(60);
        doc.moveDown(0.6);
        doc.fontSize(10).font(F_BOLD ? "BodyBold" : "
            .text("■ Визуализация: " + vizEntry.title,
        d.y = doc.y + 6;
        let drawn = 0;
        for (const img of art.images) {
            if (drawn >= 4) break;
            drawImage(d, img);
            drawn++;
        }
        for (const svg of art.svgs) {
            if (drawn >= 5) break;
            drawSvg(d, svg);
            drawn++;
        }
        for (const grid of art.grids) {
            if (drawn >= 7) break;
            drawGrid(d, grid);
            drawn++;
        }
    } else if (art?.error) {
        // SSR упал – рисуем текстовую заглушку, PDF
    }
}
});

stats.chapterStart = d.chapterStart;
stats.total = doc.bufferedPageRange().count;
finish(stats);
});
}

```

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
  const expanded = line.replace(/\t/g, " ").replace(/<\/p>
  if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = " ".repeat(Math.min((expanded.match(/<\/p>
  let rest = expanded;
  let first = true;
  while (rest.length > 0) {
    const width = first ? PAGE.cols : PAGE.cols - indent;
    parts.push((first ? " " : indent) + rest.slice(0, width));
    rest = rest.slice(width);
    first = false;
  }
  return parts;
}
```

```

    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
    "-pointsize", String(PAGE.pointsize),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
    "-colorspace", "Gray",
    ...(PAGE.monochrome ? ["-monochrome"] : []),
    "-strip",
    "-define", "png:compression-level=9",
    job.pngFile,
  ];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`      отрендерено ${done}\n`);
    }
  }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.pngFile).join(" ")}

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`      страниц: ${total}`);
console.log(`      выход:   ${path.relative(ROOT, PAGE.pngFile)}`);

```

```

-----
const kmp = init("string-kmp");
ok(kmp.includes('pattern = "aba"') && kmp.includes('s =

// 4. Форду-Беллману – рёбра реального графа (S→A = 4,
const bf = PAGE_SYNC["bellman-ford"].code ?? "";
ok(bf.includes('("S", "A", 4)') && bf.includes('("D", "
ok(!bf.includes('("S", "A", 5)'), "bellman-ford: заглущ
ok(!/break/.test(bf), "bellman-ford: без break");

// 5. Джонсон: h(B) = -2, ребро -2, w' = 0
const jh = PAGE_SYNC["johnson-algo"].code ?? "";
ok(jh.includes('"B": -2') && jh.includes("w_new = w + h

// 6. Краскал/Прим/Борувка: 9 вершин и вес 51
for (const id of ["mst-kruskal", "mst-boruvka"]) {
  const c = PAGE_SYNC[id].code ?? "";
  ok(/ABCDEFGHI/.test(c), `${id}: 9 вершин A..I`);
}
ok((PAGE_SYNC["mst-prima"].code ?? "").match(/"I":/g)?
ok((PAGE_SYNC["mst-kruskal"].code ?? "").includes('{13,

// 7. prefix-sums-2d#2d: пометка 1-индексных клеток
const p2 = PAGE_SYNC["prefix-sums-2d#2d"].code ?? "";
ok(p2.includes("клетки 1-индексные"), "prefix#2d: комме

// 8. графы top-sort/scc/graph-dfs-bfs/graph-components
ok((PAGE_SYNC["top-sort"].code ?? "").includes("0: [1,
ok((PAGE_SYNC["scc-kosaraju"].code ?? "").includes("2:
  емо");
ok((PAGE_SYNC["graph-dfs-bfs"].code ?? "").includes('ad
ok((PAGE_SYNC["graph-components"].code ?? "").includes(

// 9. каждый шаблон непуст и у каждого id есть variable
for (const [id, sync] of Object.entries(PAGE_SYNC)) {
  if (sync.code) {
    ok(sync.code.trim().length > 0, `${id}: код непуст`
    ok(sync.variables.length > 0, `${id}: variables опи
  }
}

```

```

-----
// Самотест lazy-вкладки дерева отрезков: пять массивов
// Корень после обновления и query(uL,ur) = base(uL,ur)
import { genLazySteps, buildTreeFull } from "../src/

const base = [5, 8, 3, 12, 7, 2];
const sum = (l: number, r: number) => base.slice(l, r + 1).reduce((a, b) => a + b, 0);

const cases: [number, number, number][] = [
  [1, 3, 10], [0, 5, 1], [2, 2, 7], [0, 5, 100], [4, 5, 100],
];

// query по дереву с обещаниями: pending накапливается
function q(step: { treeState: Record<number, number>; lazyState: Record<number, number>; pending: number; }, l: number, r: number, ql: number, qr: number, d: number): number {
  if (qr < l || r < ql) return 0;
  // Собственный lazy[v] уже учтён в tree[v] (обещание)
  // добавляем только накопленные обещания предков (pending)
  if (ql <= l && r <= qr) return (step.treeState[v] ?? 0) + d;
  const m = (l + r) >> 1;
  const d = pending + (step.lazyState[v] ?? 0);
  return q(step, 2 * v, l, m, ql, qr, d) + q(step, 2 * v + 1, m, r, ql, qr, d);
}

let fails = 0;
for (const [uL, uR, add] of cases) {
  const steps = genLazySteps(base, uL, uR, add);
  const last = steps[steps.length - 1];
  const wantRoot = sum(0, 5) + add * (uR - uL + 1);
  const wantQ = sum(uL, uR) + add * (uR - uL + 1);
  const gotRoot = last.treeState[1];
  const gotQ = q(last, 1, 0, 5, uL, uR, 0);
  const okR = gotRoot === wantRoot, okQ = gotQ === wantQ;
  console.log(`(uL,ur,add)=(${uL},${uR},${add}) root=${gotRoot} want=${wantRoot} q=${gotQ} want=${wantQ} ${!okR || !okQ ? "FAIL" : "OK"} | шагов=${steps.length}`);
  if (!okR || !okQ) fails++;
}

```

```
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
deploy:
```


Универсальное пособие • полный исходный код

```
-----
#
#                                     └─ full_code_all_pages.pdf
#
#                                     └─ guide_all_topics.pdf
#                                     PDF-конспект: текст всех тем + с
#
# Готовые TXT и PDF коммитаются обратно в ветку (их отда
# промежуточные PNG сохраняются как artifacts запуска.

on:
  push:
    branches:
      - "**"
    paths-ignore:
      # чтобы коммит самого workflow не запускал бескон
      - "public/export/**"
      - "**/*.md"
  workflow_dispatch:
    inputs:
      density:
        description: "DPI растеризации страниц (по умол
        required: false
        default: "150"

concurrency:
  group: rebuild-pdf-`${github.ref}`
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[
```

- run: npm run export:pdf
- name: Step 4 – PDF-конспект (текст тем + визуал)
run: npm run export:guide
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full_code_all_pages
 echo "| \public/export/full_code_all_pages
 echo "| \public/export/guide_all_topics.pdf
 echo "Страниц в конспекте: \$(node -e "console.log(require('fs').readdirSync('tmp/export-pages').length)"
 echo "| PNG-страниц | \${ls tmp/export-pages
 } >> "\$GITHUB_STEP_SUMMARY"
- name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 public/export/guide_all_topics.pdf
 if-no-files-found: error
 retention-days: 30
- name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v7
with:
 name: full-code-pages-png